



BACHELOR THESIS

Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware

Studierender:
Nassim AKALAI
Matrikelnummer:
1456967

Erstprüfer:
Prof. Dr. Oliver Hahm

Zweitprüfer:
David Merkl

*Eingereicht zur Erlangung
des akademischen Grades des Bachelor of Science
an der*

Fachbereich 2
Informatik

21.09.2026

Eidesstattliche Erklärung

Ich, Nassim AKALAI, versichere, dass ich die vorliegende Bachelorarbeit mit dem Titel „**Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware**“ selbstständig und ohne unzulässige fremde Hilfe angefertigt habe. Alle verwendeten Quellen und Hilfsmittel sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Nassim Akalai
Frankfurt am Main, 21.09.2026

Zusammenfassung

Bachelor of Science

Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware

von Nassim AKALAI

Zusammenfassung

Diese Arbeit untersucht, wie sich verschiedene Konfigurationen des Echtzeitbetriebssystems Zephyr auf dessen Laufzeitverhalten und Speicherbedarf auf ressourcenbeschränkter Hardware auswirken. Als Zielplattform dient das nRF52840 Development Kit mit einem Arm-Cortex-M4F-Prozessor. Verglichen werden eine Minimal-Konfiguration, eine darauf aufbauende Konfiguration mit aktiviertem Logging-Subsystem und eine IoT-Konfiguration mit OpenThread, CoAP und DTLS. Zur reproduzierbaren Leistungsbewertung werden Kontextwechsel-Latenz, Interrupt-Latenz, Timer-Jitter, Heap-Allokationsgeschwindigkeit, Heap-Fragmentierung und der statische RAM- und ROM-Bedarf untersucht. Kurze Laufzeitvorgänge werden mit dem DWT-Zyklenzähler erfasst, während für die periodische Timer-Messung der RTC-basierte Systemtimer benutzt wird. Minimum, Maximum, Mittelwert und Stichprobenstandardabweichung werden mit dem Welford-Algorithmus laufend berechnet. Getrennte Firmware-Abbilder für jede Kombination aus Konfiguration und Benchmark begrenzen dabei die gegenseitige Beeinflussung der Messungen. Die entwickelte Messumgebung stellt ein nachvollziehbares Verfahren bereit, mit dem sich der Einfluss von ausgewählten Zephyr-Subsystemen auf derselben Hardwareplattform vergleichen lässt. Darüber hinaus demonstriert eine praxisnahe Sensoranwendung die Verwendung von CoAP über DTLS und einen Firmware-Aktualisierungspfad mit MCUboot und MCUmgr.

FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Zusammenfassung

Cluster Informatik
Fachbereich 2

Bachelor of Science

Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware

von Nassim AKALAI

This thesis investigates how different configurations of the Zephyr real-time operating system affect runtime behavior and memory requirements on resource-constrained hardware. The target platform is the nRF52840 Development Kit with an Arm Cortex-M4F processor. Three configurations are compared: a minimal configuration, a configuration derived from it with the logging subsystem enabled, and an IoT configuration which implements OpenThread, CoAP, and DTLS. The reproducible performance evaluation covers context-switch latency, interrupt latency, timer jitter, heap allocation time, heap fragmentation and static RAM and ROM usage. Short runtime operations are measured using the DWT cycle counter, while periodic timer measurements use the RTC-based system timer. Minimum, maximum, mean, and sample standard deviation are calculated online using Welford's algorithm. Separate firmware images for every combination of configuration and benchmark limit interference between measurements. The resulting measurement environment provides a traceable method for comparing the impact of selected Zephyr subsystems on the same hardware platform. In addition, a practical sensor application demonstrates CoAP over DTLS and a firmware update path based on MCUboot and MCUmgr.

Inhaltsverzeichnis

Eidesstattliche Erklärung	I
Zusammenfassung	II
Abstract	III
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung und Forschungsfrage	2
1.3 Zielsetzung und Beitrag	2
1.4 Aufbau der Arbeit	3
2 Grundlagen	7
2.1 Systeme und zeitliche Anforderungen	7
2.2 Real Time Systems	9
2.3 Eingebettete Systeme	9
2.4 Echtzeitbetriebssysteme	11
2.5 Zephyr RTOS	12
2.5.1 Laufzeit-Konzepte	13
Threads und Scheduling	14
Synchronisation (Semaphore)	15
Interrupts	16
Timer und Tickless-Kernel	17
Speicherverwaltung	18
Logging-Subsystem	20
Timing-API	21
2.5.2 Build-Zeit-Konzepte	22
Kconfig	23
CMake und Zephyr-Module	24
West	25
Build-System-Werkzeuge zur Analyse des Speicherbe- darfs	26
2.6 Benchmark Metriken	26
2.6.1 Context Switch	27
2.6.2 Interrupt Latency	27
2.6.3 Timer-Jitter	27
2.6.4 Heap-Allokation/ Fragmentierung	28
2.6.5 RAM/ ROM	28
2.7 Netzwerk- und Sicherheitskonzepte	29
2.7.1 Netzwerk-Stack und CoAP	29

2.7.2	Transportverschlüsselung durch TLS/ DTLS und Pre-Shared Keys	30
2.7.3	Thread, 6LoWPAN und IEEE 802.15.4	30
2.7.4	Firmware-Updates: MCUmgr und MCUboot	31
3	Verwandte Arbeiten	32
3.1	RTOS-Benchmarking allgemein	32
3.2	Bestehende RTOS-Benchmarks	33
3.3	Zephyr-spezifische Benchmarks	34
3.4	Abgrenzung der eigenen Arbeit	37
4	Methodik und Versuchsaufbau	38
4.1	Forschungsdesign	38
4.2	Zielplattform	39
4.3	Softwareumgebung	40
4.4	Unabhängige Variablen	40
4.4.1	Minimal Konfiguration	40
4.4.2	Logging	41
4.4.3	IoT-Stack	41
4.5	Abhängige Variablen	41
4.5.1	Context-Switch-Latenz	41
4.5.2	Interrupt-Latenz	42
4.5.3	Timer-Jitter	42
4.5.4	Heap-Allokationsgeschwindigkeit	42
4.5.5	Heap-Fragmentierung	42
4.6	Kontrollierte Variablen	43
4.6.1	Hardware/ CPU-Frequenz	43
4.6.2	Compiler und Werkzeugkette	43
4.6.3	Compiler optimierung	43
4.6.4	Zephyr Version	43
4.6.5	Build-Konfiguration (Sockel)	44
4.6.6	Messmechanismus (Timing/ Statistik)	44
4.6.7	Anzahl der Messungen	44
4.7	Ablauf einer Messung	44
5	Benchmark-Design	48
5.1	Kontextwechsel-Latenz	48
5.2	Interrupt-Latenz	49
5.3	Timer-Jitter	50
5.4	Heap-Allokationsgeschwindigkeit	51
5.5	Heap-Fragmentierung	52
6	Implementierung	54
6.1	Projektstruktur	54
6.2	Benchmark-Infrastruktur	55
6.2.1	Warum eine eigene Bibliothek?	55
6.2.2	Die öffentliche Schnittstelle	56
6.2.3	Zeitbasis und Kalibrierung des Messaufwands	58

6.2.4	Laufende Statistik	58
6.2.5	Ergebnisausgabe als CSV	60
6.2.6	Parametrisierung über Kconfig	61
6.3	Konfigurationen: Minimal / Logging / (IoT-Stack)	62
6.3.1	Gemeinsame Basis	62
6.3.2	Minimal-Konfiguration	63
6.3.3	Logging-Konfiguration	65
6.3.4	IoT-Stack-Konfiguration	67
6.4	Realistische Anwendung	71
6.4.1	Zweck und Sensor-Simulation	71
6.4.2	CoAP-Server über DTLS/ PSK	71
6.4.3	Firmware-Update über Sysbuild, MCUboot und MCUmgr	72
7	Messergebnisse	73
8	Diskussion	74
9	Validität	75
9.1	Interne Validität	75
9.2	Externe Validität	75
9.3	Messbedingter Zusatzaufwand	76
9.4	Limitierungen	76
10	Fazit	77
11	Ausblick	78

Abbildungsverzeichnis

Einleitung

Das Internet der Dinge (Internet of Things, IoT) beschreibt die zunehmende Vernetzung von Alltagsgeräten und industriellen Sensoren zu einer gemeinsamen Kommunikations-Infrastruktur, in welcher Menschen, Geräte und Prozesse jederzeit Daten untereinander austauschen können. Viele dieser Geräte befinden sich hierbei jedoch nicht im Kern von leistungsstarken Rechenzentren, sondern am äußersten Rand des Netzwerks. Sie befinden sich also da, wo Sensorik und Aktorik direkt auf die physische Umgebung treffen. An der Netzwerkkante werden oft batteriebetriebene Geräte mit begrenzter Rechen- und Speicherkapazität eingesetzt. Als eingebettete Systeme gelten sie jedoch nicht nur wegen dieser Beschränkungen, sondern weil ihre Hardware und Software für eine bestimmte Aufgabe oder einen begrenzten Funktionsumfang gemacht sind (Silva et al., 2019, S. 10375; Das, 2013, S. 34).

Einfache eingebettete Anwendungen können ohne Betriebssystem als fortlaufende Hauptschleife mit erweiternden Interrupt-Routinen umgesetzt werden. Wenn jedoch mehrere Aufgaben, Kommunikationswege und gemeinsam genutzte Ressourcen dabei koordiniert werden müssen, kann ein eingebettetes Betriebssystem die Verwaltungs- und Abstraktionsdienste bereitstellen, die dafür benötigt werden (Das, 2013, S. 296–297; Silva et al., 2019, S. 10375).

Hier kommen Echtzeitbetriebssysteme (Real-Time Operating Systems, RTOS) ins Spiel. Im Gegensatz zu normalen Allzweck-Betriebssystemen kommt es hierbei nicht auf einen hohen Durchsatz an, sondern darauf, dass zeitkritische Prozesse ihre Reaktionszeit zuverlässig und vorhersagbar einhalten. (Altoumi & Ahmed, 2025, S.1) Ein quelloffener Vertreter ist das Zephyr RTOS, welches speziell für ressourcenbeschränkte, vernetzte eingebettete Systeme (Embedded Systems) konzipiert wurde und über eine modulare Architektur verfügt.

1.1 Motivation

Die Skalierbarkeit und Konfigurierbarkeit von modernen OS wie Zephyr bieten Softwareentwicklern eine hohe Flexibilität. Auf der anderen Seite stehen sie jedoch vor einem Dilemma: Die Aktivierung weiterer Systemdienste, Netzwerkprotokolle oder Schutzmechanismen kann den RAM- und Flash-Bedarf erhöhen und gleichzeitig das Laufzeitverhalten beeinflussen. Wie groß dieser Einfluss ist, hängt von der jeweiligen Konfiguration, den aktiven Hintergrundfunktionen und dem erzeugten Firmware-Abbild ab (Silva et al., 2019, S. 10375–10376).

Vor allem bei ressourcenbeschränkten Geräten können kleine Anpassungen in der Systemkonfiguration darüber entscheiden, ob eine Anwendung ihre vorgegebenen Fristen einhält oder der physisch vorhandene Speicher überschritten wird. Gerade im Kontext zu eingebetteten Systemen spielt der Determinismus eine wichtige Rolle, da das Verpassen der Fristen zu Konsequenzen führen kann. Daher ist das zeitliche Verhalten eines RTOS wichtig.

Die Leistung eines RTOS hängt daher davon ab, wie die Parameter und die aktivierten Module gewählt sind. Daher ist es besonders wichtig in der Praxis, die genauen Auswirkungen und Einflüsse verschiedener Konfigurationen bzgl. der Systemleistung (empirisch) zu messen und die Ergebnisse zu verstehen.

1.2 Problemstellung und Forschungsfrage

Obwohl Zephyr ein weitverbreitetes RTOS ist und für eine Vielzahl von Hardware-Plattformen konzipiert ist, liefert die hier verwendete Literatur nur begrenzte Aussagen über den Einfluss von Zephyr-Konfigurationen auf derselben Hardwareplattform. Das Problem liegt darin, dass die Abhängigkeiten zwischen den aktivierten Softwaremodulen, wie bspw. Logging, und dem daraus entstehenden Ressourcenverbrauch auf realer Hardware nur schwer abschätzbar ist.

Aus dieser Herausforderung leiten sich der Titel der Arbeit, *„Leistungsbewertung eines Echtzeitbetriebssystems auf ressourcenbeschränkter Hardware“* und die Forschungsfrage ab:

Wie verhalten sich Laufzeitverhalten und Speicherverbrauch von Zephyr RTOS in Abhängigkeit von der Konfiguration auf ressourcenbeschränkter Hardware?

Aus dieser Frage leiten sich zudem folgende Unterfragen ab:

- *Wie verändert sich die Context-Switch-Latenz?*
- *Wie verändert sich die Interrupt-Latenz?*
- *Wie verändert sich das Timer-Jitter?*
- *Wie beeinflussen die Konfigurationen Heap-Allokation und Speicherfragmentierung?*
- *Wie beeinflussen die Konfigurationen den Speicherbedarf (RAM/ ROM)?*

1.3 Zielsetzung und Beitrag

Das Ziel dieser Arbeit ist es, ein systematisches und reproduzierbares Leistungsbewertungsverfahren (Benchmark) für das Zephyr RTOS auf einer ressourcenbeschränkten Zielplattform durchzuführen und zu evaluieren. Dabei werden drei verschiedene praxisnahe Konfigurationsszenarien definiert. Diese werden dann mit unterschiedlichen Benchmarks miteinander verglichen.

Die Konfigurationen umfassen:

- **Minimal:** Das absolute Minimum an Funktionen und Funktionen, die benötigt werden, um alle Benchmarks auf dem verwendeten Mikrocontroller durchführen und messen zu können.
- **Logging:** Die Minimal-Konfiguration in Verbindung mit dem Logging-Subsystem, welches aktiviert wird. Diese Konfiguration dient dazu, den Einfluss von oft benutzten Systemfunktionen zu veranschaulichen.
- **IoT-Stack:** Die Benchmark-Konfiguration erweitert die Minimal-Konfiguration um OpenThread, CoAP und DTLS/ PSK. Firmware-Updates über MCUmgr und MCUboot werden davon getrennt in der Demonstrationsanwendung `iot_sensor` untersucht.

Im Fokus der beschriebenen Untersuchung stehen zudem zwei Kernbereiche:

- **Speicherverbrauch:** Die Messung, wie sich die verschiedenen Konfigurationen auf den statischen und dynamischen Speicherbedarf (RAM und ROM) auswirken.
- **Laufzeitverhalten:** Die Analyse von Echtzeit-Metriken, wie der Dauer von Kontextwechseln und der Latenz bei der Verarbeitung von Interrupts.

Der Beitrag dieser Arbeit liegt darin, Metriken zu messen, um somit die Mechanismen des Systems anhand von empirischen Messdaten aufzuzeigen und zu evaluieren. Die daraus resultierenden Erkenntnisse sollen Entwicklern eine Hilfestellung und Entscheidungsgrundlage bieten, um Zephyr für zukünftige IoT-Anwendungen ressourcenschonend und optimal zu konfigurieren. Die dokumentierte Methodik kann außerdem als Ausgangspunkt für spätere RTOS-Vergleiche dienen. Ein direkter Zahlenvergleich mit RIOT oder FreeRTOS wäre jedoch nur bei vergleichbarer Hardware, Anwendung, Konfiguration, Werkzeugkette und Messpunktdefinition belastbar (Silva et al., 2019, S. 10378–10381).

1.4 Aufbau der Arbeit

Die vorliegende Arbeit ist in mehrere, aufeinander aufbauende Kapitel unterteilt. Zuerst werden die relevanten theoretischen und technischen Grundlagen und der aktuelle Stand der Forschung dargestellt. Aufbauend darauf werden die Methodik, der Versuchsaufbau und das Benchmark-Design beschrieben. Anschließend werden die Ergebnisse diskutiert, die Validität der Untersuchung betrachtet und die Forschungsfrage, inklusive der Unterfragen beantwortet. Zuletzt werden ein Fazit und ein Ausblick auf mögliche weitere Untersuchungen gebildet.

Kapitel 1 Das erste Kapitel bildet die Einleitung der Arbeit und führt in die Thematik ein. Dabei werden Konzepte des IoT und von ressourcenbeschränkten Systemen eingeführt. Dabei werden die besonderen Anforderungen solcher Systeme angesprochen und die Grenzen von Bare-Metal-Programmierung und klassischen Betriebssystemen erläutert. Anschließend

werden Echtzeitbetriebssysteme vorgestellt, wobei Zephyr RTOS in dieser Untersuchung verwendet wird.

Kapitel 2 Im zweiten Kapitel werden die für die Arbeit relevanten Konzepte von Systemen und deren zeitlichen Anforderungen eingeführt. Dabei werden Begriffe wie Freigabezeitpunkt, Abschlusszeitpunkt, Antwortzeit, Frist und Fristüberschreitung definiert. Außerdem werden zeitliche Anforderungen und Determinismus erläutert. Anschließend werden Echtzeitsysteme und deren unterschiedlichen Varianten erläutert. Basierend darauf werden eingebettete Systeme, Mikrocontroller und eingebettete Betriebssysteme vorgestellt. Anschließend werden die Eigenschaften von Echtzeitbetriebssystemen erläutert. Demnach wird das für die Untersuchung verwendete Zephyr RTOS vorgestellt. Neben der Architektur und modularer Konfigurierbarkeit werden dabei die für die spätere Untersuchung relevanten Kernel-Dienste im Detail behandelt, darunter Threads und Scheduling, Synchronisation mittels Semaphore, Interrupts, Timer und das Konzept des tickless Kernels, die Speicherverwaltung über System- und eigene Heaps und das Logging-Subsystem. Dies wird zudem ergänzt um die Timing-API, mit der einige Laufzeitmessungen dieser Arbeit durchgeführt werden. Darüber hinaus wird das Zephyr-eigene Build- und Konfigurationssystem eingeführt, darunter das Kconfig-Konfigurationssystem, inklusive Kconfig-Fragmenten, der Modul-Mechanismus zur Einbindung eigener Bibliotheken über CMake, das Werkzeug West zur Verwaltung des Arbeitsbereichs (*workspace*) und die im Build-System enthaltenen Werkzeuge zur Analyse des Speicherbedarfs. Zuletzt werden die für die spätere Untersuchung verwendeten Benchmark-Metriken, darunter Context-Switch-Zeit, Interrupt-Latenz, Timer-Jitter, Speicherverbrauch und das Heap-Verhalten, konzeptionell eingeführt.

Kapitel 3 Im dritten Kapitel wird der aktuelle Stand der Forschung zum Thema RTOS-Benchmarking dargestellt und diese Arbeit darin eingeordnet. Zuerst wird erläutert, welche Probleme beim Vergleich und der Bewertung von Echtzeitsystemen entstehen, darunter Fragen zur Reproduzierbarkeit und Vergleichbarkeit mit anderen Betriebssystemen wie FreeRTOS, RIOT oder Contiki-NG. Anschließend werden bestehende RTOS-Vergleichsstudien vorgestellt, die Zephyr unter anderem bzgl. Speicherverbrauch, Zeit-Determinismus und Kontextwechsel-Latenz messen. Danach werden Zephyr-spezifische Arbeiten behandelt, darunter Zephyrs eigene Referenz-Benchmarks und Studien, die Zephyr in bestimmten Anwendungsszenarien einsetzen, bspw. zur Absicherung durch TLS/ DTLS oder zur KI-Inferenz. Zuletzt wird diese Arbeit von den anderen Studien abgegrenzt, indem herausgearbeitet wird, wie sich ihre Methodik von den anderen Studien unterscheidet.

Kapitel 4 Im vierten Kapitel werden die Methodik und der Versuchsaufbau der Untersuchung beschrieben. Zuerst wird das quantitative, experimentelle Forschungsdesign vorgestellt. Anschließend wird das Design anhand der unabhängigen Variable (der jeweils aktiven Zephyr-Konfiguration), der fünf Laufzeit- und Heap-Benchmarks und den dabei bestimmten statischen RAM-/ROM-Fußabdruck und der kontrollierten variablen erklärt. Dabei wird auch

die technische Isolation der einzelnen Messungen auf Ebene der Firmware-Builds und die kumulative Anlage der Konfigurationsstufen begründet. Anschließend werden die verwendete Zielplattform, das nRF52840 Development Kit mit seiner für die Untersuchung relevanten Interrupt- und Zeitgeberhardware und die eingesetzte Softwareumgebung beschrieben. Danach werden die drei Konfigurationen Minimal, Logging und IoT-Stack als unabhängige Variable und die fünf Benchmarks Context-Switch-Latenz, Interrupt-Latenz, Timer-Jitter, Heap-Allokationsgeschwindigkeit und Heap-Fragmentierung als abhängige Variablen vorgestellt. Anschließend werden die über alle Konfigurationen und Metriken hinweg konstant gehaltenen Variablen dargestellt, darunter Hardware, Compiler und Werkzeugkette (*toolchain*), Zephyr-version, die gemeinsame Build-Konfiguration, der verwendete Mess- und Statistikmechanismus und die Anzahl der Messwiederholungen. Zuletzt wird der gesamte, für jede Messzelle gleiche Ablauf einer Messung dargestellt, darunter das Einrichten des Arbeitsbereichs, das Bauen und Flashen der jeweiligen Konfiguration und das Auslesen und Dokumentieren der Ergebnisse.

Kapitel 5 Benchmark-Design Im fünften Kapitel wird das Benchmark-Design im Detail beschrieben. Für jede der fünf Laufzeit- und Heap-Benchmarks und den dabei bestimmten statischen RAM-/ROM-Fußabdruck wird dabei dargestellt, was genau gemessen wird, wie die jeweilige Messmethode technisch umgesetzt ist, welche Fehlerquellen kontrolliert werden müssen und warum die jeweilige Metrik für die Forschungsfrage relevant ist. Dabei werden die Kontextwechsel-Latenz in beiden varianten Semaphor-Weckung und kooperatives Yielding, die Interrupt-Latenz eines softwaregetriggerten Interrupts, der Jitter eines periodischen Timers, die Geschwindigkeit von Heap-Allokation und Freigabe und zuletzt der Zustand des Heapspeichers nach einem fragmentierten Zugriffsmuster. Zu Beginn wird jedoch erstmal der Selbsttest als Validierungsschritt der Messkette erwähnt, der selbst keine der fünf Metriken darstellt, aber für die interne Validität der Messungen relevant ist.

Kapitel 6 Im sechsten Kapitel wird die softwaretechnische Umsetzung der Untersuchung beschreiben. Zuerst wird die Struktur des entwickelten Projekts und dessen Einbindung in den West-Arbeitsbereich vorgestellt. Anschließend wird die gemeinsam von allen Benchmarks verwendete Mess-Infrastruktur erläutert. Dabei werden die öffentliche Schnittstelle der entwickelten Messbibliothek, die verwendete Zeitbasis und deren Kalibrierung des Messaufwands beschrieben. Außerdem wird die laufende statistische Auswertung der Mesergebnisse beschrieben. Darüber hinaus wird das einheitliche Ausgabeformat der Ergebnisse und die Parametrisierung der Messbibliothek über Kconfig erläutert. Anschließend werden die für die Untersuchung entwickelten Konfigurationsstufen vorgestellt. Hierbei wird zuerst die gemeinsame Basis-Konfiguration beschrieben. Danach werden die einzelnen Minimal-, Logging- und IoT-Konfigurationen und deren jeweiligen Entwurfsentscheidungen erläutert. Bei der IoT-Konfiguration liegt zudem der Schwerpunkt auf den Netzwerk- und Sicherheits-Subsystemen Thread, CoAP und DTLS/PSK

und auf der bewussten Abgrenzung zu dem getrennt behandelten Firmware-Update-Subsystem. Der Fokus des Kapitels liegt dabei auf der nachvollziehbaren Umsetzung der in den vorherigen Kapiteln beschriebenen Methodik und des Benchmark-Designs.

Kapitel 7 Messergebnisse

Kapitel 8 Diskussion

Kapitel 9 Validität

Kapitel 10 Fazit

Kapitel 11 Ausblick

Grundlagen

Im folgenden Abschnitt werden grundlegende Begriffe für das Verständnis dieser Arbeit definiert. Dabei werden unter anderem die Begriffe *Job* und *Task* verwendet. Ein Job ist eine bestimmte Ausführung einer Aufgabe. Ein Task beschreibt auf der anderen Seite die dazugehörige Folge oder Klasse von Jobs. Bei einem periodischen Task entstehen solche Jobs in bestimmten Zeitabständen (Liu, 2000, S. 27–28). Bspw. könnte ein Task wie folgt aussehen: „Lese alle 10 ms einen Sensor aus.“, wobei die einzelnen Jobs unterteilt werden:

- Job 1 bei $t = 0$ ms
- Job 2 bei $t = 10$ ms
- Job 3 bei $t = 20$ ms
- (...)

2.1 Systeme und zeitliche Anforderungen

System Ein System lässt sich allgemein über den Zusammenhang zwischen einer Menge von Eingaben und einer daraus entstehenden Menge an Ausgaben beschreiben. Wie dabei dieser Zusammenhang intern entsteht, ist dabei oft nebensächlich. Ein Thermostat nimmt bspw. als Eingabe die gemessene Raumtemperatur und macht daraus ein Heiz- oder Kühlsignal als Ausgabe. Dabei ist von außen komplett unbekannt, wie die interne Regellogik aufgebaut ist. Dabei wird solch ein Fall Black-Box-Betrachtung genannt. (Laplante & Ovaska, 2012, S.3)

Ereignis Laplante und Ovaska benutzen den Begriff Ereignis (*event*) für einen Wechsel im Programmablauf, bei dem die Ausführung nicht einfach mit der nächsten sequenziellen Instruktion fortgesetzt wird. Solche Wechsel können dabei aus dem Programm selbst entstehen oder von außen ausgelöst werden. Zudem unterscheiden die Autoren synchrone und asynchrone Ereignisse. Eine programmgesteuerte Verzweigung gehört zur ersten Gruppe, eine von außen eintreffende Unterbrechung zur zweiten. Die Freigabe eines Jobs kann zudem mit einem solchen Ereignis verbunden sein (Laplante & Ovaska, 2012, S. 10).

Freigabezeitpunkt Der Freigabezeitpunkt (*release time*) gibt an, ab wann ein Job zur Ausführung bereitsteht. Er ist jedoch vom echten Beginn der Verarbeitung zu unterscheiden, da der Job nach seiner Freigabe noch auf Rechenzeit

warten kann. Die Freigabe kann bspw. durch das Eintreffen von neuen Sensordaten ausgelöst werden. Ein Ereignis und ein Job sind dabei nicht gleich. Das Ereignis ist der auslösende Vorgang, während der Job die anschließend zu bearbeitende Arbeitseinheit ist (Laplante & Ovaska, 2012, S. 10; Liu, 2000, S. 27).

Abschlusszeitpunkt Der Abschlusszeitpunkt (*completion time*) eines Jobs bezeichnet den Zeitpunkt, an welchem dieser fertig umgesetzt wurde (Liu, 2000, S.28).

Antwortzeit Wenn ein System auf eine Eingabe reagiert, dann vergeht eine gewisse Zeit, bis das System das angeforderte Verhalten und alle dazugehörigen Ausgaben komplett realisiert hat. Genau dieser Zeitraum zwischen dem Anlegen der Eingabe und der kompletten Reaktion wird Antwortzeit (*response time*) genannt. Bei dem Thermostat-Beispiel wäre das bspw. die Zeit zwischen dem Erfassen einer niedrigen Raumtemperatur und dem Einschalten der Heizung. Welche Antwortzeit dabei akzeptable ist, hängt vom Anwendungsfall ab. Eine Sicherheitsabschaltung darf z.B. weniger Zeit in Anspruch nehmen als eine Komfortfunktion (Laplante & Ovaska, 2012, S.4). Für die Antwortzeit eines einzelnen Jobs werden dessen Freigabe und Abschluss als zeitliche Bezugspunkte verwendet. Wenn r_i den Freigabezeitpunkt bezeichnet und f_i den Abschlusszeitpunkt, ergibt sich die Antwortzeit als $R_i = f_i - r_i$. Sie beinhaltet damit sowohl mögliche Wartezeiten als auch die eigentliche Ausführung des Jobs (Liu, 2000, S. 28).

Frist Eine Frist (*deadline*) legt fest, bis zu welchem Zeitpunkt ein Job abgeschlossen sein muss. Die relative Frist D_i beschreibt die verfügbare Zeitspanne ab seiner Freigabe. Zusammen mit dem Freigabezeitpunkt r_i ergibt sich daraus die absolute Frist $d_i = r_i + D_i$. Ob die zeitliche Vorgabe eingehalten wurde, lässt sich somit durch den Vergleich mit dem Abschlusszeitpunkt f_i bestimmen: Der Job ist dabei fristgerecht beendet, wenn $f_i \leq d_i$ gilt (Liu, 2000, S.27-28).

Fristüberschreitung Die Fristüberschreitung (*tardiness*) gibt an, wie lange ein Job nach seiner absoluten Frist noch bis zum Abschluss benötigt. Sie wird aus Abschlusszeitpunkt f_i und absoluter Frist d_i als $T_i = \max(0, f_i - d_i)$ berechnet. Bei einem rechtzeitig beendeten Job ist $T_i = 0$. Nur ein verspäteter Abschluss führt zu einem positiven Wert (Liu, 2000, S. 28).

Zeitliche Anforderung Zeitliche Anforderungen (*timing constraints*) legen fest, welche Zeitpunkte oder Zeiträume ein Job bei seiner Ausführung einhalten muss. Eine einfache Form ergibt sich dabei aus Freigabezeitpunkt und Frist. Anwendungen können jedoch weitere Bedingungen vorgeben. Für ihre Bewertung ist es relevant, welche Folgen eine Fristüberschreitung hat. Bei einer weichen zeitlichen Anforderung bleibt ein verspätetes Ergebnis an sich verwendbar, verliert aber an Nutzen. Eine verzögert aktualisierte Sensoranzeige kann bspw. weiterhin Informationen liefern. Bei einer harten zeitlichen Anforderung kann ein verspätetes Ergebnis jedoch unbrauchbar oder sogar schädlich sein, bspw. wenn eine Schutzabschaltung zu spät erfolgt (Liu, 2000, S. 28).

Ausgefallenes System Ein System gilt genau dann als ausgefallen (*failed system*), wenn es eine oder mehrere Anforderungen nicht mehr erfüllt, welche

in seiner Spezifikation festgelegt wurden. Dies gilt dabei unabhängig davon, ob die Ursache in der Hardware, der Software oder im zeitlichen Verhalten liegt. (Laplante & Ovaska, 2012, S.5)

Deterministisches System Ein System heißt deterministisch (*deterministic system*), wenn sein Verhalten komplett vorhersagbar ist. Wenn sich das System hierbei in einem bestimmten Zustand befindet und es eine bestimmte Eingabe erhält, dann steht fest, welche Ausgabe es liefert und in welchen Zustand es als Nächstes wechselt. Dabei gibt es in diesem Sinne keinen Zufall und keine Mehrdeutigkeit im Verhalten. Eine schwächere, aber verwandte Eigenschaft ist der Ereignisdeterminismus. Hierbei muss nur für die Eingaben, die ein Ereignis auslösen, feststehen, wie das System reagiert. Jedes deterministische System ist also dadurch automatisch auch ereignisdeterministisch, andersherum jedoch nicht. (Laplante & Ovaska, 2012, S.11)

2.2 Real Time Systems

Echtzeitsystem Bei einem Echtzeitsystem (*real-time system, RTS*) reicht ein Ergebnis, welches inhaltlich richtig ist, nicht allein aus. Es muss außerdem innerhalb der Zeit verfügbar sein, die für die Anwendung vorgegeben ist. Die zeitliche Anforderung gehört daher zur Korrektheitsbedingung des Systems. Welche Folgen dabei eine verspätete Reaktion hat, hängt von der Anwendung und ihren Anforderungen ab (Laplante & Ovaska, 2012, S. 5).

Echtzeitsysteme stehen oft in einer laufenden Wechselwirkung mit ihrer Umgebung. Eingaben können neue Verarbeitungsaufgaben auslösen, deren Ergebnisse auf der anderen Seite können auf die Umgebung einwirken. Diese reaktive Arbeitsweise sorgt dafür, dass periodisch geplante Aufgaben eingeschlossen werden (Laplante & Ovaska, 2012, S. 5).

Bei einem Temperaturregler könnte bspw. festgelegt werden, dass innerhalb einer vorgegebenen Frist reagiert werden muss, wenn eine Grenzwertüberschreitung erkannt wird. Ob die Temperatur dafür regelmäßig oder ereignisgesteuert erfasst wird, ist hierbei eine getrennte Entwurfsentscheidung.

Weiches Echtzeitsystem Bei weicher Echtzeit beeinträchtigt eine verspätete Reaktion die Leistungs- oder Dienstqualität, ohne die Funktion des Systems direkt aufzuheben (Laplante & Ovaska, 2012, S. 6).

Hartes Echtzeitsystem Bei harter Echtzeit kann schon eine einzelne verfehlte Frist zu einem kompletten oder katastrophalen Ausfall führen (Laplante & Ovaska, 2012, S. 6).

Festes Echtzeitsystem Bei fester Echtzeit können wenige Fristverletzungen toleriert werden. Eine größere Zahl kann dabei zum Systemausfall führen (Laplante & Ovaska, 2012, S. 7).

2.3 Eingebettete Systeme

Eingebettetes System Bei einem eingebetteten System werden Hardware und Software auf eine bestimmte Aufgabe oder eine bestimmte Menge an

Funktionen begrenzt. Dies stellt hierbei auch den Unterschied des Entwurfsziels von dem eines PCs dar, der für mehrere unterschiedliche Anwendungen benutzt werden kann. Das unterscheidet dabei zwischen einem zweckgebundenen Rechner (*special-purpose computer*) und einem Universalrechner (*general-purpose computer*). Die zum Gerät dazugehörige Software wird zudem Firmware genannt (Das, 2013, S. 34).

Für die Beschreibung des Aufbaus verwendet Das ein simples Modell aus Prozessor, Speicher, Sensoren und Aktoren. Sensoren stellen Eingabedaten bereit, der Prozessor verarbeitet sie und Aktoren machen eine Reaktion auf die Umgebung möglich. Der Speicher hält die Programme und Daten, die für den Betrieb gebraucht werden (Das, 2013, S. 37–38).

Ein Beispiel wäre ein Temperaturregler. Ein Sensor liefert einen Temperaturwert. Das Programm vergleicht ihn dabei mit einem vorgegebenen Sollwert und entscheidet dann anschließend, ob eine Heizung ein- oder ausgeschaltet werden soll.

Die zeitlichen Anforderungen eines eingebetteten Systems basieren auf der Wechselwirkung mit der physischen Umgebung. Sensoren bestimmen dabei, wann neue Zustandsinformationen verfügbar werden. Aktoren müssen innerhalb der für den jeweiligen Prozess vorgegebenen Zeit reagieren. Aus diesen Eigenschaften werden die Fristen der dazugehörigen Verarbeitungsaufgaben festgelegt. Ob dabei harte oder weniger strenge Echtzeitanforderungen gebraucht werden, hängt von den möglichen Folgen einer verspäteten Reaktion ab (Liu, 2000, S. 30).

Eingebettetes Betriebssystem Ein eingebettetes System braucht nicht für jede Anwendung ein Betriebssystem. Bei einer einfachen Softwareorganisation kann eine sich wiederholende durchlaufene Hauptschleife die Aufgaben bearbeiten. Ein solcher Superloop kann zudem durch Interrupt-Routinen erweitert werden, die auf bestimmte Ereignisse reagieren. Die Verwendung von Interrupts allein macht dabei noch keinen Superloop aus. Der entscheidende Punkt ist hierbei die Organisation der Verarbeitung in einer wiederkehrenden Schleife (Das, 2013, S. 296).

Wenn jedoch mehrere Aufgaben und gemeinsam genutzte Ressourcen koordiniert werden müssen, dann kann ein Betriebssystem diese Verwaltung übernehmen. Es stellt dann grundlegende Dienste bereit, auf denen die Anwendungssoftware aufbaut. Ein Betriebssystem, das für solche Einsatzbedingungen benutzt wird, wird eingebettetes Betriebssystem (*embedded operating system*) genannt (Das, 2013, S. 41, 296).

Mikrocontroller Ein Mikrocontroller (Microcontroller Unit, MCU) integriert eine Recheneinheit und weitere für Komponenten, die für den Gerätebetrieb gebraucht werden, auf einem Chip. Dazu gehören Speicher, Zeitgeber und bspw. serielle und parallele Schnittstellen. Dadurch lassen sich viele Aufgaben ohne einen Aufbau aus getrenntem Prozessor, externem Speicher und zusätzlichen Schnittstellenbausteinen realisieren. Nichtflüchtiger Speicher kann den Programmcode halten, während RAM für sich verändernde Daten während der Ausführung gebraucht wird (Das, 2013, S. 39).

Das beschreibt darüber hinaus auch die Integration von weiteren Systemfunktionen auf einem Chip im Kontext zu dem Begriff „System on Chip“ (SoC).

Für diese Arbeit ist vor allem die gemeinsame Bereitstellung von Prozessor, Speicher und Peripheriefunktionen auf dem verwendeten Mikrocontroller relevant (Das, 2013, S. 39).

2.4 Echtzeitbetriebssysteme

Betriebssystem Ein Betriebssystem (*operating system, OS*) übernimmt grundlegende Verwaltungsaufgaben, die sonst von den einzelnen Anwendungen selbst gelöst werden müssten. Dazu gehören die Nutzung der CPU, die Speicherverwaltung und der Zugriff auf Ein- und Ausgabegeräte. Dabei stellt es dafür Dienste und Schnittstellen bereit, damit Anwendungen nicht die Einzelheiten der benutzten Hardware berücksichtigen müssen. Diese Abstraktion macht die Nutzung der Ressourcen leichter und trennt dabei anwendungsspezifische Aufgaben von grundlegenden Systemfunktionen. Das Betriebssystem gehört damit zur Systemsoftware, wobei Anwendungsprogramme auf deren Diensten aufbauen (Das, 2013, S. 222).

Kernel Der Kernel stellt die grundlegenden Laufzeitdienste eines Betriebssystems bereit. Dazu zählen bspw. Aufgabenplanung, Interrupt-Behandlung und Speicherverwaltung. Das zeigt hierbei seine Stellung durch ein Zwiebelmodell. Um den Kernel herum liegen weitere Teile der Systemsoftware, Entwicklungswerkzeuge und anschließend die Anwendungsprogramme. Die Darstellung ordnet zudem diese Bausteine nach ihrer Rolle im Gesamtsystem (Das, 2013, S. 231–232).

Für Betriebssysteme, die eine Trennung zwischen Kernel- und Nutzerbereich implementieren, beschreibt Das außerdem verschiedene Zugriffsrechte. Privilegierte Operationen werden vom Kernel ausgeführt. Anwendungen können seine Dienste über Systemaufrufe (System Calls) anfordern. Dadurch lässt sich der Zugriff auf grundlegende Systemressourcen kontrollieren. Ob und wie diese Trennung dabei umgesetzt wird, hängt von der Betriebssystem- und Hardwarekonfiguration ab (Das, 2013, S. 232).

Kernel-Design Monolithische und mikrokernbasierte Entwürfe unterscheiden sich vor allem darin, wie sie Betriebssystemdienste auf privilegierte und getrennt ausgeführte Komponenten verteilen. Bei einem monolithischen Kernel sind dabei einige Dienste im Kernelbereich zusammengefasst. Ein Fehler in einer dieser Komponenten kann dadurch auch andere Teile des Systems beeinflussen. Das nennt Linux als Beispiel für diese Architekturvariante (Das, 2013, S. 233–234).

Ein Mikrokern schränkt den Kern auf der anderen Seite auf grundlegende Funktionen ein. Zudem verlagert es weitere Dienste in getrennte Komponenten bzw. Serverprozesse. Diese Trennung kann die Ausbreitung von Fehlern eingrenzen, braucht jedoch eine Kommunikation zwischen den beteiligten Diensten. Daraus entsteht eine Balancesuche zwischen Fehlerisolation, Modularität und Kommunikationsaufwand (Das, 2013, S. 234).

Die für die Untersuchung relevanten Eigenschaften des Zephyr-Kernels werden im folgenden Abschnitt anhand seiner Dokumentation beschrieben.

Echtzeitbetriebssystem Ein Echtzeitbetriebssystem (*real-time operating system*, RTOS) stellt wie andere Betriebssysteme Dienste zur Verwaltung von Aufgaben und Hardware-Ressourcen bereit. Für seinen Einsatz sind jedoch noch die zeitlichen Anforderungen der Anwendung relevant. Die Aufgabenplanung muss deshalb darauf achten, welche Verarbeitungen rechtzeitig abgeschlossen werden sollen. Das hebt Scheduling als einen relevanten Unterschied im Vergleich zu allgemeinen Betriebssystemen hervor (Das, 2013, S. 296–297). Für diese Arbeit ist daran vor allem relevant, wie die Priorisierung von Aufgaben das zeitliche Verhalten des Systems beeinflusst. Die genaue Auswahlregel des Zephyr-Schedulers wird in Abschnitt 2.5.1a erläutert.

2.5 Zephyr RTOS

Zephyr ist ein quelloffenes (open-source) RTOS, das für den Einsatz auf ressourcenbeschränkten eingebetteten Systemen entwickelt wurde. Das Projekt wird von der Linux Foundation als ein gemeinschaftlich entwickeltes Open-Source-Projekt unterstützt. Zephyr verfolgt das Ziel, ein skalierbares und plattformübergreifendes OS für unterschiedliche Embedded-Anwendungen bereitzustellen. Zephyr kann, durch die Unterstützung unterschiedlicher Prozessorarchitekturen und Hardwareplattformen auf Systemen mit verschiedenen Ressourcen- und Leistungsanforderungen eingesetzt werden. Die Architektur ist modular und konfigurierbar aufgebaut. Dadurch können benötigte Komponenten deaktiviert werden. Der Zephyr-Kernel unterstützt zudem kooperatives und präemptives Threading und stellt Funktionen für die Aufgabenverwaltung, Synchronisation, Speicherverwaltung und Interaktion mit Hardware bereit. Außerdem verfügt Zephyr über integrierte Schnittstellen für Gerätetreiber und Subsysteme für bspw. Metzwirkkommunikation, Bluetooth, USB, Dateisysteme und Logging. Durch die Möglichkeit, Zephyr zu konfigurieren, kann der benötigte Speicher- und Funktionsumfang an die verfügbare Hardware angepasst werden. Auf der anderen Seite unterstützt Zephyr auch leistungsfähigere Systeme. (Zephyr Project, 2024, „About the Zephyr Project“)

Zephyr basiert auf einem Kernel mit geringem Speicherbedarf. Dieser wurde für ressourcenbeschränkte und eingebettete Systeme entwickelt. Der Zephyr Kernel unterstützt hierbei eine Vielzahl von CPU-Architekturen, darunter die in dieser Arbeit verwendete ARMv7E-M (Cortex-M4F) Architektur. Zephyr wurde zudem durch Subsysteme modular konzipiert. Subsysteme umfassen hierbei logisch abgegrenzte Teile des OS, welche bestimmte Dienste bereitstellen, darunter bspw. Netzwerke, Dateisysteme oder Kommunikationsprotokolle, etc. Jedes Subsystem ist modular aufgebaut und kann somit konfiguriert, angepasst und erweitert werden. (Zephyr Project, 2024, „Introduction“)

Die Messanwendung dieser Arbeit ist in C implementiert und nutzt dabei die über Zephyrs Header bereitgestellten Funktionen und Makros. Welche Dienste verfügbar sind, hängt dabei auch von der ausgewählten Konfiguration und Zielplattform ab. Für den Anwendungseinstieg schreibt die Dokumentation den Rückgabety `int` für `main()` vor. Beim Beenden soll die Funktion den

Wert null zurückgeben. Andere Rückgabewerte sind nämlich bereits reserviert (Zephyr Project, 2023, „C Language Support“).

2.5.1 Laufzeit-Konzepte

Der Zephyr-Kernel bildet das Herzstück jeder Zephyr-Anwendung. Er stellt dabei eine ressourcenschonende, performante mehr-Thread-Ausführungsumgebung mit einem umfangreichen Funktionsangebot bereit. Dabei bauen darauf mehrere weitere Bestandteile des Systems auf, darunter Gerätetreiber, Netzwerk-Stacks und anwendungsspezifischer Code. Durch die Konfigurierbarkeit lässt sich der Kernel auf genau die Funktionen beschränken, die eine Anwendung wirklich benötigt. Dadurch ist er sowohl für Systeme mit geringem Speicherbedarf als auch für deutlich komplexere Anwendungen mit mehreren Kommunikationsstellen und aufwendigerer Nebenläufigkeit geeignet.

Die Zephyr-Dokumentation unterteilt dabei die Kernel-Dienste in mehrere Kategorien:

- „Scheduling, Interrupts and Synchronization“, darunter Threads, Scheduling, Interrupts, Semaphores and Synchronization
- „Data Passing“, darunter Mechanismen zum Datenaustausch zwischen Threads und ISRs
- „Memory Management“
- „Timing“, darunter Zeitgeber und Uhren

Für die Untersuchung wurden aus diesen Kategorien besonders die folgenden Themen ausgesucht, da sie die Grundlagen für die später gemessenen Benchmarks bilden:

- Threads und Scheduling
- Synchronisation mittels Semaphoren
- Interrupts
- Timer und die zugehörige Zeitbasis des Kernels
- Speicherverwaltung

Ergänzend zu diesen Themen wird zudem das Logging-Subsystem definiert, welches als eigenständiger Dienst außerhalb des Kernel-Services in der Zephyr-Dokumentation dokumentiert wird. Außerdem wird die Timing-API zur Zeitmessung behandelt, da diese und das Logging-Subsystem beide für die Konfiguration bzw. für die Mess-Infrastruktur dieser Arbeit wichtig sind.

Threads und Scheduling

Für den Kontextwechsel-Benchmark ist vor allem wichtig, wann Zephyr die Auswahl des laufenden Threads neu prüft. Solche Scheduling-Gelegenheiten entstehen bspw., wenn ein Thread blockiert, ein anderer Thread bereit wird oder der laufende Thread die CPU über `k_yield()` freiwillig abgibt. Auch die Rückkehr aus einer Interrupt-Behandlung kann hierbei zu einer neuen Auswahl führen. Die Dokumentation nennt diese Stellen Reschedule Points (Zephyr Project, 2024, „Scheduling“, Abschnitt „Concepts“).

Ob ein anderer Thread ausgeführt wird, hängt dabei von den bereiten Threads, ihren Prioritäten und den Scheduling-Regeln ab, die gerade gelten. Bei einem Wechsel müssen die Zustandsinformationen erhalten bleiben, die für die spätere Fortsetzung gebraucht werden. Die in Kapitel 5.1 untersuchten Varianten benutzen dafür die Aktivierung eines wartenden Threads bzw. die freiwillige CPU-Abgabe.

Auswahlkriterium des Schedulers Der Scheduler wählt den bereiten Thread mit der höchsten Priorität als aktuellen Thread aus. Existieren mehrere bereite Threads derselben Priorität, wird derjenige gewählt, der am längsten wartet. Die Ausführung von Interrupt-Service-Routinen (ISRs) hat dabei Vorrang vor der Ausführung jedes Threads. Dies gilt hierbei sowohl für kooperative als auch präemptive Threads.

Kooperative Threads Kooperative und präemptive Threads unterscheiden sich darin, unter welchen Bedingungen andere Threads ihre Ausführung verdrängen können. Ein kooperativer Thread gibt die CPU im regulären Scheduling selbst ab, bspw. durch Blockieren oder durch `k_yield()`. Bei einem Yield bekommen andere bereite Threads gleicher oder höherer Priorität die Möglichkeit, ausgeführt zu werden. Wenn kein geeigneter Thread vorhanden ist, dann kann der aufrufende Thread direkt weiterlaufen. Längere Berechnungen ohne freiwillige Abgabe können deshalb andere Threads verzögern (Zephyr Project, 2024, „Scheduling“, Abschnitt „Cooperative Time Slicing“).

Präemptive Threads Ein präemptiver Thread kann auf der anderen Seite verdrängt werden, wenn ein höherpriorer Thread bereit ist. Darüber hinaus kann Zephyr die CPU-Zeit zwischen gleichprioren präemptiven Threads über Zeitscheiben verteilen. Nach Ablauf einer Zeitscheibe bekommen andere passende Threads dieser Priorität eine Möglichkeit, ausgeführt zu werden. Ob und für welche Prioritäten diese Rotation benutzt wird, ist dabei konfigurierbar (Zephyr Project, 2024, „Scheduling“, Abschnitt „Preemptive Time Slicing“).

Diese Unterscheidung ist für die Thread-Auswahl relevant. Sie bedeutet nicht, dass kooperative Threads von Grund auf vor Interrupts geschützt sind.

Scheduler Locking Ein präemptiver Thread, der während einer kritischen Operation nicht unterbrochen werden möchte, kann den Scheduler über `k_sched_lock()` anweisen, ihn wie einen kooperativen Thread zu behandeln. Danach stellt `k_sched_unlock()` den ursprünglichen präemptierbaren Status wieder her.

Thread Sleeping und Busy Waiting Der Kernel stellt, außer dem bereits beschriebenen `k_sleep()`, eine zweite Möglichkeit durch `k_busy_wait()` bereit, die Ausführung für eine bestimmte Zeit zu verzögern, ohne die CPU dabei

an einen anderen Thread abzugeben. Laut Zephyr-Dokumentation wird ein Busy-Wait normalerweise dann verwendet, wenn die Verzögerung zu kurz ist, um den Aufwand eines Kontextwechsels zu einem anderen Thread und zurückzurechtfertigen.

Empfohlene Einsatzgebiete Die Zephyr-Dokumentation empfiehlt kooperative Threads unter anderem für Gerätetreiber und andere leistungs-kritische Arbeiten. Diese Threads werden außerdem auch zur gegenseitigen Ausschlusssicherung ohne ein eigenes Kernel-Objekt, bspw. einem Mutex, empfohlen. Auf der anderen Seite werden präemptive Threads empfohlen, um zeitkritischer Verarbeitung Vorrang vor weniger zeitkritischer Verarbeitung zu geben.

Synchronisation (Semaphore)

Ein Semaphor macht es möglich, die Verfügbarkeit einer Ressource oder die Freigabe eines Verarbeitungsschritts zwischen Ausführungskontexten zu organisieren. Zephyr stellt dafür einen zählenden Semaphor bereit. Sein Zähler beschreibt die aktuell verfügbare Anzahl von Freigaben. Ein Grenzwert begrenzt den maximalen Zählerstand. Bei der Initialisierung darf der Zähler auch null sein, muss aber innerhalb des zugelassenen Bereichs bis zum Grenzwert liegen (Zephyr Project, 2022, „Semaphores“, Abschnitt „Concepts“).

Eine erfolgreiche Entnahme verbraucht dabei eine Freigabe. Wenn keine verfügbar ist, dann kann ein Thread auf eine spätere Freigabe warten. Bei mehreren wartenden Threads achtet Zephyr zuerst auf die Priorität und bei gleicher Priorität auf die Wartezeit. Ein Semaphor kann dabei auch aus einer ISR gegeben werden. Eine Entnahme aus einer ISR muss dabei nichtblockierend mit `K_NO_WAIT` erfolgen (Zephyr Project, 2022, „Semaphores“, Abschnitte „Concepts“ und „API Reference“).

Im Kontextwechsel-Benchmark wird das Semaphor zuerst leer angelegt. Der Worker wartet anschließend darauf, dass der Hauptthread eine Freigabe auslöst. Das Semaphor dient in diesem Fall dazu, einen definierten Übergang zu synchronisieren und nicht dazu, mehrere gleichartige Ressourcen zu verwalten.

Zwei Einsatzmuster Laut der Zephyr-Dokumentation kann ein „volles“ Semaphor (Zählerstand gleich Grenzwert) dafür verwendet werden, die Anzahl der Threads zu begrenzen, die gleichzeitig einen kritischen Abschnitt ausführen dürfen. Ein „leeres“ Semaphor (Zählerstand gleich Null, Grenzwert größer Null) erzeugt auf der anderen Seite eine Art Schranke, an der kein wartender Thread vorbei darf, bis das Semaphor erhöht wird. Dieses zweite Muster bildet die Grundlage für die Freigabe eines wartenden Threads durch einen Erzeuger-Thread oder eine ISR.

Definition und Nutzung im Code Ein Semaphor wird über eine Variable vom Typ `k_sem` angelegt und anschließend mit `k_sem_init(&my_sem, initial_count, limit)` initialisiert. Es kann aber auch mit `K_SEM_DEFINE(name, initial_count, count_limit)` zur Übersetzungszeit definiert und initialisiert werden. Das Geben ist durch `k_sem_give(&my_sem)` möglich. Das Nehmen funktioniert auf

der anderen Seite durch `k_sem_take(&my_sem, timeout)`, wobei `timeout` als Wartezeit, z.B. `K_MSEC(50)`, oder als einer der Sonderwerte `K_NO_WAIT` / `K_FOREVER` angegeben werden kann.

Empfohlene Einsatzgebiete Die Zephyr-Dokumentation empfiehlt Semaphoren zur Zugriffskontrolle auf eine Menge von Ressourcen durch mehrere Threads und zur Synchronisation der Verarbeitung zwischen erzeugenden und verbrauchenden Threads oder ISRs.

Interrupts

Eine Interrupt-Service-Routine (ISR) übernimmt die Bearbeitung von einer Unterbrechungsanforderung außerhalb des regulären Thread-Ablaufs. Dafür wird eine Interruptquelle mit einem Handler verbunden. Bei der Registrierung werden unter anderem die IRQ-Nummer, die Priorität, die auszuführende Funktion und ein Argument festgelegt, das ggf. übergeben werden soll (Zephyr Project, 2024, „Interrupts“, Abschnitt „Concepts“).

Die Zuordnung wird über die Interrupt-Infrastruktur des Systems gemacht. Interrupts, die nicht eingerichtet sind, werden dabei durch einen Standardhandler behandelt, der einen unerwarteten Aufruf als Fehler meldet. Zephyr unterstützt außerdem gemeinsam genutzte Interruptleitungen. Für diese Untersuchung wird jedoch eine einzelne, dedizierte Zuordnung verwendet (Zephyr Project, 2024, „Interrupts“, Abschnitte „Concepts“ und „Sharing interrupt lines“).

Für die Messung in Kapitel 5.2 ist insbesondere der Weg von der ausgelösten IRQ-Anforderung bis zum Zeitstempel innerhalb des registrierten Handlers relevant.

Interrupt-Verschachtelung Der Kernel unterstützt das Verschachteln von Interrupts. Eine ISR kann dabei mitten in ihrer Ausführung von einer höherpriorigen ISR unterbrochen werden. Dabei setzt die niedrigerpriorige ISR ihre Verarbeitung fort, sobald die höherpriorige fertig ist. Eine ISR wird dabei im sogenannten „Interrupt-Kontext“ des Kernels ausgeführt, der über einen eigenen dedizierten Stack-Bereich verfügt. Laut der Zephyr-Dokumentation dürfen viele Kernel-APIs nur von Threads und nicht von ISRs aufgerufen werden. Jedoch stellt der Kernel für Routinen, die von beiden aufgerufen werden können, die Funktion `k_is_in_isr()` bereit. Damit kann eine Routine ihr Verhalten basierend auf dem Aufrufkontext anpassen.

Unterbrechungen verhindern Ein Thread kann Interrupt-Verarbeitungen im System temporär über eine IRQ-Sperre (`irq_lock()` / `irq_unlock()`) unterbinden. Diese Sperre ist laut der Zephyr-Dokumentation thread-spezifisch. Auf der anderen Seite kann ein einzelnes Interrupt gezielt über `irq_disable()` / `irq_enable()` deaktiviert bzw. aktiviert werden. Dadurch wird die zugehörige ISR währenddessen nicht ausgeführt.

Auslagern von ISR-Arbeit Um ein vorhersagbares (deterministisches) System sicherstellen zu können, muss eine ISR schnell ausgeführt werden. Daher empfiehlt die Zephyr-Dokumentation, zeitaufwendige Verarbeitungen an Threads auszulagern. Dies ist bspw. möglich, indem die ISR über ein Kernel-Objekt wie ein Semaphor einen Hilfs-Thread benachrichtigt.

Definition einer ISR **Definition einer ISR:** Statische Interrupt-Verbindungen werden mit `IRQ_CONNECT(irq, priority, isr, arg, flags)` zur Übersetzungszeit festgelegt. IRQ-Nummer, Priorität und Flags müssen dabei konstante Ausdrücke sein. Anschließend wird die Leitung mit `irq_enable()` aktiviert. Für dynamische Verbindungen stellt Zephyr getrennte APIs bereit (Zephyr Project, 2024, „Interrupts“).

Empfohlene Einsatzgebiete und Konfiguration Die Zephyr-Dokumentation empfiehlt reguläre bzw. „direkte“ ISRs für Interrupt-Verarbeitung, die eine schnelle, nicht-blockierende Reaktion erfordert. Dabei sollen zeitaufwendige oder blockierende Verarbeitungen an einen Thread übergeben werden. Zudem kann man die Konfigurationsoption `CONFIG_ISR_STACK_SIZE` hinzufügen, welche die Größe des Interrupt-Stacks festlegt.

Timer und Tickless-Kernel

Der Timer als Kernel-Objekt Ein Timer ist laut Zephyr-Dokumentation ein Kernel-Objekt, welches den Zeitablauf anhand der Systemuhr des Kernels misst. Es können dabei mehrere Timer angelegt werden, wobei die Menge nur durch den verfügbaren RAM begrenzt wird. Ein Zephyr-Timer kann einen einmaligen oder wiederkehrenden Ablauf auslösen. Beim Start werden dafür zwei Zeitangaben festgelegt, darunter die Dauer, welche den ersten Ablauf bestimmt und die Periode, welche das Zeitraster der folgenden Abläufe bestimmt. Mit `K_NO_WAIT` oder `K_FOREVER` als Periodenparameter wird der Timer dabei einmalig betrieben. Eine zugeordnete Ablauffunktion wird dann im Kontext des Systemclock-Interrupt-Handlers aufgerufen (Zephyr Project, 2024, „Timers“, Abschnitt „Concepts“).

Für den Benchmark wird ein periodischer Timer benutzt. Dabei ist die Unterscheidung zwischen seinem vorgesehenen Ablaufzeitpunkt und dem Eintritt in die Ablauffunktion für die Interpretation der Zeitmessung relevant.

Wichtige Präzisierung für Zeitmessungen Die Zephyr-Dokumentation erwähnt, dass Dauer und Periode eines Timers Mindest-Verzögerungen festlegen. Das liegt daran, dass durch die interne Präzision des Systemzeitgebers und andere mögliche Laufzeiteinflüsse wie Interrupt-Verzögerungen mehr Zeit vergehen kann, als von den zugehörigen System-Zeit-APIs gemessen wird. Die angegebene Dauer ist eine Mindestverzögerung bis zum Ablauf. Daraus folgt jedoch nicht, dass jedes zwischen zwei Callback-Eintritten gemessene Intervall nur nach oben abweichen kann, weil Verzögerungen einzelner Callback-Aufrufe auch die benachbarten Intervalle beeinflussen können (Zephyr Project, 2024, „Timers“).

Definition und Start eines Timers. Ein Timer wird über eine Variable vom Typ `k_timer` angelegt und mit `k_timer_init()` initialisiert. Auf der anderen Seite kann dies schon zur Übersetzungszeit durch `K_TIMER_DEFINE(name, expiry_fn, stop_fn)` definiert werden. Ein Timer wird über `k_timer_start(timer, duration, period)` gestartet. Dabei wird sein Statuswert auf Null zurückgesetzt und der Timer fängt an, herunterzuzählen. Die Ablauffunktion

(*expiry function*), welche dem Timer zugeordnet wird, wird laut der Zephyr-Dokumentation vom Interrupt-Handler der Systemuhr selbst ausgeführt. Dies passiert jedes Mal, wenn der Timer abläuft.

Zeiteinheiten im Kernel. Die Zephyr-Dokumentation erwähnt verschiedene interne Zeiteinheiten:

- Reale Zeitwerte (Millisekunden/ Mikrosekunden) als portable Standarddarstellung
- Einen „CycleZähler über `k_cycle_get_32()` / `k_cycle_get_64()`, der den schnellsten verfügbaren Zyklenzähler des Systems darstellt
- Sogenannte „Ticks“ als die interne Zählereinheit, in der der Kernel sein gesamtes Uptime- und Timeout-Bookkeeping durchführt

Kernel-Timeouts und Uptime werden intern in Ticks verwaltet. Deren Frequenz wird über `CONFIG_SYS_CLOCK_TICKS_PER_SEC` festgelegt (Zephyr Project, 2024, „Kernel Timing“). Die Tick-Rate ist dabei über `CONFIG_SYS_CLOCK_TICKS_PER_SEC` konfigurierbar. Die Uptime des Systems lässt sich zudem über `k_uptime_get()` in Millisekunden abfragen. Über `k_uptime_ticks()` kann die interne 64-Bit-Tick-Zählung abgefragt werden. Innerhalb der Verarbeitung eines Timeout-Ereignisses entspricht dabei dieser Wert in Zephyr v3.7.2 jedoch dem geplanten Timeout-Zeitpunkt und nicht unbedingt dem Eintrittszeitpunkt in die Ablauffunktion. Für die Messung der realen Callback-Abstände wird deshalb in dieser Arbeit der frei laufende Systemtimer über `sys_clock_cycle_get_32()` ausgelesen.

Der tickless Kernel. Ein Zeitgebertreiber muss dem Kernel neue Ticks über `sys_clock_announce()` melden. Die Zephyr-Dokumentation beschreibt dabei zwei Implementierungsarten. Eine „natürliche“ Umsetzung dieser Schnittstelle führt zu einem „tickless“ Kernel, der Timer-Interrupts nur für registrierte Ereignisse empfängt und verarbeitet. Hierbei wird dies auf programmierbare Hardware-Zähler gestützt, die unregelmäßige Interrupts auslösen können. Auf der anderen Seite steht eine traditionelle, „getaktete“ Umsetzung, bei der der Treiber Interrupts in einer regelmäßigen Frequenz empfängt, die der Systemtakt rate entspricht. Zudem wird jedes Mal genau ein Tick gemeldet, wobei dies passiert, wenn ein Ereignis ansteht oder auch nicht.

Empfohlene Einsatzgebiete. Ein Timer soll laut Dokumentation verwendet werden, um einen asynchronen Vorgang nach einer bestimmten Zeit auszulösen, oder um festzustellen, ob eine bestimmte Zeit bereits verstrichen ist. Das gilt hierbei besonders dann, wenn eine höhere Präzision benötigt wird, als es die Aufrufe `k_sleep()` und `k_usleep()` bieten. Auf der anderen Seite empfiehlt die Zephyr-Dokumentation, für die Messung der für einen Vorgang selbst benötigten Zeit direkt die System- bzw. Hardware-Uhr auszulesen, anstatt dafür einen Timer zu benutzen.

Speicherverwaltung

Laut Zephyr-Dokumentation werden von Zephyr mehrere Werkzeuge bereitgestellt, mit denen Threads dynamisch Speicher anfordern können. Für diese Arbeit sind davon zwei Ebenen relevant:

- Der synchronisierte `k_heap`, den die Anwendung selbst für einen eigenen Speicherbereich anlegen kann
- Der vordefinierte System-Heap, der über `k_malloc()`/`k_free()` angesprochen wird

Der synchronisierte Heap (`k_heap`). Der einfachste Weg, einen Heap anzulegen, ist laut Zephyr-Dokumentation die statische Definition über das Makro `K_HEAP_DEFINE(name, bytes)`. Dieses erzeugt eine statische `k_heap`-Variable, die einen Speicherbereich der angegebenen Größe verwaltet. Speicher wird dabei über `k_heap_alloc(heap, bytes, timeout)` angefordert. Die Funktion verhält sich hierbei ähnlich wie das `malloc()` aus der Standard-C-Bibliothek und liefert bei einem Fehler einen Nullzeiger (`nullpointer`) zurück. Der Heap unterstützt dabei auch blockierende Operationen. Ein Thread kann in den Schlafzustand versetzt werden, bis Speicher verfügbar ist. Dabei gibt der Timeout-Parameter an, wie lange maximal gewartet werden darf. Freigegeben wird zuvor angeforderter Speicher über `k_heap_free()`, genau wie bei `free()`.

Die Implementierung (`sys_heap`) Die Speicherzuteilung eines `k_heap` basiert auf dem `sys_heap`-Allokator. Dabei ergänzt `k_heap` die benötigte Synchronisation. Bei direkter Nutzung von `sys_heap` muss der Aufrufer miteinander konkurrierende Zugriffe selbst organisieren (Zephyr Project, 2024, „Memory Heaps“, Abschnitt „Low Level Heap Allocator“).

Der Allokator verwaltet den Speicher in Einheiten von acht Byte und hält Informationen über belegte und freie Bereiche in Verwaltungsstrukturen. Freie Blöcke werden dabei nach Größenklassen organisiert, damit für eine Anforderung ein passender Block gesucht werden kann. Bei einer Freigabe können benachbarte freie Bereiche zusammengeführt werden. Diese Verfahren sorgen dafür, dass Fragmentierung verringert wird. Jedoch schließen sie dies nicht aus (Zephyr Project, 2024, „Memory Heaps“, Abschnitt „Low Level Heap Allocator“).

Die Suche innerhalb einer Größenklasse ist durch `CONFIG_SYS_HEAP_ALLOC_LOOPS` begrenzt. Der dokumentierte Standardwert beträgt hierbei drei. Dadurch wird der Suchaufwand begrenzt, jedoch kann die Auswahl eines passenderen Blocks darunter leiden. Ein begrenzter algorithmischer Aufwand wird dabei von einer konstanten, bei jedem Aufruf gemessenen Zyklenzahl unterschieden (Zephyr Project, 2024, „Memory Heaps“, Abschnitt „Low Level Heap Allocator“).

Der System-Heap (`k_malloc`/`k_free`). Der System-Heap ist ein vordefinierter Speicherzuteiler, aus dem Threads in einer `malloc()`-ähnlichen Weise dynamisch Speicher aus einem gemeinsamen Speicherbereich anfordern können. Anders als bei anderen Heaps kann der System-Heap nicht direkt über eine Speicheradresse referenziert werden. Es gibt laut Zephyr-Dokumentation nur einen einzigen System-Heap im gesamten System. Seine Größe wird über die Kconfig-Option `CONFIG_HEAP_MEM_POOL_SIZE` festgelegt. Dabei ist der Standardwert Null Byte, was bedeutet, dass der Kernel gar kein Speicherpool-Objekt anlegt. Wenn die angeforderte Größe hierbei größer ist als der verfügbare Speicher, dann schlägt der Build schon in der Linker-Phase fehl. Speicher

wird über `k_malloc(size)` angefordert und über `k_free(ptr)` wieder freigegeben. Zudem entsprechen beide Funktionen, was die Semantik angeht, den gleichnamigen Standard-C-Funktionen.

Logging-Subsystem

Die Logging-API von Zephyr stellt laut Zephyr-Dokumentation eine Schnittstelle bereit, über die von Entwicklern ausgegebene Nachrichten verarbeitet werden. Nachrichten laufen dabei zuerst durch ein Frontend und werden dann von aktiven Backends verarbeitet. Die Architektur des Logging-Subsystems besteht dabei laut Zephyr-Dokumentation aus genau drei Hauptteilen, dem Frontend, dem Core und den Backends.

Das Standard-Frontend Das Standard-Frontend wird immer dann aktiv, wenn die Logging-API in einer Quelle aufgerufen wird, z.B. durch das Makro `LOG_INF`. Es ist dafür zuständig, eine Nachricht zu filtern, sowohl zur Übersetzungs- als auch zur Laufzeit. Außerdem ist es dafür zuständig, Speicher für die Nachricht zu reservieren, die Nachricht zu erzeugen und sie dann zu übergeben. Da die Logging-API laut Zephyr-Dokumentation auch innerhalb eines Interrupts aufgerufen werden kann, ist das Frontend darauf optimiert, eine Nachricht so schnell wie möglich zu protokollieren. Eine Log-Nachricht wird dabei in einem zusammenhängenden Speicherblock abgelegt, der aus einem „zirkulären Paketpuffer“ reserviert wird.

Zwei Die Zephyr-Dokumentation unterscheidet drei globale Logging-Modi, darunter `Deferred`, `Immediate` und den auf minimalen Funktionsumfang ausgerichteten `Minimal-Modus` (Zephyr Project, 2024, „Logging“). Die Zusammenfassung der Logging-Funktionen auf derselben Zephyr-Dokumentationsseite notiert dazu, dass „deferred logging“ die für eine Log-Ausgabe benötigte Zeit reduziert. Dies wird möglich, indem zeitintensive Operationen in einen bekannten, dafür vorgesehenen Kontext verschoben werden, anstatt die Nachricht schon beim eigentlichen Aufruf zu verarbeiten und auszugeben.

Auswirkung auf den Stack-Verbrauch Die Zephyr-Dokumentation erläutert im Abschnitt „Stack usage“, dass die Aktivierung des Logging-Subsystems den Stack-Verbrauch des jeweils aufrufenden Kontexts beeinflusst:

- Wird `CONFIG_LOG_MODE_DEFERRED` verwendet, wird dieser Verbrauch geringer, da die Logging-Funktion an der Aufrufstelle nur die Nachricht erzeugt und im Puffer ablegt.
- Wird `CONFIG_LOG_MODE_IMMEDIATE` verwendet, wird die Nachricht direkt vom Backend verarbeitet, was auch die Zeichenketten-Formatierung inkludiert. Der Stack-Verbrauch hängt in diesem Fall auch davon ab, welche Backends verwendet werden.

Verarbeitung im Deferred-Modus Normalerweise wird die Verarbeitung von Nachrichten im `Deferred-Modus` intern durch einen dafür vorgesehenen Task übernommen, der automatisch startet. Dieser ist steuerbar über

`CONFIG_LOG_PROCESS_THREAD`. Die Aktivierungsschwelle lässt sich über die Anzahl gepufferter Nachrichten durch `CONFIG_LOG_PROCESS_TRIGGER_THRESHOLD` konfigurieren. Die Größe des dafür verwendeten Ringpuffers wird dabei über `CONFIG_LOG_BUFFER_SIZE` festgelegt. Über `CONFIG_LOG_DEFAULT_LEVEL` lässt sich auch der Schweregrad festlegen, der für Module ohne eigene Log-Level-Angabe benutzt wird. Die Zephyr-Dokumentation nennt vier verfügbare Schweregrade. Diese sind Fehler (Error), Warnung (Warning), Information (Info) und Debug. Als Ausgabekanal (*Backend*) steht unter anderem `CONFIG_LOG_BACKEND_UART` zur Verfügung. Laut Zephyr-Dokumentation unterstützt das Subsystem bis zu neun gleichzeitig aktive Backends. Über `CONFIG_LOG_FUNC_NAME_PREFIX_DBG` lässt sich zudem festlegen, ob Debug-Nachrichten automatisch mit dem Namen der aufrufenden Funktion versehen werden.

Vom Projekt selbst gemessene Größenordnung Die Dokumentation enthält im Abschnitt „Benchmarkeigene, auf der Plattform `qemu_x86` gemessene Vergleichswerte des Zephyr-Projekts (Testsuite `tests/subsys/logging/log_benchmark`), die den Zeitaufwand von einzelnen Logging-Operationen in der Größenordnung von ca. 7 bis 50 μ s notieren, abhängig von Modus und Art der Daten.

Timing-API

Die Zephyr-Timing-API bietet Funktionen, mit denen die Ausführungszeit eines Codeabschnitts herausgefunden werden kann, um diesen zu analysieren oder zu optimieren. Die Zephyr-Dokumentation erwähnt dabei, dass die Timing-Funktionen einen anderen Zeitgeber benutzen können als der Standard-Kernel-Timer. Welcher Zeitgeber hierbei verwendet wird, ist architektur-, SoC- oder board-spezifisch festgelegt. Die API ist daher bewusst allgemein gehalten und empfiehlt keinen bestimmten Hardware-Zähler. Die in dieser Arbeit verwendete Hardware-Plattform wird in Kapitel 4.2 genauer erklärt.

Voraussetzung Um die Timing-Funktionen nutzen zu können, muss die Kconfig-Option `CONFIG_TIMING_FUNCTIONS` aktiviert sein.

Vorgesehener Ablauf laut Dokumentation Die Nutzung der API folgt einem festen Schema. Die Zeitmessung kann in Vorbereitung, Erfassung und Auswertung unterteilt werden. Zur Vorbereitung initialisieren `timing_init()` und `timing_start()` die benutzte Timing-Infrastruktur und starten ihre Nutzung. Für einen Codeabschnitt werden dann anschließend mit `timing_counter_get()` zwei Zählerstände erfasst. `timing_cycles_get()` bestimmt daraus dann die vergangene Zyklenzahl. Wenn dies gebraucht wird, kann die Zyklenzahl mit `timing_cycles_to_ns()` in Nanosekunden umgerechnet werden. Nach Abschluss der Erfassung kann `timing_stop()` benutzt werden (Zephyr Project, 2023, „Executing Time Functions“, Abschnitt „Usage“).

Die Messbibliothek dieser Arbeit übernimmt die Initialisierung gemeinsam für die Benchmarks und unterteilt die Zählerzugriffe in `bench_now()` und `bench_elapsed()`.

Weitere Funktionen der API Außer den genannten Kernfunktionen stellt die API `timing_freq_get()`, welches die Frequenz des verwendeten Zählers in Hz liefert und `timing_freq_get_mhz()`, was dasselbe in MHz ist, zur Verfügung. Die Zephyr-Dokumentation notiert außerdem, dass bei manchen Architekturen oder SoCs die vom Zähler gelieferten Rohwerte noch skaliert werden müssen, um die Zyklenzahl zu erhalten. Diese Umrechnung übernimmt `timing_cycles_get()` intern.

2.5.2 Build-Zeit-Konzepte

Zephyr stellt ein eigenes Build- und Konfigurationssystem bereit, welches festlegt, wie eine Anwendung aus Quellcode zu einer lauffähigen Firmware zusammengesetzt wird. Die Zephyr-Dokumentation fasst darunter das CMake-basierte Build-System, den Devicetree, das Kconfig-Konfigurationssystem, Snippets und das Sysbuild-System für Mehr-Image-Builds zusammen.

Das Konfigurationssystem Kconfig macht es möglich, den Zephyr-Kernel und seine Subsysteme zur Build-Zeit an die Anforderungen einer bestimmten Anwendung und Zielplattform anzupassen. Dabei handelt es sich um dasselbe Konfigurationssystem, das auch der Linux-Kernel verwendet. Das Ziel ist dabei eine Konfigurierbarkeit, ohne dass dafür Quellcode verändert werden muss. Hierbei werden Konfigurationsoptionen (auch „Symbole“ genannt) in Kconfig-Dateien definiert. Diese legen dabei gleichzeitig die Abhängigkeiten zwischen den Symbolen fest, die eine gültige Konfiguration bestimmen. Aus Kconfig entsteht als Ergebnis eine Header-Datei mit Makros, die zur Build-Zeit ausgewertet werden können. Dadurch kann nicht benötigter Code aus dem Build ausgeschlossen werden.

Damit auch externe Software-Projekte in dieses Build-System eingebunden werden können, ohne deren Code direkt in den Zephyr-Quellbaum zu kopieren, verwendet Zephyr das Konzept der Module. Dabei handelt es sich um Repositories, die eine eigene `module.yml`-Datei mitbringen und dadurch vom Build-System automatisch mit ihren eigenen `CMakeLists.txt`- und Kconfig-Dateien eingebunden werden.

Das Kommandozeilenwerkzeug West übernimmt dabei die Verwaltung mehrerer, teilweise unabhängiger Repositories, darunter Zephyr selbst, Module und eigene Anwendungsprojekte. Die Zephyr-Dokumentation bezeichnet dieses Werkzeug auch als eine Art „Schweizer Taschenmesser“ für das Projekt. West stellt hierbei ein Mehr-Repository-Verwaltungssystem bereit, das von Googles Repo-Werkzeug und Git-Submodulen inspiriert ist. Dabei ist West zudem erweiterbar. Zephyr selbst nutzt diese Erweiterbarkeit, um weitere Kommandos für das Bauen, Flashen und Debuggen von Anwendungen bereitzustellen.

Für diese Arbeit sind dabei vor allem die Funktionsweise von Kconfig-Fragmenten, der Modul-Mechanismus zur Einbindung von eigenen Bibliotheken, West als Werkzeug zur Versionsverwaltung und zum Bauen/ Flashen und zuletzt die im Build-System enthaltenen Werkzeuge zur Analyse des Speicherbedarfs relevant.

Kconfig

Grundprinzip Kconfig legt fest, welche konfigurierbaren Softwarefunktionen in einen Zephyr-Build eingehen. Dazu definiert das Projekt Symbole mit ihren möglichen Werten und Abhängigkeiten. Die Anwendung weist dann diesen Symbolen Werte zu, ohne dass dafür die Implementierung der jeweiligen Systemfunktion verändert werden muss. Aus der aufgelösten Konfiguration entstehen dann anschließend unter anderem Makros, auf die der Übersetzungsprozess achten kann (Zephyr Project, 2024, „Setting Kconfig configuration values“).

Für den Versuchsaufbau ist vor allem die Trennung zwischen gemeinsamer Anwendungskonfiguration und weiteren Fragmenten relevant. `CONF_FILE` legt dabei die benutzte Anwendungskonfiguration fest. `EXTRA_CONF_FILE` erweitert weitere Konfigurationsdateien. Auf diesem Mechanismus basiert die Variation der Profile in Kapitel 4.4 (Zephyr Project, o. D., „Application Development“, Abschnitt „Important Build System Variables“).

Sichtbare und unsichtbare Symbole Die Zephyr-Dokumentation unterteilt die Konfigurationsoptionen in „sichtbare“ und „unsichtbare“ Symbole. Sichtbare Symbole sind über einen Prompt-Text definiert, erscheinen in interaktiven Konfigurationswerkzeugen und können in Konfigurationsdateien gesetzt werden. Unsichtbare Symbole werden ohne einen Prompt definiert. Deren Wert ergibt sich nur aus Vorgabewerten oder über `select`-Beziehungen anderer Symbole.

Zuweisung in Konfigurationsdateien Sichtbare Symbole werden in Konfigurationsdateien über `CONFIG_<Symbolname>=<Wert>` gesetzt. Dabei dürfen um das Gleichheitszeichen keine Leerzeichen stehen. Boolesche Symbole werden zudem über `y` bzw. `n` aktiviert oder deaktiviert. Ein auf `n` gesetztes Symbol kann auf der anderen Seite auch als Kommentar in der Form `# CONFIG_<Symbolname> is not set` geschrieben werden. Genau dieses Format befindet sich auch in der zusammengeführten Konfigurationsdatei `zephyr/.config`.

Die initiale Konfiguration Laut der Zephyr-Dokumentation entsteht die initiale Konfiguration einer Anwendung durch Zusammenführen von drei Quellen:

- Die board-spezifische `<BOARD>_defconfig`-Datei
- Die CMake-Cache-Einträge, die mit `CONFIG_` beginnen
- Die Anwendungskonfiguration selbst, standardmäßig `prj.conf`

Ist anstelle von `prj.conf` die Variable `CONF_FILE` gesetzt, werden die dort angegebenen Konfigurationsdateien zusammengeführt und als Anwendungskonfiguration benutzt. Jede Datei dieser Art wird dabei als „Kconfig-Fragment“ bezeichnet. `CONF_FILE` kann unter anderem direkt über `-DCONF_FILE=<Datei(en)>` beim Aufruf von `west build` gesetzt werden. Wird ein Symbol sowohl in der `<BOARD>_defconfig` als auch in der Anwendungskonfiguration zugewiesen, hat laut Zephyr-Dokumentation der Wert Vorrang, der in der Anwendungskonfiguration gesetzt wurde. Das Ergebnis der Zusammenführung wird als `zephyr/.config` im Build-Verzeichnis gespeichert.

Konfigurieren eines choice Ein choice, also mehrere Symbole, von denen laut Zephyr-Dokumentation zu jedem Zeitpunkt genau eines aktiv sein kann, lässt sich konfigurieren, indem eines seiner Symbole in einer Konfigurationsdatei auf `y` gesetzt wird. Dadurch werden automatisch alle restlichen Symbole des choice auf `n` gesetzt. Wenn mehrere Symbole vom selben choice auf `y` gesetzt werden, dann wird laut Zephyr-Dokumentation nur das zuletzt gesetzte berücksichtigt, während die restlichen auf `n` zurückgesetzt werden.

CMake und Zephyr-Module

Build-System CMake beschreibt den Aufbau der Firmware über Targets und deren Abhängigkeiten. Für Anwendungscode stellt Zephyr dabei vor allem das Bibliotheks-Target `app` bereit. Die dazugehörigen Quelldateien werden bspw. mit `target_sources(app PRIVATE ...)` zugeordnet. `PRIVATE` grenzt dabei diese Quellenzuordnung auf das betreffende Target ein. Mit `PUBLIC` kann sie zudem Teil seiner Schnittstelle für abhängige Targets werden (Zephyr Project, 2024, „Build System (CMake)“).

Aus diesen Beschreibungen erzeugt CMake zuerst die Bauvorschriften für das ausgewählte Host-Werkzeug, bspw. Ninja oder Make. Dieses führt dann anschließend die Übersetzungs- und Linkschritte aus. Dadurch sind die Konfiguration des Bauprozesses und seine eigentliche Ausführung voneinander getrennt (Zephyr Project, 2024, „Build System (CMake)“, Abschnitt „Configuration Phase“).

Externe Projekte als Module Um auf den Quellcode externer Projekte zurückzugreifen, benutzt Zephyr das Konzept der „Module“. Ein Modul muss dafür über eine Datei `module.yml` verfügen, durch welche das Zephyr-Build-System den Quellcode aus dem betreffenden Repository einbindet. Laut Zephyr-Dokumentation soll ein Modul dabei kein Code sein, der nur für Zephyr geschrieben wurde. Solcher Code sollte direkt in den Zephyr-Baum eingebaut werden.

Technische Einbindung Die Build-System-Variable `ZEPHYR_MODULES` ist eine CMake-Liste absoluter Pfade zu den Verzeichnissen, die Zephyr-Module enthalten. Diese Module enthalten `CMakeLists.txt`- und `Kconfig`-Dateien, welche beschreiben, wie sie gebaut bzw. konfiguriert werden. Die `CMakeLists.txt`-Dateien der Module werden dabei dem Build über CMakes `add_subdirectory()`-Befehl hinzugefügt. Die `Kconfig`-Dateien werden in den `Kconfig`-Menübaum eingebunden. Wenn West installiert und `ZEPHYR_MODULES` nicht gesetzt ist, dann findet das Build-System die Module selbst. Es ruft dabei `west list` auf, um die Pfade aller Projekte im West-Arbeitsbereich zu erhalten. Anschließend prüft es für jedes Projekt, ob eine `module.yml`-Datei vorhanden ist. Wenn dem so ist, dann wird ihr Inhalt zur Einbindung verwendet. Fehlt jedoch diese Datei, dann prüft das Build-System, ob im Projekt eine `zephyr/CMakeLists.txt` und eine `zephyr/Kconfig` vorhanden sind. Wenn beide vorhanden sind, dann gilt das Projekt ebenfalls als Modul.

West

West hat im Zephyr-Umfeld zwei miteinander verbundene Aufgaben. Seine eingebauten Kommandos verwalten die Repositories, die zu einem Arbeitsbereich gehören und deren im Manifest festgelegte Revisionen. Zudem ergänzen Erweiterungskommandos projektspezifische Funktionen. Zephyr benutzt diesen Mechanismus unter anderem für das Bauen, Flashen und Debuggen von Anwendungen (Zephyr Project, 2023, „West (Zephyr’s meta-tool)“).

Für diese Untersuchung ist vor allem wichtig, dass die Zephyr-Version und die dazugehörigen Module zusammen über das Projektmanifest festgelegt werden können. Dadurch lässt sich der Softwarestand nachvollziehbar beschreiben, der für eine Messreihe vorgesehen ist.

Aufrufsyntax Wie bei git oder docker lautet ein Aufruf von West: `west [allgemeine-Optionen] <Kommando> [Optionen] <Argumente>`. Seit Version 0.8 lässt sich West auch über `python3 -m west [...]` aufrufen. Über `west --help` bzw. `west <Kommando> -h` zeigt das Werkzeug Hilfetexte zu den verfügbaren bzw. zu einem bestimmten Kommando.

Unterstützte Topologien Die West-Dokumentation beschreibt drei unterstützte Arbeitsbereich-Topologien:

- **T1:** „StarTopologie. Zephyr selbst ist das Manifest-Repository. Dies ist die Standardeinstellung, analog zu Git-Submodulen mit Zephyr als Super-Projekt
- **T2:** „StarTopologie. Eine Zephyr-Anwendung ist das Manifest-Repository, was nützlich ist für Projekte, die sich auf eine einzelne Anwendung konzentrieren
- **T3:** Wald-Topologie mit einem eigenen, quellcodefreien Manifest-Repository für mehrere unabhängige Anwendungen

T2-Topologie als Projektaufbau Ein Repository, das eine Zephyr-Anwendung enthält, fungiert als zentrales Repository und nennt in seiner eigenen `west.yml` die weiteren für den Build benötigten Projekte, inklusive des Zephyr-Repositorys selbst und aller Module. Die West-Dokumentation zeigt dazu ein Beispiel-Manifest, das über den Schlüssel `import: true` beim Projekt `zephyr` und den Schlüssel `self: path: application` genau dieses Muster umsetzt. Der Vorteil des import-Mechanismus liegt laut West-Dokumentation darin, dass die Revisionen der importierten Module nicht getrennt nachverfolgt werden müssen, sondern automatisch aus der Revision der `zephyr/west.yml` bestimmt werden.

Der manifest-rev-Branch West legt in jedem Projekt einen Git-Branch namens `manifest-rev` an und verwaltet diesen dann selbst. Dieser zeigt zudem auf die Revision, die das Manifest beim letzten Aufruf von `west update` für das jeweilige Projekt angegeben hat. Die West-Dokumentation warnt jedoch davor, diesen Branch manuell zu verändern, da West ihn beim nächsten Update zurücksetzt.

Das Manifest-Kommando Über `west manifest --path` kann der Pfad zur obersten Manifest-Datei ausgegeben werden. Genau dieser Befehl wird zur Kontrolle nach der Einrichtung des Arbeitsbereichs benutzt.

Build-System-Werkzeuge zur Analyse des Speicherbedarfs

Das Zephyr-Build-System bietet in der, bei dieser Untersuchung verwendeten Version v3.7.2 (Zephyr-Dokumentation: v3.7.0) genau drei Targets zur Analyse des Speicherbedarfs: `ram_report`, `rom_report` und `puncover`, welches um `pahole` zur Analyse von Datenstrukturen ergänzt wird.

Grundprinzip Das Build-System bietet laut Zephyr-Dokumentation drei Targets, um RAM-, ROM- und Stack-Nutzung in erzeugten Firmware-Abbildern zu betrachten und zu analysieren. Die Werkzeuge arbeiten auf dem fertigen Abbild und liefern Informationen über die Größe von Symbolen und Code sowohl im RAM als auch im ROM. Symbole, die sich nicht anhand von Metadaten einer Quelldatei zuordnen lassen, werden dabei in zwei Sammelkategorien ausgewiesen. `Hidden` bezeichnet ein Symbol ohne zugeordnete Datei. `No paths` sagt aus, dass der Funktionsname bekannt, aber kein eindeutiger Pfad im Verzeichnisbaum auffindbar ist.

Target `ram_report` Dieses Target listet alle kompilierten Objekte und ihre RAM-Nutzung in Form einer Tabelle auf, mit Byte-Angabe und Prozentanteil pro Symbol. Die Daten werden dabei basierend auf den Speicherort des Objekts im Verzeichnisbaum und der Datei, die das Symbol enthält, gruppiert. Aufgerufen wird das Target laut Zephyr-Dokumentation nach einem vorherigen Build über `west build -t ram_report`.

Target `rom_report` Dieses Target funktioniert analog zu `ram_report`, listet jedoch die ROM-Nutzung (Flash-Speicher) statt der RAM-Nutzung auf. Der Aufruf läuft über `west build -t rom_report`.

Target `puncover` Dieses Target verwendet das gleichnamige Drittanbieter-Werkzeug, das nach dem Bauen einen lokalen Webserver startet, über den sich die Dateien des Projekts, inklusive ihrer ROM-, RAM- und Stack-Nutzung, im Browser interaktiv durchsuchen lassen. Für eine Worst-Case-Stack-Nutzungsanalyse muss dafür noch `CONFIG_STACK_USAGE=y` gesetzt werden. Das Werkzeug muss zudem vor der Nutzung getrennt per `pip3` installiert werden und wird von der Zephyr-Dokumentation als Drittanbieter-Werkzeug mit möglicherweise wechselnder Funktionsfähigkeit notiert.

Werkzeug `pahole` Bei `pahole` handelt es sich um ein Werkzeug zur Analyse von Objektdateien, das die Größe von Datenstrukturen und, durch Alignment auf die Wortbreite der CPU entstehende „Löcher“ in diesen Strukturen zeigt. Es muss getrennt installiert werden, z. B. über das Paket `dwarves` unter Ubuntu. Jedoch ist es für diese Untersuchung nicht weiter relevant, da hier keine Datenstruktur-Layout-Optimierung im Fokus steht.

2.6 Benchmark Metriken

In den vorherigen Abschnitten wurden die Grundlagen zu Echtzeitsystemen, eingebetteten Systemen und das in dieser Arbeit verwendeten Zephyr RTOS definiert. Zudem wurden Netzwerk- und Sicherheitsprotokolle erläutert. In diesem Abschnitt werden die fünf in dieser Untersuchung gemessenen benchmark-Metriken definiert. Diese Definitionen sind bewusst basierend auf der allgemeinen Literatur zu Echtzeitsystemen definiert, damit sie auch

unabhängig von einer bestimmten RTOS-Implementierung gültig sind. Die Implementierung dieser Metriken innerhalb von Zephyr wird in Kapitel 4.5 und 5 erläutert.

2.6.1 Context Switch

Kontext & Kontextwechsel Eine unterbrochene Aufgabe muss bei ihrer späteren Fortsetzung auf ihren vorherigen Zustand zurückgreifen können. Zu diesem Kontext gehören dabei vor allem Registerinhalte, der Programmzähler und ggf. weitere aufgabenspezifische Informationen zum Zustand. Diese werden bspw. auf einem Stack gesichert (Laplante & Ovaska, 2012, S. 88–89). Beim Wechsel zu einer anderen Aufgabe wird der Kontext gesichert und der für die nächste Aufgabe gebrauchte Zustand wiederhergestellt. Dieser Verwaltungsvorgang wird Kontextwechsel genannt. Seine Dauer ist Teil der Antwortzeit, ohne die eigentliche Anwendungsaufgabe voranzubringen. Deshalb sollte die Sicherung auf die Informationen begrenzt werden, die für eine korrekte Fortsetzung gebraucht werden (Laplante & Ovaska, 2012, S. 88–89). Abschnitt 2.5.1 erläutert, wann Zephyr einen anderen Thread auswählen kann. Der allgemeine Kontextwechsel beinhaltet das Sichern des für die Fortsetzung gebrauchten Zustands des bisherigen Threads und das Wiederherstellen des Zustands des nächsten Threads (Laplante & Ovaska, 2012, S. 88–89; Zephyr Project, 2024, „Scheduling“).

2.6.2 Interrupt Latency

Interrupt-Latenz Zwischen einer Interrupt-Anforderung und dem Beginn der zugehörigen Interrupt-Service-Routine kann Zeit vergehen. Dieses Intervall wird hierbei als Interrupt-Latenz bezeichnet. Für Echtzeitsysteme ist vor allem relevant, welche Verzögerung unter schlechten Bedingungen auftreten kann (Laplante & Ovaska, 2012, S. 396–397).

Die Latenz wird sowohl durch die Hardware als auch durch den aktuellen Systemzustand beeinflusst. Bspw. kann eine softwareseitige Interrupt-Sperre die Behandlung verzögern. Auch die Bedingungen für die Unterbrechung des gerade ausgeführten Maschinenbefehls spielen eine Rolle. Deshalb ist die Interrupt-Latenz nicht allein eine Eigenschaft der Anwendungsfunktion, die später in der ISR ausgeführt wird (Laplante & Ovaska, 2012, S. 396–397).

Abschnitt 2.5.1c beschreibt die Interrupt-Infrastruktur von Zephyr. Wie die Interrupt-Anforderung auf der Zielplattform softwareseitig über den NVIC ausgelöst und ihre Latenz gemessen wird, wird erst in Kapitel 5.2 behandelt (Zephyr Project, 2024, „Interrupts“).

2.6.3 Timer-Jitter

Pünktlichkeit & Jitter Pünktlichkeit beschreibt bei Laplante und Ovaska, wie nah die Antwortzeiten an einen vorgesehenen Antwortzeitpunkt liegen. Für eine Anwendung können dabei sowohl zu frühe als auch zu späte Reaktionen relevant sein. Abweichungen entstehen unter anderem durch die zeitlichen

Eigenschaften der Hardware- und Softwarekomponenten (Laplante & Ovaska, 2012, S. 8–9).

Wenn die Antwortzeiten zwischen wiederholten Vorgängen schwanken, dann wird diese zeitliche Variation Jitter genannt. Für periodische Vorgänge kann das Konzept auf die Abstände zwischen aufeinanderfolgenden Ausführungen übertragen werden. In dieser Arbeit ist die festgelegte Timerperiode der Bezugspunkt für die Untersuchung von solchen Intervallen. Die Messgröße und ihre Erfassung werden in Kapitel 5.3 erläutert (Laplante & Ovaska, 2012, S. 8–9).

Auf den periodischen Zephyr-Timer übertragen bedeutet Jitter in dieser Arbeit die Abweichung des gemessenen Abstands zwischen zwei Callback-Messpunkten von der vorgesehenen Timerperiode. Die bestimmte Messgröße wird in Kapitel 5.3 festgelegt (Laplante & Ovaska, 2012, S. 8–9; Zephyr Project, 2024, „Timers“).

2.6.4 Heap-Allokation/ Fragmentierung

Dynamische Speicherverwaltung Bei der dynamischen Speicherverwaltung wird Speicher während der Ausführung angefordert und wieder freigegeben. In Echtzeitsystemen muss dabei sowohl die verfügbare Kapazität berücksichtigt werden als auch der Aufwand der Verwaltung und das Verhalten bei fehlendem Speicher. Laplante und Ovaska verbinden die Speicherverwaltung deshalb mit der Vorhersagbarkeit des Gesamtsystems (2012, S. 127–132).

Für diese Untersuchung sind zwei Aspekte relevant, darunter die Dauer von einzelnen Allokations- und Freigabeoperationen und die Verteilung des verfügbaren Speichers nach wiederholten Anforderungen und Freigaben. Diese werden in getrennten Benchmarks untersucht.

Fragmentierung Wenn Speicher mehrmals zugewiesen und wieder freigegeben wird, kann der verfügbare Speicherbereich aus belegten und freien Abschnitten bestehen. Wenn es insgesamt genug freien Speicher gibt, eine Anfrage jedoch nicht bearbeitet werden kann, weil es keinen zusammenhängenden Block der angeforderten Größe gibt, dann nennt man dies externe Fragmentierung. Auf der anderen Seite kann eine Anfrage einen größeren, fest vorgegebenen Speicherblock zugewiesen bekommen, als gebraucht wird. Der übrigbleibende Teil wird dabei nicht benutzt. Dies wird interne Fragmentierung genannt. Dabei kann eine Kompaktierung des Speichers eine externe Fragmentierung beheben. Dies ist jedoch selbst rechenintensiv und ist daher in harten Echtzeitsystemen unpraktisch. (Laplante & Ovaska, 2012, S. 131–132) In Abschnitt 2.5.1e ist die Speicherverwaltung von Zephyr über `k_heap` bzw. dem System-Heap durch `k_malloc()` und `k_free()` erklärt. Dabei handelt es sich um eine Umsetzung dieser Konzepte.

2.6.5 RAM/ ROM

Speicherklassen Halbleiterspeicher kann in zwei Klassen eingeteilt werden. Dabei handelt es sich um flüchtigen Lese-Schreib-Speicher (Random Access Memory, RAM) und nichtflüchtigen Speicher (Read-Only Memory, ROM).

Klassischer ROM lässt sich dabei nur bei der Herstellung beschreiben. Auf der anderen Seite gibt es Vertreter wie EEPROM und Flash, welche auch im eingebauten Zustand neu beschreiben werden können. Dies passiert jedoch langsamer als bei RAM und auch nur für eine begrenzte Anzahl an Schreibzyklen. In eingebetteten Systemen wird ROM/ Flash daher oft zum Ablegen von Programmcode benutzt, während RAM für Stack und Laufzeitdaten reserviert bleibt. (Laplante & Ovaska, 2012, S. 36-37)

Speicher-Fußabdruck Der Speicherbedarf kann für verschiedene Bereiche getrennt betrachtet werden, bspw. für Programmcode, Daten und Stacks. Dabei wird zwischen der absoluten Belegung und ihrem Anteil an der verfügbaren Kapazität unterschieden. Teile des Programmspeicherbedarfs können hierbei schon aus dem erzeugten Programmabbild bestimmt werden. Auf der anderen Seite hängt die wahre Stack-Auslastung noch vom Ausführungsverlauf ab, bspw. von verschachtelten Funktionsaufrufen und Interrupts (Laplante & Ovaska, 2012, S. 408–409).

Daher reicht für die Untersuchung der verfügbaren Ressourcen nicht nur die Gesamtsumme. Ein einzelner Speicherbereich kann seine Kapazität nämlich erreichen, obwohl an anderer Stelle noch Speicher frei ist. In dieser Arbeit wird der statische RAM-/ROM-Fußabdruck anhand der erzeugten Firmware-Abbilder untersucht. Diese Auswertung soll dabei von einer Messung der echten Stack- oder Heap-Auslastung während des Betriebs unterschieden werden (Laplante & Ovaska, 2012, S. 408–409).

In Abschnitt 2.5.2d sind die Werkzeuge `ram_report` und `rom_report` vom Zephyr-Build-System erwähnt worden. Diese benutzen genau diese Art der Speicher-Fußabdruck-Analyse für fertig gebaute Firmware-Abbildungen um.

2.7 Netzwerk- und Sicherheitskonzepte

Die untersuchte IoT-Stack-Konfiguration beinhaltet OpenThread, 6LoWPAN, CoAP und DTLS mit Pre-Shared Keys. MCUmgr und MCUboot gehören auf der anderen Seite nur zur getrennten Demonstrationsanwendung `iot_sensor`. Die dafür relevanten Netzwerk-, Sicherheits- und Updatekonzepte werden in diesem Abschnitt erläutert.

2.7.1 Netzwerk-Stack und CoAP

Das Constrained Application Protocol (CoAP) ist ein eigenes, für ressourcenbeschränkte Geräte gemachtes Web-Übertragungsprotokoll. Dabei folgt es demselben Anfrage-Antwort-Modell wie HTTP, darunter auch Methoden wie GET, POST, PUT und DELETE. CoAP benutzt UDP und erweitert dies mit eigenen Nachrichtentypen und optionale Zuverlässigkeitsmechanismen. Confirmable-Nachrichten werden bestätigt und bei ausbleibender Bestätigung neu übertragen. Non-confirmable-Nachrichten verlangen hierbei keine solche Bestätigung (Shelby et al., 2014, Abschnitte 2 und 4). Zudem kann CoAP über eine gesicherte Variante, dem URI-Schema „coaps“, laufen, bei welcher Nachrichten durch DTLS abgesichert werden (Shelby et al., 2014)

(siehe 2.7.2). Bei der von Silva et al. (2019, S. 10376) beschriebenen Netzwerk-Stack-Architektur bildet CoAP dabei die Anwendungsschicht, während UDP, IPv6 und die 6LoWPAN-Adaptionsschicht (siehe 2.7.3) die Schichten darunter übernehmen.

2.7.2 Transportverschlüsselung durch TLS/ DTLS und Pre-Shared Keys

Transport Layer Security (TLS) ist ein verbreitetes Protokoll zur Absicherung von Netzwerkverbindungen. Es braucht jedoch einen zuverlässigen, verbindungsorientierten Transportkanal wie TCP. Da CoAP auf dem verbindungslosen UDP basiert, wird daher Datagram Transport Layer Security (DTLS) benutzt. Dabei handelt es sich um eine Variante, die bewusst so nah wie möglich an TLS gehalten ist, dafür aber über weitere Mechanismen für den Umgang mit verlorenen oder vertauschten Paketen verfügt (Rescorla und Modadugu, 2012). DTLS nutzt dabei für die Absicherung Kryptographie-Verfahren, die durch sogenannte Cipher Suites festgelegt werden. Für ressourcenbeschränkte Geräte ist dabei eine Absicherung über einen davor ausgetauschten, gemeinsamen Schlüssel (Pre-Shared Key, PSK) verbreitet, da dafür kein aufwendiger Zertifikats- oder Schlüsselaustausch zur Laufzeit gebraucht wird. Die Cipher Suite TLS_PSK_WITH_AES_128_CCM_8 ist dabei die PSK-Absicherungsvariante, die für Constrained-Geräte unterstützt werden muss (Shelby et al., 2014). Dabei gibt es sowohl einen Nach- als auch einen Vorteil. Die in dieser Arbeit benutzte Cipher Suite TLS_PSK_WITH_AES_128_CCM_8 bietet keine Perfect Forward Secrecy, besitzt dafür aber einen vergleichsweise kleinen Speicherbedarf und eine für ressourcenbeschränkte Geräte passende Ausführungseffizienz (Tschofenig & Fossati, 2016, Abschn. 9). Lee et al. implementierten für Zephyr unter anderem PSK_WITH_AES128_CCM8 und ECDHE_ECDSA_WITH_AES128_CCM8. Ihre Hardwaretests benutzen ein FRDM-K64F mit Cortex-M4, 1 MiB Flash und 256 KiB RAM. Trotz gleicher Speichergrößen ist dieses Board wegen weiterer Hardware- und Softwareunterschiede keine direkt vergleichbare Messplattform (Lee et al., 2018, S. 1292–1294).

2.7.3 Thread, 6LoWPAN und IEEE 802.15.4

IEEE 802.15.4 ist ein Standard für stromsparende Funknetze mit niedriger Datenrate. Dieser Standard erlaubt dabei nur kleine Nutzlasten pro Übertragungsrahmen. Diese sind kleiner als die von IPv6 mindestens geforderte Paketgröße. Daher wird eine weitere Zwischenschicht gebraucht, die IPv6-Pakete auf mehrere kleine 802.15.4-Rahmen aufteilt und dabei die IPv6-Header komprimiert, da sie sonst zu viel Platz verbrauchen würden. Diese Zwischenschicht wird 6LoWPAN genannt (Montenegro et al., 2007). Thread ist hierbei ein darauf aufbauendes, IPv6-basiertes Mesh-Netzwerkprotokoll, das genau diese 6LoWPAN-Kompression über IEEE-802.15.4-Funkverbindung benutzt, um ein IPv6-basiertes Mesh-Netz aus ressourcenbeschränkten Geräten zu bilden (Zephyr Project, 2024, „Thread protocol“). Damit aber ein solches Thread-Netz mit dem klassischen Internet kommunizieren kann, wird ein weiterer

sogenannter Border Router gebraucht, welcher zwischen dem Thread-Mesh und einem externen IP-Netz vermittelt (Zephyr Project, 2024, „Thread protocol“). Die für diese Arbeit ausgesuchte IoT-Stack-Konfiguration benutzt dabei das nRF52840 DK als sogenanntes Minimal Thread Device (MTD), also als ressourcenschonenden Netzknoten ohne eigene Routing-Funktion innerhalb vom Mesh (siehe 4.4.3).

2.7.4 Firmware-Updates: MCUmgr und MCUboot

MCUboot verwaltet Firmware-Slots und kann dabei entsprechend konfigurierte und signierte Firmware-Abbilder vor dem Start prüfen. MCUmgr stellt Managementfunktionen bereit, über die neue Images übertragen und verwaltet werden können. Zephyr unterstützt dafür getrennte SMP-Transporte über Bluetooth LE, serielle Verbindungen und UDP. In der Demonstrationsanwendung dieser Arbeit wird UDP über den vorhandenen IPv6-/ Thread-Stack verwendet (Zephyr Project, 2024, „MCUmgr“ und „Device Firmware Upgrade“).

Verwandte Arbeiten

Im dritten Kapitel wird der aktuelle Stand der Forschung zum Thema RTOS-Benchmarking dargestellt und diese Arbeit darin eingeordnet. Zuerst wird erläutert, welche Herausforderungen sich beim Vergleich und der Bewertung von Echtzeitsystemen ergeben. Darunter werden auch Fragen zur Reproduzierbarkeit und Vergleichbarkeit zu anderen Betriebssystemen wie FreeRTOS, RIOT oder Contiki-NG bzgl. Speicherverbrauch und Laufzeitverhalten untersucht. Danach werden Zephyr-spezifische Arbeiten behandelt, darunter Zephyrs eigene Referenz-Benchmarks und Studien, die Zephyr in bestimmten Anwendungsszenarien, wie bspw. die Absicherung durch TLS/ DTLS oder KI-Inferenz, einsetzen und teilweise messen. Zuletzt wird diese Arbeit von den vorgestellten Studien abgegrenzt, indem analysiert wird, worin sich ihre Methodik, also der Vergleich von Konfigurationsstufen innerhalb desselben RTOS auf einer spezifizierten Zielplattform, von den anderen bestehenden Vergleichsstudien unterscheidet.

3.1 RTOS-Benchmarking allgemein

Das Benchmarking von Echtzeitbetriebssystemen besteht in der Literatur aus einer Kombination von Laufzeit- und Speichermetriken. Silva et al. (2019) definieren für den Vergleich von Contiki-NG, RIOT und Zephyr vier Bewertungsdimensionen, darunter Leistung, Zeit-Determinismus, Speicher-Fußabdruck und Energieverbrauch. Für die Leistungs-Bewertung benutzen Silva et al. die Thread-Metric Benchmark Suite. Sie besteht aus acht Tests, darunter kooperative und präemptive Kontextwechsel, Interrupt-Verarbeitung, Semaphore-Operationen, Speicherallokation und -freigabe (Silva et al., 2019, S. 10379). Die Zeitvorhersagbarkeit untersuchen die Autoren zudem anhand eigener Mikrobenchmarks für ausgewählte Thread-Verwaltungsfunktionen. Darüber hinaus beziehen sie einen Thread-Metric-Test ein (Silva et al., 2019, S. 10380). Diese Sammlung an Benchmark-Kategorien ähnelt den in dieser Arbeit gewählten fünf Metriken. Hierbei wurde in dieser Arbeit jedoch nicht die Thread-Metric Benchmarks-Suite verwendet, was genauer in Abschnitt 3.4 begründet wird. Für die Messung des Zeit-Determinismus benutzen Silva et al. (2019, S. 10379) eigenentwickelte Mikrobenchmarks für vier Thread-Verwaltungs-APIs, darunter „Thread Create“, „Resume“, „Suspend“ und „Delete“. Dabei protokollieren sie Minimum, Maximum und Mittelwert als Jitter-Indikator. Auch Leotescu und Vlad (2018) benutzen Mikrobenchmarks in ihrem Ansatz. Sie entwickeln mit „heap_bench“, „thread_bench“, „mutex_bench“ und „context_switch“ vier eigene, in C geschriebene Testprogramme, die jeweils eine

einzelne Kernel-Operation isoliert messen (Leotescu & Vlad, 2018, S. 3-7). Methodisch ähnelt dies hierbei dem Aufbau dieser Arbeit, wobei hier ebenfalls jede Metrik in einer eigenen, isolierten `bm_*.c`-Datei gemessen wird (siehe Kapitel 4.1). Außerdem benutzen Leotescu und Vlad (2018, S. 3) für die Zeitmessungen unter Zephyr bewusst die Funktion `TIMING_INFO_PRE_READ()` anstatt der POSIX-Funktion `clock_gettime()`, mit der Begründung, dass die POSIX-Funktion aufgrund des Zephyr-Schedulers inkonsistente Werte liefere. Auch diese Arbeit benutzt mit `timing_counter_get()` eine Zephyr-spezifische Zeitmessschnittstelle anstatt einer POSIX-Zeitabfrage. Dies entspricht der allgemeinen Entscheidung von Leotescu und Vlad für eine hardwarenahe Zephyr-Zeitbasis. Ihr dabei verwendetes Makro `TIMING_INFO_PRE_READ()` soll jedoch mit der hier eingesetzten API unterschieden werden (Leotescu & Vlad, 2018, S. 3).

Jaskani et al. (2019) ergänzen dabei diese beiden messtechnisch orientierten Arbeiten um eine breitere, qualitative Einordnung. Sie erwähnen nämlich sieben Auswahlkriterien für ein IoT-Betriebssystem, darunter Architektur, Programmiermodell, Scheduling-Strategie, Speicher-Fußabdruck, Vernetzung, Portabilität und Energieeffizienz (Jaskani et al., 2019, S. 3-4). Zudem ordnen sie Zephyr darin als Mikrokern- / Nanokern-Hybrid mit präemptiven, prioritätsbasierten und EDF-fähigem Scheduler ein (Jaskani et al., 2019, S. 5). Diese Einordnung wird jedoch in dieser Arbeit nicht selbst empirisch nachgeprüft, da der Fokus hier nicht auf einem Architektur-, sondern Konfigurationsvergleich liegt.

3.2 Bestehende RTOS-Benchmarks

Silva et al. benutzen für die betreffende Leistungs-Auswertung 1000 Stichproben. Damit liegt der Stichprobenumfang in derselben Größenordnung wie bei den zeitbezogenen Benchmarks dieser Arbeit, ohne dass daraus schon eine Gleichheit der Messbedingungen folgt (Silva et al., 2019, S. 10379).

Für den untersuchten Zephyr-Aufbau berichten die Autoren außerdem einen mittleren RAM-Bedarf von 52,9 KB und einen Flash-Bedarf von 130,5 KB. Die Werte für Contiki-NG betragen 29,8 KB bzw. 50,6 KB und für RIOT 33,0 KB bzw. 59,9 KB. Diese Zahlen basieren dabei auf die jeweils untersuchten Firmware-Aufbauten inklusive Anwendung und Netzwerkfunktionen und nicht auf einen allgemeinen Mindestbedarf der Betriebssysteme (Silva et al., 2019, S. 10381).

Bei den untersuchten Leistungs- und Zeitvorhersagbarkeits-Metriken dokumentieren Silva et al. schlechtere Ergebnisse für Zephyr als für die beiden Vergleichssysteme. Sie diskutieren dabei, dass weitere Verarbeitungs- und Moduswechsel eine mögliche Erklärung sein können. Auf der anderen Seite notieren sie jedoch, dass der Zeitstempel in ihrem Messaufbau nicht immer am vorgesehenen Endpunkt erfasst wird. Die beobachteten Schwankungen dürfen daher nicht unbegrenzt als den realen Jitter der untersuchten API-Operationen interpretiert werden (Silva et al., 2019, S. 10381).

Für diese Arbeit ist daraus vor allem die Bedeutung von einer klaren Definition und Überprüfung der Messpunkte relevant. Die Ergebnisse von Silva et

al. dienen als Vergleichskontext, ersetzen aber dabei keine Validierung der Zeitmessung, die hier eingesetzt werden.

Für diese Arbeit ist dieses Erkenntnis auf zwei Arten relevant. Erstens zeigt er, dass Zephyrs Fußabdruck und Determinismus-Verhalten stark von der aktivierten Konfiguration abhängen können. Genau dieser Zusammenhang wird in dieser Arbeit durch die drei Konfigurationsstufen untersucht. Silva et al. beschreiben die Bedingungen ihres Versuchsaufbaus und benutzen dabei Standard- bzw. empfohlene Einstellungen. Eine komplette Aufstellung aller eingeschalteten Zephyr-Konfigurationssymbole wird jedoch nicht angegeben. Zudem werden innerhalb der Untersuchung unter anderem Thread- und Netzwerkbedingungen variiert. Der Vergleich ist deshalb nicht mit einem gezielten Vergleich von den drei hier definierten Konfigurationsprofile gleichzusetzen (Silva et al., 2019, S. 10378–10380).

Die veröffentlichten Werte machen eine Einordnung der Größenordnung möglich. Unterschiede in Hardware, Softwarestand, Anwendung und Messverfahren begrenzen dabei jedoch den direkten Vergleich der Zahlen. Daher bestätigt eine ähnliche Größenordnung allein nicht direkt die Richtigkeit der eigenen Messmethode.

Altoumi und Ahmed beziehen sich dabei UL Solutions Kontextwechselwerte von 223 CPU-Zyklen für FreeRTOS und 524 CPU-Zyklen für Zephyr. Diese Zahlen sind in diesem Kontext keine eigenen Messwerte der Autoren. Sie diskutieren außerdem die Bedeutung von unterschiedlichen MPU- bzw. Sicherheitseinstellungen für die Vergleichbarkeit von solchen Ergebnissen (UL Solutions, 2022, zitiert nach Altoumi & Ahmed, 2025, S. 2–3; Altoumi & Ahmed, 2025, S. 3).

Für diese Arbeit ist daran vor allem der methodische Hinweis relevant, dass Schutzfunktionen bei Zeitmessungen genau dokumentiert werden sollen. Die Minimal-Konfiguration deaktiviert die MPU. Dadurch beziehen sich die Werte, die ermittelt werden, auf eine reduzierte Schutzkonfiguration. Ohne einen weiteren vergleichbaren Messlauf mit aktivierter MPU kann sich der vermiedene Aufwand, der durch diese Abschaltung entsteht, nicht getrennt quantifizieren.

Jaskani et al. (2019) dient in dieser Arbeit an erster Stelle als Kontext- und Einordnungsquelle, nicht als Messreferenz. Das liegt daran, dass das Paper selbst keine eigenen Benchmarks durchführt. Stattdessen fasst sie bestehende Literatur zu sechs IoT-Betriebssystemen, darunter Contiki, TinyOS, RIOT, Zephyr, Mbed und Brillo zusammen (Jaskani et al., 2019, S. 5-6).

3.3 Zephyr-spezifische Benchmarks

Ergänzend zu den in 3.2 vorgestellten RTOS-übergreifenden Vergleichsstudien bringt Zephyr selbst eigene Referenz-Benchmarks im Quellbaum mit, der unter `tests/benchmarks/` liegt. Diese Benchmarks liefern Zephyr-interne Referenzmessungen für ausgewählte Kerneloperationen. In dieser Arbeit werden ihre dokumentierten Messkategorien als Nachvollziehbarkeits- und Abgrenzungspunkte benutzt. Eine unabhängige Validierung der eigenen Messmethodik entsteht daraus allein jedoch noch nicht (Zephyr Project, 2024,

tests/benchmarks/latency_measure/README.rst). Im Folgenden wird für jede der fünf untersuchten Metriken notiert, ob ein Zephyr-Äquivalent existiert, was davon übernommen und was bewusst anders gemacht worden ist. Ergänzend dazu werden drei Studien erwähnt, die Zephyr nicht mit anderen RTOS vergleichen, sondern eine Zephyr-Anwendung auf realer Hardware messen.

Kontextwechsel-Latenz Für diese Metrik existiert durch tests/benchmarks/latency_measure ein Zephyr-eigenes Äquivalent, das laut der darin enthaltenen README-Datei unter anderem die Kontextwechselzeit über `k_yield()` misst, sowohl präemptiv als auch kooperativ. Es wird zudem auch die Kontextwechselzeit durch Semaphore-Weckung gemessen. Dieser Benchmark wurde in 5.1 schon als Vergleichspunkt genannt. Wichtig ist hierbei jedoch eine Einschränkung. Der genaue Quellcode von `latency_measure` wurde im Rahmen dieser Arbeit nicht Zeile für Zeile aus dem GitHub-Repository übernommen, sondern nur über die Dokumentation durch die README-Datei eingeordnet. Aus der README werden nur die dort beschriebenen Messkategorien übernommen. Die eigene Kalibrierung des Messaufwands ist davon unabhängig entwickelt worden. Die eigene Auswertung führt Minimum, Maximum, Mittelwert und Stichproben-Standardabweichung laufend über $N=1000$ Wiederholungen. Sie speichert dabei jedoch keine komplette Verteilung der Einzelwerte.

Interrupt-Latenz Auch dies ist in Zephyrs tests/benchmarks/latency_measure vorhanden. Zephyrs Suite erfasst nämlich die Zeit vom Ende der ISR bis zur Fortsetzung eines Threads. Der in dieser Arbeit gemessene Schritt davor, also vom Software-Trigger bis zur ersten ISR, wird von `latency_measure` jedoch nicht erfasst. Diese Arbeit misst eine Zeitgröße derselben Größenordnung, wobei der Hinweg vom Auslösen bis zur ersten ISR-Instruktion statt des Rückwegs gemessen wird. Die README beschreibt für `latency_measure` den Rückweg von einer ISR zum unterbrochenen oder zu einem anderen Thread. Diese Arbeit untersucht auf der anderen Seite den Hinweg vom softwareseitigen Setzen einer NVIC-Anforderung bis zum Zeitstempel am Beginn des eigenen C-Handlers. Aussagen über die bestimmte interne Triggerquelle von `latency_measure` werden daraus nicht abgeleitet.

Timer-Jitter Im für diese Arbeit geprüften Benchmark-Verzeichnis von Zephyr v3.7 wurde kein Referenzbenchmark gefunden, der die Abweichung der Callback-Intervalle eines periodischen `k_timer` von seiner Sollperiode misst (Zephyr Project, 2024, tests/benchmarks/).

Heap-Allokationsgeschwindigkeit und -Fragmentierung Für Kernel-Operationen gibt es durch tests/benchmarks/sys_kernel ein Zephyr-eigenes Benchmark. Dieses misst jedoch nur Semaphore, LIFO, FIFO, Stack und Memory Slab durch `k_mem_slab_alloc` und `k_mem_slab_free`. Es wird also nicht der allgemeine System-Heap über `k_malloc()` und `k_free()` untersucht. Ein passenderes Zephyr-Äquivalent wäre dafür `latency_measure` selbst, da es durch `heap_malloc.immediate` und `heap_free.immediate` die Heap-Allokation durchführt. Dabei liefert es, laut der README, in der Standardkonfiguration von Zephyr v3.7.0 Referenzwerte von durchschnittlich 627 Zyklen für `k_malloc()` und 432 Zyklen für `k_free()`. Für die Heap-Fragmentierung als Zustandsmaß ist kein Zephyr-eigener Benchmark gefunden worden, da `sys_heap`

selbst keine öffentliche Kennzahl für den größten zusammenhängenden Freiblock bringt (siehe 5.5). Dies ist auch der Grund für die Eigenentwicklung der Bisektions-Probe in dieser Arbeit.

Zephyr in Anwendungsszenarien Leotescu und Vlad untersuchen vier Mikrobenchmarks für Speicher-, Thread-, Mutex- und Kontextwechsel-Operationen. Dabei vergleichen sie Zephyr auf einem i.MX RT1050 mit Cortex-M7 bei 600 MHz mit Linux auf einem i.MX 6UL mit Cortex-A7 bei 528 MHz. Die verschiedenen Hardware begrenzt den direkten Vergleich der Betriebssysteme (Leotescu & Vlad, 2018, S. 1–3).

Für die Zeitmessung unter Zephyr benutzen die Autoren `TIMING_INFO_PRE_READ()` anstatt `clock_gettime()`, weil sie mit der POSIX-Zeitabfrage inkonsistente Messwerte beobachtet haben. Dies unterstreicht hierbei die Bedeutung der ausgewählten Zeitbasis. Auch diese Arbeit benutzt für ihre kurzen Laufzeitmessungen eine Zephyr-spezifische Timing-Schnittstelle. Die Eignung muss jedoch für den eigenen Messaufbau beurteilt werden (Leotescu & Vlad, 2018, S. 3).

In drei wiederholten Benchmark-Läufen notieren die Autoren gleiche zusammengefasste Ergebnisse unter Zephyr. Für den Kontextwechsel nennen sie einen Mittelwert von 47 Zyklen. Unter Linux beobachten sie Abweichungen von bis zu neun Prozent vom jeweiligen Mittelwert. Diese Ergebnisse gelten dabei für den beschriebenen Versuchsaufbau. Sie zeigen jedoch nicht die Verteilung aller einzelnen Operationen. Außerdem zeigen sie auch nicht, dass unabhängige Wiederholungsläufe durch viele Messungen innerhalb eines einzigen Laufs ersetzt werden können (Leotescu & Vlad, 2018, S. 8–9).

Lee et al. untersuchen TLS-/DTLS-Client-Funktionalität auf Zephyr zuerst in QEMU und anschließend auf einem FRDM-K64F-Board. Das Board benutzt einen Cortex-M4 und verfügt über 1 MB Flash und 256 KB RAM. Bei den Hardwaretests notieren die Autoren Kommunikationsprobleme und Paketverluste, die dabei in den vorherigen Simulationstests nicht sichtbar waren (Lee et al., 2018, S. 1293–1294).

Die Arbeit dient hier als Beispiel dafür, dass ein erfolgreicher Simulationstest nicht direkt alle Bedingungen von einem realen Gerät abbildet. Sie ist dadurch eine Kontextquelle für die Hardware der IoT-Anwendung, aber keine direkte Referenzmessung für die untersuchten Kernel-Latenzen.

Zhang et al. portieren Paddle Lite auf Zephyr und vergleichen die Inferenz auf einem RK3568 mit Cortex-A55 mit einer Linux-Ausführung. Für die untersuchten Modelle notieren sie dabei durchschnittlich sieben Prozent geringere Inferenzzeiten und einen um 78 Prozent geringeren Systemspeicherverbrauch während der Inferenz unter Zephyr (Zhang et al., 2026, S. 263–264).

Bei diesen Ergebnissen handelt es sich um einen Anwendungsprozessor und eine KI-Inferenzlast. Sie sind daher nicht direkt auf die Cortex-M4-Plattform und die Kernel-Mikrobenchmarks dieser Arbeit übertragbar. Die Studie zeigt jedoch dafür, dass Zephyr auch für größere Anwendungen als ressourcenschonende Ausführungsumgebung untersucht wird.

3.4 Abgrenzung der eigenen Arbeit

Durch die vorgestellten Arbeiten lässt sich die Position dieser Arbeit ableiten. Vier Aspekte unterscheiden sie von den genannten Studien.

Erstens vergleicht diese Arbeit nicht, wie die meisten der vorgestellten Studien (Silva et al., 2019; Altoumi & Ahmed, 2025; Jaskani et al., 2019), unterschiedliche RTOS-Punkte miteinander. Stattdessen wird ein einzelnes RTOS, nämlich Zephyr, über drei Konfigurationsstufen hinweg, darunter Minimal, Logging und IoT-Stack, verglichen. Diese Methodik beantwortet daher keinen Produktvergleich wie „Welches RTOS ist am schnellsten?“, sondern den Einfluss von Konfigurationsentscheidungen auf Laufzeit- und Speicherverhalten desselben Systems. Silva et al. (2019) und Altoumi und Ahmed (2025) bringen zwar interessante Vergleichswerte zwischen RTOS, sagen aber wenig darüber aus, welcher Teil der beobachteten Unterschiede durch die Konfiguration entsteht, anstatt durch das RTOS-Produkt. Diese Arbeit untersucht genau diese Lücke, indem sie die Konfiguration zur einzigen unabhängigen Variable macht (siehe 4.1).

Zweitens untersucht diese Arbeit Laufzeitmetriken und auch den statischen RAM-/ROM-Fußabdruck für die eigenen Zephyr-Konfigurationsstufen auf einer festgelegten Zielplattform. Die Kombination dieser Metrik-Arten ist nicht neu. Silva et al. achten schon auf Leistung, Zeitvorhersagbarkeit, Speicher und Energieverbrauch. Der weitere Schwerpunkt dieser Untersuchung liegt auf der Variation der Konfiguration innerhalb desselben Betriebssystems (Silva et al., 2019, S. 10378–10382).

Drittens legt diese Arbeit vor allem Wert auf die Nachvollziehbarkeit des eigenen Versuchsaufbaus. Dazu werden die verwendeten Softwarestände, Konfigurationsfragmente, Messpunkte, Kalibrierungsschritte und Auswertungsgrößen dokumentiert. Diese Angaben sollen dabei eine Wiederholung der Untersuchung erleichtern. Die daraus resultierenden Ergebnisse sagen jedoch nicht aus, dass die Dokumentation den Vergleichsstudien überlegen ist.

Viertens dienen Beobachtungen aus der Literatur als Ausgangspunkt für eigene Entwurfsentscheidungen. Die Diskussion von unterschiedlichen Schutzkonfigurationen bei Altoumi und Ahmed motiviert bspw., die MPU-Einstellung bewusst zu dokumentieren. Die Ergebnisse zum Speicherbedarf von Silva et al. sind zudem ein Grund für die Untersuchung des Speicherbedarfs von unterschiedlichen Zephyr-Aufbauten. Diese Arbeit übernimmt dadurch Fragestellungen aus der Literatur, ohne deren Experimente komplett zu kopieren (Altoumi & Ahmed, 2025, S. 2–3; Silva et al., 2019, S. 10381).

Insgesamt ist diese Arbeit als eine vertiefte Einzelfallanalyse eines RTOS und kontrollierter Konfigurationsvariation positioniert, die bestehende Literatur-Beobachtungen behandelt und diese auf einer einzigen, genau spezifizierten Hardware-Plattform empirisch nachprüft.

Methodik und Versuchsaufbau

In diesem Kapitel wird gezeigt, wie die in Kapitel 1 formulierte Forschungsfrage empirisch untersucht wird, bevor in den nachfolgenden Kapiteln die Implementierung und die Ergebnisse folgen. Zuerst wird das grundlegende Forschungsdesign vorgestellt und in die unabhängige, die abhängigen und die kontrollierten Variablen des Versuchsplans unterteilt (4.1). Anschließend werden die verwendete Zielplattform (4.2) und die Softwareumgebung (4.3) beschrieben. Zudem wird die unabhängige variable, die drei Konfigurationen (4.4) und die fünf Laufzeit- und Heap-Benchmarks und den dabei bestimmten statischen RAM-/ROM-Fußabdruck, also die eigentlichen Benchmark-Metriken (4.5) erläutert. Zuletzt wird eine vollständige Übersicht der kontrollierten Variablen (4.6) erstellt, welche über alle Konfigurationsstufen und Metriken hinweg konstant bleibt. Dadurch werden die Reproduzierbarkeit und die Aussagekraft des Konfigurationsvergleichs sichergestellt.

4.1 Forschungsdesign

Die Arbeit verfolgt ein quantitatives und experimentelles Forschungsdesign mit kontrollierter Variation der Zephyr-Konfiguration. Dabei werden wiederholte Messungen auf derselben Hardwareplattform durchgeführt. Dieselbe physische Plattform, ein nrf52840-DK, durchläuft nacheinander drei Firmware-Builds, die sich jeweils in der zu untersuchenden Kernel-/ Stack-Konfiguration unterscheiden. Alle übrigen relevanten Rahmenbedingungen, darunter Hardware, CPU-Taktfrequenz, Zephyr-Version, Compiler und Messverfahren, werden dabei konstant gehalten. Dadurch werden hardwarebedingte Unterschiede zwischen den Messungen minimiert. Zudem können Unterschiede in den Messergebnissen auf die Konfigurationsänderungen zurückgeführt werden, soweit andere Einflussfaktoren kontrolliert werden. Quantitativ bedeutet im Kontext dieser Untersuchung, dass diese anhand von messbaren numerischen Größen durchgeführt wird. Zudem bedeutet experimentell, dass Einflussfaktoren gezielt verändert werden und beobachtet wird, wie sich dadurch eine abhängige Messgröße verändert.

Das Versuchsdesign folgt somit der klassischen Struktur eines kontrollierten Experiments:

- **Unabhängige Variable (UV):** Die aktive Zephyr-Konfiguration (Minimal, Logging, IoT-Stack) ist der zu manipulierende Faktor.
- **Abhängige Variablen (AV):** Die fünf gemessenen RTOS-Leistungsmetriken (Context-Switch, Interrupt Latenz, Timer-Jitter, Heap-Allokationsgeschwindigkeit,

Heap-Fragmentierung), wobei hierbei die Wirkung der Zephyr-Konfigurationen beobachtet wird.

- **Kontrollierte Variablen:** Alle für die Messung relevanten Rahmenbedingungen, die zwischen den Durchläufen konstant gehalten werden, um Reproduzierbarkeit zu ermöglichen. Dazu gehören die verwendete Hardware, CPU-Taktfrequenz, Compiler, Zephyr-Version, Messverfahren, Stichprobenumfang und Umgebung. Die Anzahl der Messwiederholungen und der Messmechanismus für die Datenerhebung für alle Konfigurationen sind ebenfalls festgelegt.

Jede Kombination aus Konfiguration und Metrik (In der restlichen Arbeit als „Zelle“ bezeichnet) wird als eigenständiges, minimales Firmware-Image gebaut. Dabei wählt CMakeLists.txt durch die Methode `target_sources_ifdef` genau eine `bm_*.c`-Datei zur Build-Zeit aus. `Kconfig` macht darüber hinaus die Auswahl über `choice BENCH_TYPE` explizit. Die getrennten Firmware-Images verringern dabei die gegenseitige Beeinflussung durch gleichzeitig eingebundenen Benchmark-Code. Unterschiede im erzeugten Speicherlayout bleiben jedoch dennoch möglich.

Die drei Konfigurationsstufen sind zudem kumulativ / inkrementell angelegt. Das Logging-Profil entsteht, indem `minimal.conf` und das Konfigurationsfragment `logging.conf` gemeinsam eingebunden werden. Dabei enthält `logging.conf` nur die Einstellungen für das Logging-Subsystem und bildet damit die Konfigurationsdifferenz im Vergleich zu der Minimal-Konfiguration. Dadurch wird nicht nur das IoT-Stack mit dem Kernel-Build verglichen, sondern auch der isolierte Effekt einzelner Subsysteme, in diesem Fall das Logging-Subsystem, dass sauber quantifizierbar bleibt. Durch diese Methodik wird eine Stufenanalyse ermöglicht anstelle eines Zwei-Gruppen-Vergleichs.

Für jede Zelle des Versuchsplans wird eine Stichprobe von N Messwerten erhoben, aus der deskriptive Kennzahlen berechnet werden. Diese sind das Minimum, Maximum, der Mittelwert und die Stichproben-Standardabweichung. Außerdem existiert ein Referenzpunkt gegen Zephyrs eigene Benchmarks, welche dieselbe Timing-API benutzen. Dies dient als externe Validierung der selbst entwickelten Messungen.

4.2 Zielplattform

Als Zielplattform für die Untersuchungen wird das nRF52840 Development Kit von Nordic Semiconductor verwendet. Die Plattform basiert auf einem 32-Bit Arm Cortex-M4 mit einer Taktfrequenz von 64 MHz und verfügt über 1 MB Flash-Speicher sowie 256 kB RAM. Damit bietet der Mikrocontroller eine klar definierte und ressourcenbeschränkte Hardwarebasis für die Untersuchung des Laufzeit- und Speicherverhaltens von Zephyr RTOS.

Für die Untersuchung der Laufzeiteigenschaften sind vor allem der Nested Vectored Interrupt Controller (NVIC) und der Data Watchpoint and Trace (DWT) des Cortex-M4 relevant. Der NVIC übernimmt die Verarbeitung und Priorisierung von Interrupts und bildet damit die hardwareseitige Grundlage

für die Messung der Interrupt-Latenz. Der DWT ermöglicht die Erfassung von CPU-Taktzyklen und wird für die hochauflösende Laufzeitmessung der Benchmarks eingesetzt. Bei 64 MHz entspricht ein CPU-Zyklus 15,625 ns, wodurch bspw. Kontextwechsel- und Interrupt-Laufzeiten mit hoher zeitlicher Auflösung bestimmt werden können.

Für die Untersuchung des Timer-Jitters wird auf der anderen Seite die RTC-basierte Zeitbasis von Zephyr verwendet. In der verwendeten Konfiguration wird hierfür RTC1 mit einem 32,768-kHz-Low-Frequency-Clock betrieben. Da diese Zeitbasis auch während des Prozessorschlafs weiterläuft, eignet sie sich zur Messung periodischer Timerintervalle im Tickless-Betrieb. Die resultierende Auflösung beträgt etwa 30,5 µs pro Tick. Die Kombination aus Cortex-M4, Interrupt- und Zeitgeberhardware sowie den begrenzten Speicherressourcen bildet damit die technische Grundlage für die durchgeführten Benchmarks (Nordic Semiconductor, o. D., Abschnitt 6.20; Zephyr Project, 2024, `nrf_rtc_timer.c`, Tag v3.7.2).

4.3 Softwareumgebung

Das Build befindet sich auf demselben Ubuntu-Host im Arbeitsbereich `~/zephyr-ws`. Dabei handelt es sich um eine eigene Python-venv (Virtual Environment). Das Flashen auf den Mikrocontroller erfolgt über denselben J-Link-OB-Debugprobe via `west flash` (nrfjprog-Runner). Die Ausgabe erfolgt dann über denselben UART/ VCOM-Kanal. Pro Messung werden ebenfalls der Commit-Hash und das verwendete `EXTRA_CONF_FILE` notiert.

4.4 Unabhängige Variablen

Die UV werden als Kconfig-Fragmente realisiert, das über `EXTRA_CONF_FILE` zusätzlich zur minimalen Basis `prj.conf` der Mess-App eingebunden wird. Die Basis enthält nur das, was für jede Messung zwingend nötig ist, darunter die Timing-API, UART/ `printf`-Ausgabe und das Heap für die Allocation-Benchmarks. Diese Basis ist bewusst identisch in allen drei Konfigurationsstufen, damit diese nicht Teil der Variation wird.

4.4.1 Minimal Konfiguration

Der schlanke RTOS-Kern ohne jegliche Zusatzdienste. Boot-Banner, Timeslicing, Assertions, Logging, Thread-Namen, MPU/ Stack-Guard, SEGGER-RTT und GPIO-Treiber sind explizit deaktiviert. Diese Konfigurationsstufe dient als Baseline und liefert die verringerten Kosten der untersuchten Kerneloperationen inklusive der im Messpfad verbleibenden Anteile. Besonders ohne MPU-bedingte Stack-Guard-Reprogrammierung bei jedem Context Switch und ohne durch Timeslicing verursachte Scheduler-Unterbrechungen.

4.4.2 Logging

Diese Konfiguration baut auf der Minimal-Konfiguration auf und aktiviert das Zephyr-Logging-Subsystem im Deferred-Modus. `CONFIG_LOG_PRINTK=n` verhindert, dass die über `printk()` ausgegebenen Benchmark-Ergebnisse in das Logging-Subsystem umgeleitet werden. `BENCHMETA-`, `BENCH-` und `BENCHEND-`Zeilen bleiben dadurch auf dem synchronen `printk-/ UART-`Pfad. Das Logging-Backend verwendet dabei ebenfalls UART. Die Benchmark-Implementierungen erzeugen jedoch selbst keine `LOG_*`-Nachrichten. Gemessen werden daher vor allem Speicherbedarf, Initialisierung, vorhandene Logging-Strukturen und mögliche indirekte Layout- oder Hintergrundeffekte, nicht die Kosten eines fortlaufenden Nachrichtenstroms.

4.4.3 IoT-Stack

Diese Konfiguration baut ebenfalls auf der Minimal-Konfiguration auf. Es wird zudem durch Netzwerk- und Sicherheits-Subsysteme erweitert, welche für einen IoT-Anwendungsfall typisch sind. Dabei handelt es sich um OpenThread als Minimal Thread Device (MTD), CoAP und eine DTLS-Absicherung über einen Pre-Shared Key (siehe 2.7). `MCUmgr` und `MCUboot` sind hierbei bewusst nicht Teil der Konfigurationsstufe, da diese am Sysbuild-Mechanismus hängen und daher nichts zur Frage beitragen, wie sich Netzwerk- und Sicherheits-Subsysteme die fünf Laufzeitmetriken beeinflussen. Sie werden daher in der Anwendung `iot_sensor` betrachtet (siehe 6.4). Zudem wurde über `CONFIG_OPENTHREAD_MANUAL_START=y` der Thread konfiguriert, damit die Benchmarks nicht durch endlose Netzwerk-Beitrittsversuche ohne vorhandenen Thread-Koordinator gestört werden. Durch `CONFIG_OPENTHREAD_MANUAL_START=y` wird kein automatischer Netzwerkbeitritt gestartet. Der OpenThread-Netzwerkbetrieb bleibt während der Benchmarks inaktiv. Erfasst werden dabei das aktivierte Softwareprofil, sein statischer Speicherbedarf, Initialisierung und vorhandene Kernelobjekte. Der aktive Mesh-Verkehr und ein DTLS-Handshake werden hierbei nicht gemessen.

4.5 Abhängige Variablen

Die DWT-basierten Benchmarks für Kontextwechsel, Interrupt-Latenz und Heap-Allokation benutzen Zeitstempel über `bench_now()`, einen kalibrierten Abzug des Messaufwands und `struct bench_stats`. Der Timer-Benchmark benutzt dieselbe Statistik und Ergebnisausgabe, liest jedoch RTC-Systemtimerzyklen über `sys_clock_cycle_get_32()`. Die Heap-Fragmentierung ist keine Zeitmessung und besitzt deshalb keinen Zeitstempel und keinen Abzug des Messaufwands oder Welford-Statistik.

4.5.1 Context-Switch-Latenz

Der Context-Switch wurde im Benchmark `bm_ctx_switch.c` implementiert. Hier wird die Zeit vom Auslösen eines Threadwechsels bis zum Zeitstempel

als erster C-seitiger Aktion im Zielthread gemessen. Dies erfolgt in zwei Varianten:

`ctx_switch_sem_wakeup`: Semaphore-geweckter höherpriorer Worker-Thread
`ctx_switch_yield`: Kooperatives Ping-Pong von gleichprioren Threads via `k_yield()`.

Dieser Benchmark erfasst dabei die Scheduler-Entscheidung inklusive PendSV-Kontextwechsel, die Grundkosten jeder Multithreading-Architektur. Dies bildet die zentrale Vergleichsgröße zwischen den drei Konfigurationen.

4.5.2 Interrupt-Latenz

Die Interrupt-Latenz wurde im Benchmark `bm_irq_latency.c` implementiert. Es wird die Zeit vom Software-Trigger bis zum Zeitstempel am Anfang des registrierten C-Handlers gemessen. Der Exception-Eintritt, Zephyr-Wrapper und Compiler-Prolog liegen schon davor und sind Teil des Messwerts. Dies erfolgt durch `NVIC_SetPendingIRQ` auf einer ungenutzten Peripherie-Leitung des `nrf52840`. Dieser Benchmark zeigt, wie stark Kernel-Funktionen, wie bspw. Logging, den Determinismus-kritischsten Pfad eines RTOS beeinflussen.

4.5.3 Timer-Jitter

Der Benchmark `bm_timer_jitter.c` erfasst die Abstände zwischen aufeinanderfolgenden Zählerzugriffen im Callback eines periodischen 10-ms-`k_timer`. Dafür wird `sys_clock_cycle_get_32()` verwendet. Die Ausgabe heißt `timer_period_rtc_cycles` und besitzt die Einheit RTC-Systemtimerzyklen. `TIMERMETA` gibt die Frequenz und die mit `k_ms_to_cyc_ceil32()` berechnete Periode an. Der Benchmark gibt Periodenwerte aus. Die Abweichung und Streuung im Vergleich zur Sollperiode werden daraus ausgewertet.

4.5.4 Heap-Allokationsgeschwindigkeit

Die Heap-Allokationsgeschwindigkeit wurde im Benchmark `bm_heap_alloc.c` implementiert. Hier wird die Zeit für die Funktionen `k_malloc` und `k_free` eines 64-Byte-Blocks gemessen. Allokation und Freigabe folgen direkt aufeinander. Dadurch bleibt der Systemheap nahezu leer und unfragmentiert, sodass ein vorteilhafter Ausgangszustand untersucht wird. Dieser Benchmark ist relevant, weil `sys_heap` eine variable Laufzeit hat, was für RTS kritisch ist.

4.5.5 Heap-Fragmentierung

Die Heap-Fragmentierung wurde im Benchmark `bm_heap_frag.c` implementiert. Hierbei handelt es sich nicht um ein Zeit-, sondern um ein Zustandsmaß. Ein eigener 8-KiB `k_heap` wird mit 32x128-Byte-Blöcken gefüllt. Dabei wird im Schachbrettmuster jeder zweite freigegeben und anschließend die angeforderten Nutzbytes, die rechnerisch nicht angeforderten Bytes und die größte im 8-Byte-Suchraster erfolgreich geprüfte Einzelanforderung protokolliert. Dieser Freiblock wird durch eine eigene Bisektions-Probe protokolliert,

da `sys_heap` diese Kennzahl nicht direkt liefert. Dieser Benchmark zeigt ein für MMU-lose Systeme spezifisches Risiko: Rechnerisch ausreichender, aber durch Fragmentierung nicht mehr am Stück nutzbarer Speicher.

Kontextwechsel, Interrupt-Latenz und Heap-Allokation liefern CPU-Zyklen. Der Timer-Benchmark liefert RTC-Systemtimerzyklen. Die jeweilige Einheit ergibt sich dabei aus dem Metriknamen und `TIMERMETA`. Die Heap-Fragmentierung liefert Nutzbyte-Buchführungswerte und Probe-Ergebnisse in Byte und benutzt deshalb `HEAPSTATS` und `HEAPPROBE`.

4.6 Kontrollierte Variablen

Diese Variablen werden über alle Konfigurationsstufen und Metriken hinweg konstant gehalten, um zu verhindern, dass Effekte fälschlich der UV zugeschrieben werden.

4.6.1 Hardware/ CPU-Frequenz

Ein einziges physisches Board, das Nordic nrf52840 DK, mit einer CPU basierend auf der Cortex-M4F Architektur und einer Takt-Frequenz von 64 MHz. Es verfügt über 1 MB Flash und 256 KB RAM. Die Frequenz wird darüber hinaus zur Laufzeitkontrolle bei jedem Boot über die Methode `timing_freq_get()` in der `BENCHMARKMETA`-Zeile in `main.c` mit ausgegeben. Falls es also zu einer Abweichung in der Takt-Frequenz kommen sollte, wäre dies sofort sichtbar.

4.6.2 Compiler und Werkzeugkette

In dieser Untersuchung wurde das Zephyr Software Development Kit (SDK) 0.16.9 verwendet. Der Compiler ist daher GCC 12.2.0, mit Binutils 2.38 und der Picolibc 1.8.6. Diese wurden über `ZEPHYR_SDK_INSTALL_DIR` gepinnt, sodass sie identisch für alle Builds sind.

4.6.3 Compiler optimierung

Keines der drei Konfigurationsfragmente ändert das Optimierungsniveau. Vor der Ergebnisinterpretation muss durch Compileraufrufe bestätigt und dokumentiert werden, dass alle Builds mit derselben Zephyr-Optimierung erstellt wurden.

4.6.4 Zephyr Version

Für diese Untersuchung wird Zephyr v3.7.2 aus der Long-Term-Support-Reihe 3.7 verwendet. Dies wurde im `Manifest west.yml` über `revision: 3.7.2` mit `import: true` gepinnt. Dies übernimmt passende HAL-, MCUBoot-, mbedTLS- und OpenThread-Revisionen. Das gepinnte Manifest legt hierbei den vorgesehenen Versionsstand fest. Für die Reproduzierbarkeit ist darüber hinaus zu dokumentieren, welcher Stand wirklich ausgecheckt und gebaut wurde.

4.6.5 Build-Konfiguration (Sockel)

Die in `prj.conf` definierte Basis ist für alle drei UV-Stufen identisch. Diese beinhaltet `Benchlib`, `printfk/ UART/ Konsole`, `CONFIG_CBPRINTF_FP_SUPPORT` für Fließkomma-Ausgabe der Statistik und einen 64-KiB-System-Heap. Diese Basis wird nicht durch die Konfigurationen verändert, sondern lediglich ergänzt. Dadurch bleibt bspw. der ROM-Messaufwand der Fließkomma-Ausgabe in jeder Konfiguration gleich groß und ermöglicht somit einen Vergleich.

4.6.6 Messmechanismus (Timing/ Statistik)

Die kurzen Laufzeitmessungen benutzen Zephyrs Timing-API und auf dem Cortex-M4F den DWT-Zyklenzähler. Zwei direkt aufeinanderfolgende Reads werden 100-mal gemessen. Das kleinste beobachtete Delta wird von den DWT-basierten Ergebnissen abgezogen. Der Timer-Benchmark benutzt stattdessen den RTC1-basierten Systemtimer und keinen Abzug vom DWT-Messaufwand.

$$t_i = \frac{c_i}{64\,000\,000} \text{ s}$$

$$t_i = \frac{c_i}{32\,768} \text{ s}$$

$$t_i = \frac{c_i}{32\,768} \text{ s}$$

Die Welford-Statistik berechnet Minimum, Maximum, Mittelwert und Stichproben-Standardabweichung für die wiederholten Zeit- bzw. Periodenwerte. Heap-Fragmentierung und statischer Speicherbedarf werden dabei nicht mit dieser Statistik ausgewertet.

4.6.7 Anzahl der Messungen

Für die wiederholten DWT- und Timer-Messreihen werden 1000 Werte nach zehn verworfenen Warm-up-Perioden bzw. -Durchläufen ausgewertet. Diese Parameter werden über `CONFIG_BENCHLIB_ITERATIONS` und `CONFIG_BENCHLIB_WARMUP` dokumentiert. Heap-Fragmentierung und RAM-/ROM-Bedarf sind auf der anderen Seite phasenbezogene Zustands- bzw. Build-Messungen und besitzen daher keine Stichprobe aus 1000 Wiederholungen.

4.7 Ablauf einer Messung

Dieser Abschnitt beschreibt den kompletten Ablauf einer Messung, welcher für jede Messzelle gleich ist. Dabei wird kurz gezeigt, wie der Arbeitsbereich eingerichtet wird. Die Werkzeugkette wird ebenfalls einmal eingerichtet. Das eigentliche Bauen, Flashen, Auslesen und die Dokumentationspflicht sind dabei bewusst als ein fester und wiederholbarer Ablauf gestaltet, damit jede Messung genau nachvollzogen werden kann.

Einmalige Einrichtung Vor der ersten Messung wurde der West-Arbeitsbereich einmalig durch das offizielle Zephyr-Manifest in der Version v3.7.2 initialisiert. Dies wurde dabei durch das eigene Projektmanifest `thesis/west.yml` erweitert (siehe 6.1).

```
mkdir -p ~/zephyr-ws && cd ~/zephyr-ws
python3 -m venv .venv
source .venv/bin/activate
pip install west
west init --mr v3.7.2
west update
pip install -r zephyr/scripts/requirements.txt
west zephyr-export
west config manifest.path thesis
west update
```

Anschließend wurden das Zephyr SDK 0.16.9 und die für das Flashen gebrauchten Werkzeuge, darunter J-Link und nRF Command Line Tools, installiert. Diese wurden zudem über einen Smoke-Test mit `zephyr/samples/basic/blink` verifiziert.

Beginn jeder Sitzung Am Anfang jeder Mess-Sitzung wird in den Arbeitsbereich gewechselt, die Python-virtuelle Umgebung aktiviert und der aktuell ausgecheckte Commit-Hash des eigenen Projekts notiert.

```
cd ~/zephyr-ws
source .venv/bin/activate
git -C thesis pull
git -C thesis log --oneline -1
```

Dieser Commit-Hash wird dabei jeder in dieser Arbeit dokumentierten Messung zugeordnet, damit sich jede Messreihe genau auf den zum Messzeitpunkt verwendeten Software- und Konfigurationsstand zurückführen lässt (siehe Kapitel 4.3).

Baseline pro Konfigurationsstufe Für jede der drei Konfigurationsstufen wird zuerst, noch vor den eigentlichen fünf Benchmarks, ein `CONFIG_BENCH_NONE`-Build erzeugt. Dieser Build enthält keine `bm_*.c`-Datei, aber weiterhin `main.c`, `benchlib`, Timing-Initialisierung, UART-/ `printf`-Ausgabe, Fließkommaformatierung und den konfigurierten 64-KiB-Systemheap. Er dient als gemeinsamer Mess-Harness zur Bestimmung des statischen Speicherbedarfs des jeweiligen Profils ohne benchmarkspezifischen Testcode. Da `rom_report` und `ram_report` nur die Linker-Map des gebauten ELF-Abbilds auswerten, ist für diesen Schritt kein Flashen und keine angeschlossene Hardware nötig.

```
cd ~/zephyr-ws
west build -p always -b nrf52840dk/nrf52840 thesis/apps/bench -- \
  -DEXTRA_CONF_FILE=../../configs/minimal.conf -DCONFIG_BENCH_NONE=y
west build -t rom_report | tail -40
```

```
west build -t ram_report | tail -40
```

Für die Logging-Konfiguration wird dabei `minimal.conf` zusammen mit dem Fragment `logging.conf` übergeben. Für die IoT-Stack-Konfiguration wird dazu noch `iot_stack.conf` und das Devicetree-Overlay `iot_stack.overlay` für die OpenThread-Entropiequelle übergeben. Genau derselbe Baseline-Prozess wird also für alle drei Konfigurationsstufen gleich wiederholt. Nur das übergebene `EXTRA_CONF_FILE` unterscheidet sich.

Ausführung der fünf Benchmarks Nach der Baseline wird für dieselbe Konfigurationsstufe nacheinander jeder der fünf Benchmarks gebaut, geflasht und ausgelesen. Dies geschieht dabei jeweils durch das Setzen eines `CONFIG_BENCH_*`-Symbols aus der choice-Auswahl (siehe 4.1).

```
west build -p always -b nrf52840dk/nrf52840 thesis/apps/bench -- \
  -DEXTRA_CONF_FILE=../../configs/minimal.conf -
  DCONFIG_BENCH_CTX_SWITCH=y
west flash
```

Dieser Doppelschritt aus Build und `west flash` wird zudem für `CONFIG_BENCH_IRQ_LATENCY`, `CONFIG_BENCH_TIMER_JITTER`, `CONFIG_BENCH_HEAP_ALLOC` und `CONFIG_BENCH_HEAP_FRAG` wiederholt. Dazu wird vor der eigentlichen Messreihe einer Konfigurationsstufe einmalig `CONFIG_BENCH_SELFTEST` gebaut und geflasht, um die Kalibrierungsprüfung der Zeitmesskette zu verifizieren (siehe Kapitel 5). Für die Logging- und IoT-Stack-Konfiguration wird derselbe Prozess von sechs Bauvorgängen (Selbsttest und die fünf Benchmarks) mit dem jeweils passenden `EXTRA_CONF_FILE`- bzw. extra `EXTRA_DTC_OVERLAY_FILE`-Parameter wiederholt. Insgesamt besteht dies für jede der drei Konfigurationsstufen aus sieben Bauvorgängen: Die Baseline (`BENCH_NONE`), den Selbsttest und die fünf Benchmarks.

Auslesen Nach jedem Flashvorgang wird die Ausgabe über die serielle Schnittstelle des J-Link-Onboard-Debuggers ausgelesen.

```
picocom -b 115200 /dev/ttyACMO
```

Nach dem Drücken der Reset-Taste am Board erscheint die Ausgabe der jeweils aktiven Firmware. Vor jeder Auswertung wird dabei kontrolliert, ob das Build-Log für jedes benutzte Konfigurationsfragment eine Zeile in der Form Merged configuration '`.../<name>.conf`' und Found toolchain: `zephyr 0.16.9` enthält. Außerdem wird geprüft, ob bei der Minimal- und Logging-Konfiguration kein Boot-Banner beim Reset zu sehen ist. Ein Banner würde anzeigen, dass `minimal.conf` nicht richtig angewendet wurde. Eine Beispiel-Ausgabe des Selbsttests sieht wie folgt aus:

```
BENCHMETA,zephyr=3.7.2,freq_hz=64000000,overhead_cycles=12,
iterations=1000,warmup=10
```

```
BENCH,selftest_busy_wait_100us,1000,6423,6435,6434.96,0.57  
BENCHEND
```

Dokumentation Für jede Messzelle des Versuchsplans werden zuletzt das Datum, der Commit-Hash, die benutzte Konfiguration inklusive des/der `EXTRA_CONF_FILE`-Fragments/-e, der aktive Benchmark, die komplette `BENCHMETA`-Kopfzeile, die dazugehörigen `BENCH`-, `HEAPSTATS`- bzw. `HEAPPROBE`-Zeilen notiert. Außerdem wird bei Fußabdruck-Messungen die „Total“-Zeile von `rom_report` und `ram_report` dokumentiert. Dadurch kann bei jeder Kennzahl (siehe Kapitel 7) bis zur Rohausgabe der Hardware zurückverfolgt werden.

Benchmark-Design

Dieses Kapitel beschreibt für jeden der fünf Benchmarks im Detail, was genau gemessen wird, wie der Messpunkt technisch implementiert ist, welche Fehlerquellen dabei kontrolliert werden müssen und warum die jeweilige Metrik für die Forschungsfrage relevant ist. Vor den DWT-basierten Messungen wird `k_busy_wait(100)` gemessen. Bei 64 MHz liegt der nominelle Wert bei 6400 CPU-Zyklen inklusive des Eigenaufwands der Funktion. Als Abnahmekriterium gelten $|\text{mean} - 6400| < 1\%$ und eine Standardabweichung unter zwei Zyklen. Der Test erkennt dabei grobe Fehler von Taktquelle, DWT oder Abzug des Messaufwands, beweist jedoch nicht die exakte Lage asynchroner Messpunkte oder die Abwesenheit von Interrupts in anderen Benchmarks.

5.1 Kontextwechsel-Latenz

```

/* Variante 1: sem_wakeup */
...
timing_t t0 = bench_now();
k_sem_give(&wake_sem);
/* Worker hat t_stamp gesetzt und blockiert wieder */
bench_stats_add(&st, bench_elapsed(t0, t_stamp) -
    bench_overhead_cycles());
...
/* Variante 2: yield (im Worker-Thread gemessen) */
...
timing_t t1 = bench_now();
bench_stats_add(&yield_stats, bench_elapsed(t_stamp, t1) -
    bench_overhead_cycles());
k_yield();

```

Definition Gemessen wird die Zeit zwischen dem Zeitstempel direkt vor der auslösenden Kerneloperation und dem früh im Zielthread erfassten Zeitstempel. Das Intervall umfasst dabei die bis zu diesem Messpunkt durchlaufenen Verwaltungs- und Umschaltoperationen. Diese Zeit umfasst auf einem Cortex-M die Verwaltung des auslösenden Kernelaufrufs, die Scheduler-Entscheidung und das Sichern und Laden des Thread-Kontexts. Laut Zephyr-Dokumentation ist ein Kontextwechsel unter anderem genau dann möglich, wenn ein Thread über `k_sem_take()` in den wartenden Zustand übergeht oder über `k_sem_give()` / `k_yield()` einen anderen Thread bereit macht bzw. die

CPU freiwillig abgibt. Diese Reschedule Points bilden das Grundkonzept der beiden gemessenen Varianten.

Messmethode Diese Untersuchung unterscheidet zwischen zwei Varianten des Kontextwechsels:

Variante	Aufbau	Messpunkt Start	Messpunkt Ende
ctx_switch_sem_wake	Ein höherpriorer Worker-Thread blockiert auf <code>k_sem_take()</code> , der niederpriorer Haupt-Thread ruft <code>k_sem_give()</code> auf	Direkt vor <code>k_sem_give()</code>	Zeitstempel als erste C-seitige Aktion nach Rückkehr aus <code>k_sem_take()</code>
ctx_switch_yield	Zwei gleichpriorere Threads übergeben sich kooperativ über <code>k_yield()</code>	Direkt vor <code>k_yield()</code>	Zeitstempel am Beginn der entsprechenden Worker-Iteration

Bei der ersten Variante garantiert die höhere Priorität des Workers, dass dieser nach `k_sem_give()` sofort läuft, bevor der Haupt-Thread weiterläuft. Laut Zephyr-Dokumentation wird ein Semaphore an den wartenden Thread mit der höchsten Priorität vergeben. Bei der zweiten Variante wechseln sich beide Threads ab, da die Zeitscheiben-Rotation in der Minimal-Konfiguration deaktiviert ist (siehe 4.4.1). Dabei reiht `k_yield()` den aufrufenden Thread laut Zephyr-Dokumentation ans Ende der Liste der bereiten Threads mit gleicher Priorität ein und übergibt die CPU dann sofort an den nächsten. Beide Varianten werden hierbei auch von Zephyrs eigenem Referenz-Benchmark `tests/benchmarks/latency_measure` erfasst, was einen externen Validierungsvergleich ermöglicht.

Fehlerquellen Interrupts können die gemessene Zeit verlängern. Tickless-Betrieb verringert periodische Systemtimer-Unterbrechungen, schließt andere Interruptquellen jedoch nicht aus. Eine geringe Selbsttest-Streuung belegt deshalb nicht, dass während den Benchmark-Reihen keine Interrupts auftreten. Die erste Worker-Ankunft läuft dabei über den Thread-Startpfad und wird durch die Warm-up-Durchläufe verworfen. MPU und hardwaregestützter Stack-Schutz sind in allen drei Benchmark-Profilen deaktiviert. Deren Zusatzkosten werden ohne separaten Vergleichslauf nicht quantifiziert.

Relevanz Kontextwechsel wurden als Metrik genommen, weil sie die Grundkosten jeder Multithreading-Architektur sind und bestimmen, wie schnell ein RTOS auf Ereignisse reagieren kann. Zudem bestimmen sie, wie fein die Aufgaben eines Systems in einzelne Threads zerlegt werden können, ohne dass der Wechsel selbst zum limitierenden Faktor wird.

5.2 Interrupt-Latenz

```
static void trigger_isr(const void *arg)
{
    ...
    isr_stamp = bench_now();
    isr_count++;
}
...
uint32_t expected = isr_count + 1;
timing_t t0 = bench_now();
NVIC_SetPendingIRQ(TRIGGER_IRQN);
while (isr_count != expected) { /* aktives Warten */ }
bench_stats_add(&st, bench_elapsed(t0, isr_stamp) -
    bench_overhead_cycles());
```

Definition Hierbei handelt es sich um die Zeit zwischen dem softwareseitigen Auslösen eines Interrupts und dem Zeitstempel am Anfang des registrierten C-Handlers. Der Exception-Eintritt, der Zephyr-Interrupt-Wrapper und der Compilerprolog liegen vor diesem Messpunkt und sind daher Teil des Messwerts. Eine ISR ist laut Zephyr-Dokumentation eine Funktion, die asynchron als Reaktion auf einem Interrupt ausgeführt wird und dabei die Ausführung des aktuellen Threads unterbricht. Ziel ist es hierbei, mit möglichst geringem Messaufwand reagieren zu können.

Messmethode Für einen softwareseitig kontrollierten Auslösezeitpunkt benutzt der Benchmark die IRQ-Leitung von TIMER1 und setzt deren Pending-Bit über `NVIC_SetPendingIRQ()`. Die TIMER1-Peripherie muss dafür nicht selbst als laufender Zeitgeber betrieben werden. Voraussetzung ist hierbei, dass die ausgewählte IRQ-Leitung im untersuchten Build nicht wo anders benutzt wird. Der Endzeitstempel wird am Anfang des registrierten C-Handlers erfasst. Exception-Eintritt, Zephyr-Interrupt-Wrapper und Compiler-Prolog liegen vor diesem Messpunkt und sind Teil des Messwerts.

Fehlerquellen Der Trigger-IRQ besitzt die numerische Priorität 3 und kann Thread-Mode-Code unterbrechen. Er kann durch Interrupts höherer Dringlichkeit unterbrochen oder durch Interruptsperrern verzögert werden. Die Warteschleife wird erst nach dem Endzeitstempel ausgeführt und geht deshalb nicht in die Zeitdifferenz ein. Die Wahl von TIMER1 und seiner IRQ-Nummer ist plattformspezifisch. Dabei gehört `NVIC_SetPendingIRQ()` zur CMSIS-Core-Schnittstelle. Der dazugehörige Typ heißt `IRQn_Type`, und der benutzte Header ist `<nrfx.h>`.

Relevanz Ein RTOS verspricht, im Gegensatz zu einem klassischen Betriebssystem, Determinismus. Die Interrupt-Latenz zeigt, wie stark Kernel-Funktionen, wie bspw. das Logging-Subsystem, den zeitkritischen Reaktionspfad eines Systems beeinflussen.

5.3 Timer-Jitter

```
static void timer_cb(struct k_timer *t)
{
    uint32_t now_cycles = sys_clock_cycle_get_32();

    if (have_prev) {
        uint32_t delta = now_cycles - prev_cycles;

        if (count >= CONFIG_BENCHLIB_WARMUP) {
            bench_stats_add(&st, delta);
        }
        count++;
    } else {
        have_prev = true;
    }
    prev_cycles = now_cycles;

    if (count >= CONFIG_BENCHLIB_WARMUP + CONFIG_BENCHLIB_ITERATIONS)
    {
        k_timer_stop(t);
        done = true;
    }
}
```

Definition: Erfasst wird die beobachtete Periode eines periodischen Timers. Jitter wird anhand der Abweichungen und Streuung dieser Perioden im Vergleich zu, der in `TIMERMETA` angegebenen nominellen RTC-Periode beurteilt.

Messmethode: Der `k_timer` besitzt eine angeforderte Periode von 10 ms. Im Callback wird `sys_clock_cycle_get_32()` gelesen. Das Delta von zwei aufeinanderfolgenden Reads ist der Abstand der Messpunkte in RTC-Systemtimerzyklen. Der Messpunkt liegt erst im C-Callback und beinhaltet deshalb auch Verzögerungen vor seiner Ausführung.

Fehlerquellen: Der erste Callback liefert noch keinen Messwert, da keine Vorgänger existieren. Danach werden zudem zehn Perioden verworfen. Die Auflösung beträgt etwa 30,52 μ s. Interrupts mit höherer Dringlichkeit, Interruptsperrern und andere Kernelaktivität können den Callback verzögern. Ein aktiviertes Logging-Subsystem ohne erzeugte Log-Nachrichten erzeugt dabei nicht automatisch fortlaufende UART-Last.

Relevanz: Die Messung zeigt, wie stabil periodische Callback-Messpunkte unter dem jeweiligen kompletten Konfigurationsprofil sind.

5.4 Heap-Allokationsgeschwindigkeit

```
...
bench_stats_reset(&alloc_st);
bench_stats_reset(&free_st);
```

```
for (int i = 0; i < CONFIG_BENCHLIB_WARMUP +  
CONFIG_BENCHLIB_ITERATIONS; i++) {  
    timing_t t0 = bench_now();  
    void *p = k_malloc(ALLOC_SIZE);  
    timing_t t1 = bench_now();  
  
    timing_t t2 = bench_now();  
    k_free(p);  
    timing_t t3 = bench_now();  
  
    if (i >= CONFIG_BENCHLIB_WARMUP) {  
        bench_stats_add(&alloc_st, bench_elapsed(t0, t1) -  
                        bench_overhead_cycles());  
        bench_stats_add(&free_st, bench_elapsed(t2, t3) -  
                        bench_overhead_cycles());  
    }  
}  
...
```

Definition Bei dieser Metrik wurde die Zeit für die Allokation durch `k_malloc()` bzw. Freigabe durch `k_free()` eines Speicherblocks mit fester Größe aus dem Zephyr-System-Heap gemessen. Die Größe des Blocks wurde hierbei auf 64 Byte festgelegt

Messmethode Pro Iteration wird ein Block angefordert und direkt danach wieder freigegeben, anstatt mehrere Blöcke gleichzeitig zu halten. Dabei verhalten sich `k_malloc()` und `k_free()` laut Zephyr-Dokumentation semantisch wie die gleichnamigen Standard-C-Funktionen. Sie arbeiten zudem auf einem einzigen, über `CONFIG_HEAP_MEM_POOL_SIZE` definierten System-Heap. Dabei werden Allokation und Freigabe als zwei getrennte Zeiten gemessen, jeweils direkt vor und nach dem entsprechenden Aufruf.

Fehlerquellen Das direkte Freigeben hält den Systemheap nahezu leer und untersucht damit einen vorteilhaften, unfragmentierten Zustand. Der Fragmentierungsbenchmark untersucht anschließend nur die Allokierbarkeit eines getrennten `k_heap`. Er misst jedoch nicht die Laufzeit von `k_malloc()` unter Fragmentierung. Aussagen über Worst-Case-Allokationszeiten oder den Einfluss eines fragmentierten Systemheaps lassen sich aus diesen beiden Benchmarks deshalb nicht treffen.

Relevanz Der Allokationsbenchmark prüft den Rückgabewert von `k_malloc()` nicht. Seine Werte dürfen nur als erfolgreiche 64-Byte-Allokationen interpretiert werden, wenn der Erfolg im Messcode zudem bestätigt wird.

5.5 Heap-Fragmentierung

```
static void print_stats(const char *label)  
{
```

```

    printf("HEAPSTATS,%s,requested=%zu,unrequested=%zu\n",
           label, bytes_allocated,
           (size_t)FRAG_HEAP_SIZE - bytes_allocated);
}
/* Bisektion: groessten Block finden, der aktuell am Stueck
   allokierbar
   * ist. */
static size_t probe_largest_block(void)
{
    size_t lo = 0, hi = FRAG_HEAP_SIZE;

    while (lo + 8 < hi) {
        size_t mid = (lo + hi) / 2;
        void *p = k_heap_alloc(&frag_heap, mid, K_NO_WAIT);

        if (p) {
            k_heap_free(&frag_heap, p);
            lo = mid;
        } else {
            hi = mid;
        }
    }
    return lo;
}

```

Definition: Ausgegeben werden erfolgreich angeforderte Nutzbytes, die rechnerische Differenz zur bereitgestellten Heap-Größe und die größte im 8-Byte-Suchraster erfolgreich geprüfte Einzelanforderung. Dabei ist `unrequested` keine Messung von freien Allokatorbytes, da Metadaten, Ausrichtung und Rundung nicht beachtet werden.

Messmethode: Von 32 angeforderten 128-Byte-Blöcken werden nur erfolgreiche Allokationen verbucht. Beim Freigeben werden NULL-Einträge übersprungen. Die Bisektions-Probe allokiert und gibt Testblöcke direkt wieder frei. Wegen der Abbruchbedingung liefert sie eine praktische Näherung im 8-Byte-Raster und nicht den mathematisch genau größten Freiblock.

Fehlerquelle: Die Probe benutzt den untersuchten Allokator selbst. Nach jedem erfolgreichen Versuch sind keine weiteren Nutzblöcke mehr angefordert. Daraus folgt aber nicht unbedingt, dass die internen Verwaltungsstrukturen unverändert sind.

Relevanz: Externe Fragmentierung kann dazu führen, dass eine große zusammenhängende Anforderung nicht erfolgreich ist, obwohl die Summe der nicht angeforderten bzw. an sich verfügbaren Bereiche größer ist. Dieses Risiko entsteht durch dynamische Allokation und begrenzten zusammenhängenden Speicher, nicht nur durch das Fehlen einer MMU.

Implementierung

In Kapitel 4 wurde begründet, was gemessen wird und warum. Zudem wurden in Kapitel 5 die einzelnen benchmark-Metriken vorgestellt. Dieses Kapitel beschreibt die softwaretechnische Umsetzung dieser Untersuchung. Der Fokus liegt dabei auf der Erklärung, wie die einzelnen Bauteile des Projekts zusammenarbeiten und welche Entwurfsentscheidungen jeweils getroffen wurden. Zuerst wird die grobe Organisation des Projekts vorgestellt (siehe 6.1). Anschließend wird die von allen Benchmarks gemeinsam genutzte Mess-Infrastruktur erläutert (siehe 6.2). Zuletzt wird eine vollständige Darstellung der umgesetzten Konfigurationsstufen gebildet.

6.1 Projektstruktur

Der West-Arbeitsbereich trennt zwischen zwei Bereichen. Bei dem ersten Bereich handelt es sich um einen `zephyr`-Ordner, der die externe Zephyr-RTOS-Quelle enthält und von West selbst verwaltet wird und dem eigenen Projektordner, welcher den Namen „thesis“ (`thesis/`) trägt. Dieser enthält die gesamte, für diese Arbeit entwickelte Software und wird in diesem Kapitel beschrieben.

Die Datei `thesis/west.yml` wird Manifest genannt. Diese legt fest, welche exakte Zephyr-Version (v3.7.2) und welche zugehörigen Module, darunter Hardware-Abstraktionsschicht, Bootloader, Netzwerk-Stack, etc. beim nächsten west update in den `zephyr`-Ordner geladen werden (siehe Kapitel 4.6.4). Dadurch ist die verwendete Software-Version nicht davon abhängig, welche Software-Version zufällig installiert wurde, sondern in einer einzigen, versionierten Datei festgelegt. Dies ist hierbei besonders für die Reproduzierbarkeit wichtig.

Eine kleine Konfigurationsdatei, `thesis/zephyr/module.yml`, teilt dem Zephyr-Build-System mit, dass `benchlib` als sogenanntes Modul behandelt werden soll. Ohne diese Datei würde Zephyr die eigene Bibliothek beim Bauen nicht finden. Diese Datei zeigt zudem nur auf zwei Pfade; auf die CMake-Bauvorschrift und auf die Kconfig-Definition der Bibliothek.

In Kapitel 6.2 wird im Detail die gemeinsame Mess-Infrastruktur erklärt. Diese ist unterteilt in:

- eine öffentliche Schnittstelle (siehe 6.2.2): `include/benchlib.h`
- drei Implementierungsdateien (siehe 6.2.3 – 6.2.5): `src/`

- eine CMake-Datei, welche festlegt, dass diese drei Dateien nur dann übersetzt werden, wenn eine Anwendung `CONFIG_BENCHLIB=y` setzt. Dadurch bleibt die Bibliothek folgenlos für jeden Build, der sie nicht ausdrücklich anfordert: `CMakeLists.txt`
- eine Konfigurationsdatei (siehe 6.2.6): `Kconfig`

Die eigentliche Messanwendung ist zudem unterteilt in:

- eine gemeinsame Basis-Konfiguration (siehe 6.3.1): `prj.conf`
- eine Konfigurationsdatei, welche die Auswahl (`choice BENCH_TYPE`) zwischen den sieben Benchmark-Varianten (die fünf Metriken, der Selbsttest und die reine Baseline des Speicherbedarfs) definiert: `Kconfig`
- eine CMake-Datei, welche abhängig von dieser Auswahl eine der sechs `bm_*.c`-Dateien in die zu übersetzende Anwendung ein (siehe Kapitel 4.1): `CMakeLists.txt`
- einem Einstiegspunkt, welcher die Messbibliothek initialisiert, die `BENCHMETA`-Kopfzeile ausgibt und anschließend die zur Build-Zeit ausgewählte `bench_run()`-Implementierung aufruft: `src/main.c`
- je eine Implementierung pro Benchmark bzw. Referenzmessung (siehe 4.5): `src/bm_*.c`

6.2 Benchmark-Infrastruktur

Alle sechs ausführbaren Varianten benutzen die gemeinsame `bench_run()`-Schnittstelle. Die DWT-basierten Benchmarks teilen Zeitmessung, Statistik und Ausgabe. Der Timer teilt Statistik und Ausgabe, benutzt jedoch eine andere Zeitquelle. Die Heap-Fragmentierung benutzt ein eigenes Zustandsformat. Die Bibliothek vereinheitlicht damit die jeweils gemeinsamen Teile.

6.2.1 Warum eine eigene Bibliothek?

Bevor der Code erklärt wird, wird die Begründung erläutert, warum diese Funktionen nicht einfach in jeder der sechs `bm_*.c`-Dateien einzeln implementiert wurde. Es gibt dafür sowohl einen wissenschaftlichen als auch einen softwaretechnischen Grund.

Der wissenschaftliche Grund ist dabei der wichtigere: Damit die fünf Benchmarks und die drei Konfigurationen überhaupt miteinander verglichen werden können, muss die Art und Weise, wie gemessen wird, für alle exakt identisch sein. Würde jede Benchmark-Datei ihre eigene kleine Zeitmess- und Statistiklogik besitzen, könnten kleine Unterschiede in dieser Logik selbst, wie z.B. eine andere Rundung, die eigentlich untersuchten Unterschiede zwischen den Konfigurationen verfälschen. Die gemeinsame Bibliothek verringert zudem Unterschiede im Messverfahren. Weitere Einflussgrößen wie z.B.

Speicherlayout, Unterbrechungen und Hintergrundaktivität müssen dabei trotzdem beachtet werden.

Der softwaretechnische Grund lautet: Ohne eine gemeinsame Bibliothek müsste derselbe Code, also Kalibrierung, Statistik, Ausgabeformat, etc. sechsmal fast identisch geschrieben werden. Das kommt mit dem Risiko, dass ein späterer Fehler oder eine Korrektur nur in einer der sechs Kopien nachgezogen wird und die Dateien sich dadurch mit der Zeit voneinander unterscheiden.

6.2.2 Die öffentliche Schnittstelle

Die Datei `benchlib.h` legt fest, was die Bibliothek nach außen anbietet, ohne zu zeigen, wie es im Detail funktioniert. Durch diese Kapselung bleiben Einzelheiten der Implementierung, z.B. dass ein Hardware-Zähler namens DWT verwendet wird, in den `.c`-Dateien verborgen. Wer die Bibliothek nutzt, muss nur diese Header Datei verstehen.

In dieser Header Datei wurde das Messprinzip festgelegt. Es wurde ein „Sandwich“-Muster angewendet. Jeder Benchmark misst nach demselben Schema.

```
struct bench_stats {
    uint32_t n;          /* Anzahl Messwerte */
    uint64_t min;        /* Minimum in Zyklen */
    uint64_t max;        /* Maximum in Zyklen */
    double mean;         /* laufender Mittelwert */
    double m2;           /* Summe quadrierter Abweichungen (Welford) */
};

/* Einmalig vor allen Messungen aufrufen: startet den Zyklenzaehler
 * und bestimmt den Messoverhead. */
void bench_timing_init(void);

/* Overhead von zwei direkt aufeinanderfolgender Zaehler-Reads (
 * Zyklen).
 * Vom Messergebnis abziehen. */
uint64_t bench_overhead_cycles(void);

/* Aktuellen Zaehlerstand lesen. */
static inline timing_t bench_now(void)
{
    return timing_counter_get();
}

/* Zyklen zwischen zwei Zaehlerstaenden. */
static inline uint64_t bench_elapsed(timing_t start, timing_t end)
{
    return timing_cycles_get(&start, &end);
}
```

Der Zähler wird direkt vor und direkt nach diesem Vorgang ausgelesen und bildet anschließend die Differenz. Der zu messende Code befindet sich dabei dazwischen, daher die Bezeichnung „Sandwich“-Muster.

Bei der Datenstruktur `struct bench_stats` handelt es sich hierbei um die laufende Statistik für die Benchmarks. Es wurde dabei kein Feld (*array*) implementiert, das alle 1000 Einzelmesswerte speichert, sondern fünf zusammenfassende Zahlen:

- Anzahl der bisherigen Messwerte: `n`
- Bisher kleinster gemessener Wert: `min`
- Bisher größter gemessener Wert: `max`
- Laufender Mittelwert: `mean`
- Zwischengröße für die Streuungsberechnung (siehe 6.2.4): M_2

```
/* Laufende Statistik (Welford-Algorithmus: numerisch stabil,  
 * braucht keinen Array-Speicher fuer die Einzelwerte). */  
struct bench_stats {  
    uint32_t n;           /* Anzahl Messwerte */  
    uint64_t min;        /* Minimum in Zyklen */  
    uint64_t max;        /* Maximum in Zyklen */  
    double mean;         /* laufender Mittelwert */  
    double m2;           /* Summe quadrierter Abweichungen (Welford) */  
};
```

Diese Entscheidung wird in 6.2.4 genauer begründet.

Die Funktionen `bench_now()` und `bench_elapsed()` sind im Header selbst als `static inline` implementiert, nicht in einer `.c`-Datei. Das hat dabei einen messtechnischen Grund. Ein Funktionsaufruf kostet selbst schon einige CPU-Zyklen, unter anderem für das Anlegen eines Stack-Rahmens und den Rücksprung. Da hierbei teilweise nur einige Dutzend Zyklen gemessen werden, z.B. die Interrupt Latenz, würde ein normaler Funktionsaufruf für die Zeitmessung selbst schon einen störenden Anteil am Messergebnis ausmachen. Daher wurde `static inline` verwendet. Die Kennzeichnung macht es dem Compiler möglich, den Funktionskörper direkt einzubetten. Ob dadurch kein Funktionsaufruf entsteht, muss am erzeugten Maschinencode bzw. Disassembly geprüft werden.

Andere Funktionssignaturen, darunter `bench_timing_init`, `bench_overhead_cycles`, `bench_stats_reset`, `add`, `stddev`, `bench_report` und `bench_run`, werden nur deklariert und nicht direkt implementiert. Sie existieren in den jeweiligen `.c`-Dateien. Dabei nimmt `bench_run()` eine Sonderrolle ein. Sie wird hier nur deklariert, aber nirgends in `benchlib` selbst implementiert. Jede der sechs `bm_*.c`-Dateien besitzt ihre eigene Version davon und `main()` ruft sie nach der Initialisierung auf (siehe Kapitel 4.1, Kconfig-choice-Mechanismus).

6.2.3 Zeitbasis und Kalibrierung des Messaufwands

Die Datei `bench_timing.c` kümmert sich um zwei Aufgaben: Die Zeitbasis in Betrieb nehmen und den eigenen Messfehler bestimmen.

Zeitbasis starten Die Funktionen `timing_init()` und `timing_start()` sind Funktionen aus Zephyrs eingebauter Timing-Programmierschnittstelle. Auf der hier verwendeten Cortex-M4-Hardware aktivieren sie einen in den Prozessor eingebauten Hardware-Zähler namens DWT-Zyklenzähler. Dieser zählt bei jedem CPU-Takt eins hoch. Ein Auslesen dieses Zählers vor und nach einem Vorgang und die anschließende Differenzbildung ergibt dadurch die Anzahl der dafür benötigten CPU-Takte. Dabei entspricht ein Takt, bei einer 64 MHz Taktfrequenz, exakt 15,625 Nanosekunden.

$$c_i = c_{i,roh} - c_{\text{Messaufwand}}$$

Kalibrierung des Messaufwands Selbst das Auslesen des Zählers kostet eine kleine Anzahl an Zyklen. Würden diese nicht berücksichtigt werden, dann würde jede Messung um genau diesen kleinen Betrag zu groß ausfallen. Um diesen Fehler zu bestimmen, wird der Zähler zweimal direkt hintereinander ausgelesen, ohne dass dazwischen irgendein zu messender Vorgang stattfindet. Dabei ist die entstehende Differenz genau der Preis des Messens selbst.

Dieser Vorgang wird 100-mal wiederholt und das Minimum aller 100 Ergebnisse wird als der eigentliche Kalibrierungswert übernommen, nicht der Durchschnitt. Das Minimum wird als Schätzung des kleinsten beobachteten Messaufwands benutzt, weil weitere Unterbrechungen einzelne Versuche normalerweise verlängern. Es ist jedoch kein Beweis für einen genau bestimmten wahren Messaufwand. Pipeline-, Prefetch- und Aufrufkontexte können sich zwischen Kalibrierung und Benchmark unterscheiden. Die Korrektur verringert dabei den systematischen Leseaufwand, entfernt aber nicht Messpunkt- und Layout-Effekte.

```
measured_overhead = UINT64_MAX;
for (int i = 0; i < 100; i++) {
    timing_t t0 = timing_counter_get();
    timing_t t1 = timing_counter_get();
    uint64_t c = timing_cycles_get(&t0, &t1);
    if (c < measured_overhead) {
        measured_overhead = c;
    }
}
```

6.2.4 Laufende Statistik

Die Speicherung von 1000 Einzelwerten zu jeweils acht Byte würde 8000 Byte pro gleichzeitig gespeicherter Messreihe brauchen. Da die Benchmarks in getrennten Firmware-Aufbauten ausgeführt werden, summiert sich dieser

Bedarf hierbei nicht über alle Konfigurationen hinweg. Die laufende Statistik umgeht trotzdem den zusätzlichen Messwertpuffer und hält den dafür benötigten Speicher dabei unabhängig von der Anzahl der Wiederholungen. Außerdem würde es dabei dem Grundgedanken widersprechen, eine für ressourcenbeschränkte Systeme möglichst sparsame Messmethode zu benutzen. Jedoch entsteht bei dieser Methodik ein Problem. Die Formel für die Varianz, also der Durchschnitt der quadrierten Abweichungen vom Mittelwert, setzt eigentlich voraus, dass der Mittelwert bekannt ist, bevor man die Abweichung berechnen kann. Bei einer laufenden Messung ändert sich dabei der Mittelwert mit jedem neuen Wert. Eine andere Methode, durch welche man fortlaufend die Summe aller Werte und die Summe aller quadrierten Werte mitzählt und die Varianz erst am Ende aus beiden Summen berechnet, hat ein numerisches Problem. Bei vielen, ähnlich großen Werten wird dabei die Differenz von zwei sehr großen, sehr ähnlichen Zahlen gebildet. Dadurch kann ein Großteil der eigentlich relevanten Nachkommastellen durch Rundungsfehler der Gleitkomma-Arithmetik verloren gehen. Dies wird auch „Auslöschung“ genannt. Dies ist ein reales Risiko bei den hier gemessenen Zyklenzahlen. Die bei dieser Untersuchung verwendete Lösung ist der Welfords Algorithmus. Der Kerngedanke ist hierbei, den Mittelwert und eine Hilfsgröße für die Streuung (hier M_2 genannt) bei jedem einzelnen neuen Messwert direkt anzupassen, anstatt sie erst am Ende aus großen Summen zu berechnen. Es wird also berechnet, wie weit der neue Wert vom bisherigen Mittelwert entfernt liegt. Der Mittelwert wird dann um einen kleinen Bruchteil dieser Abweichung in seine Richtung verschoben. Wie groß dieser Bruchteil ist, hängt hierbei davon ab, wie viele Werte bereits eingeflossen sind. Bei wenigen Werten, zieht also ein neuer Wert den Mittelwert stärker als bei vielen Werten. Anschließend wird ein kleiner Korrekturbetrag zur Streuungsgröße hinzuaddiert. Das Verfahren umgeht hierbei die problematische Differenz von großen Rohmomentsummen. Rundungsfehler und Grenzen der benutzten Gleitkommadarstellung bleiben dabei dennoch bestehen. Dies passiert zudem bei konstantem, kleinem Speicherbedarf, da nur die fünf Felder aus `struct bench_stats` unabhängig von N gespeichert werden.

$$n_0 = 0, \quad \bar{x}_0 = 0, \quad M_{2,0} = 0$$

$$\delta_n = x_n - \bar{x}_{n-1}$$

$$\bar{x}_n = \bar{x}_{n-1} + \frac{\delta_n}{n}$$

$$M_{2,n} = M_{2,n-1} + \delta_n (x_n - \bar{x}_n)$$

$$M_{2,n} = M_{2,n-1} + \delta_n (x_n - \bar{x}_n)$$

Die Benchmarks werden in getrennten Firmware-Images ausgeführt, sodass sich Messwertarrays von den verschiedenen Benchmarks oder Konfigurationen nicht gleichzeitig im RAM summieren würden. Welford umgeht dennoch ein weiteres 8-KiB-Array pro Messfirmware und hält den Speicherbedarf unabhängig von N konstant. Aus der fortlaufend berechneten Summe quadrierter Abweichungen (M_2) wird die Stichprobenvarianz durch Division durch

$n - 1$ bestimmt. Die anschließend daraus gezogene Quadratwurzel ergibt die Stichprobenstandardabweichung. Die Benutzung von $n - 1$, auch Bessels Korrektur genannt, beachtet hierbei, dass der Mittelwert aus derselben Stichprobe geschätzt wurde. Bei unabhängigen, gleich verteilten Beobachtungen mit endlicher Varianz schätzt die so ausgerechnete Varianz die Populationsvarianz erwartungstreu. Diese Eigenschaft überträgt sich jedoch hierbei nicht allgemein auf ihre Quadratwurzel und damit auf die Standardabweichung (Weissstein, o. D., „Sample Variance“).

$$s^2 = \frac{M_{2,n}}{n - 1}$$

$$s = \sqrt{\frac{M_{2,n}}{n - 1}}$$

Die Standardabweichung wird deshalb in dieser Arbeit vor allem als deskriptive Kennzahl der Streuung verwendet, die innerhalb eines Messlaufs beobachtet werden.

6.2.5 Ergebnisausgabe als CSV

```
#include <zephyr/sys/printk.h>
#include <benchlib.h>

void bench_report(const char *name, const struct bench_stats *s)
{
    /* Alle Werte in CPU-Zyklen; Umrechnung in us erfolgt am PC
     * (1 Zyklus = 15,625 ns @ 64 MHz, siehe BENCHMARK-META-Zeile). */
    printk("BENCH,%s,%u,%llu,%llu,%.2f,%.2f\n",
           name,
           s->n,
           (unsigned long long)s->min,
           (unsigned long long)s->max,
           s->mean,
           bench_stats_stddev(s));
}
```

Diese Datei gibt am Ende eines Benchmarks genau eine Textzeile pro Metrik aus, in einem einfachen, durch Kommas getrennten Format (CSV, Comma-Separated Values)

BENCH, <Name>, <n>, <min>, ...

Dieses Format wurde aus zwei Gründen gewählt. Erstens ist es maschinenlesbar. Eine solche Zeile lässt sich später ohne manuellen Aufwand automatisiert in Tabellenprogramme oder Auswertungsskripte einlesen, ohne dass ein aufwändiger Parser für ein komplexeres Format geschrieben werden müsste. Zweitens bleibt es dabei trotzdem menschenlesbar. Während einer Messung am seriellen Terminal lässt sich das Ergebnis direkt am Bildschirm ablesen, ohne ein weiteres Werkzeug zu benötigen. Ein methodisch wichtiger

Kommentar direkt im Code zeigt zudem, dass die Ausgabefunktion erst nach Abschluss der Messschleife aufgerufen werden darf. Eine Textausgabe über die serielle Schnittstelle ist selbst ein langsamer Vorgang. Würde diese dabei innerhalb der gemessenen 1000 Iterationen aufgerufen werden, dann würde dieser große, für die Messung irrelevante Zeitaufwand in die Ergebnisse mit einfließen. Dadurch würden diese Messungen unbrauchbar gemacht werden. Bei allen ausgegebenen Werten handelt es sich um Rohwerte in CPU-Zyklen. Die Umrechnung in Mikro- bzw. Nanosekunden erfolgt dabei bewusst nicht mehr auf dem Mikrocontroller, sondern erst nachträglich bei der Auswertung am PC. Dabei wird die Taktfrequenz aus der `BENCHMARK`-Kopfzeile verwendet (siehe Kapitel 4.6.1). Das hält die Firmware selbst einfach und verlagert die eigentliche Interpretation der Daten dorthin, wo sie stattfinden soll.

6.2.6 Parametrisierung über Kconfig

Die Bibliothek wird über ein eigenes Kconfig-Symbol, `BENCHLIB`, aktiviert, das per `select TIMING_FUNCTIONS` automatisch die Zephyr-interne Zeitmess-Funktionalität mit einschaltet. Dabei ist dies eine bewusste Entscheidung. Ohne den Zyklenzähler wäre keine der Messfunktionen dieser Bibliothek überhaupt nutzbar, weshalb diese Abhängigkeit unbedingt gestaltet wurde, anstatt optional. Zwei weitere Werte sind hierbei konfigurierbar und werden bei jedem Lauf in der `BENCHMARK`-Kopfzeile mit ausgegeben, damit jede Messreihe aus den Rohdaten heraus nachvollziehbar bleibt:

`BENCHLIB_ITERATIONS`: Legt die Zahl der ausgewerteten Wiederholungen pro zeitbezogenem Benchmark fest. Der ausgewählte Wert von $N=1000$ liegt in derselben Größenordnung wie der Stichprobenumfang der Leistungsauswertung bei Silva et al. (2019, S. 10379).

Für unabhängige Beobachtungen mit gleicher Verteilung kann der Standardfehler des Mittelwerts durch die Wurzel aus N schätzen. Bei $N = 1000$ entspricht dies hierbei ungefähr 3,16 Prozent der Stichprobenstandardabweichung. Diese Beziehung ist jedoch keine nachgewiesene Genauigkeitsangabe für die vorliegenden Messreihen. Aufeinanderfolgende Ausführungen können voneinander abhängen, bspw. durch gemeinsamen Systemzustand oder periodische Hintergrundaktivität (Weisstein, o. D., „Standard Error“).

`BENCHLIB_WARMUP`: Hierbei handelt es sich um Anzahl der vor der eigentlichen Messung verworfenen „Aufwärm“-Durchläufe. Die allersten Ausführungen eines Codepfads auf einem Mikrocontroller können durch Effekte wie einem Flash-Prefetch bzw. NVMC-Cache, Pipeline, Erstinitialisierung und Thread-Startpfade oder weitere Pipeline-Effekte künstlich langsamer ausfallen als alle folgenden, identischen Durchläufe. Diese ersten zehn Durchläufe werden daher bewusst durchgeführt, aber nicht in die Statistik aufgenommen.

6.3 Konfigurationen: Minimal / Logging / (IoT-Stack)

Alle drei Konfigurationen werden jeweils in einer eigenen .conf-Datei implementiert, die dem Build zusammen mit der gemeinsamen Basis-Konfiguration der Mess-Anwendung übergeben wird. Bevor jedoch diese drei Konfigurationsstufen erklärt werden, wird zuerst die gemeinsame Basis erläutert, da sie in allen drei Stufen enthalten ist. Dadurch ist sie nicht selbst Teil der untersuchten Variation.

6.3.1 Gemeinsame Basis

```
# --- Zwingend fuer die Messung ---  
# Messbibliothek; selektiert TIMING_FUNCTIONS (DWT-Zyklenzaehler)  
CONFIG_BENCHLIB=y  
  
# --- Zwingend fuer die Ergebnisausgabe ---  
# Synchrone Minimal-Ausgabe (bewusst KEIN Log-Subsystem)  
CONFIG_PRINTK=y  
# UART-Treiber  
CONFIG_SERIAL=y  
# Konsolen-Subsystem  
CONFIG_CONSOLE=y  
# printk -> UART0 (VCOM des J-Link OB)  
CONFIG_UART_CONSOLE=y  
# %f in printk (mean/stddev); in allen Konfigurationen identisch  
# enthalten -> verzerrt den Konfigvergleich nicht  
CONFIG_CBPRINTF_FP_SUPPORT=y  
  
# --- Fuer die spaeteren Heap-Benchmarks ---  
# 64 KiB System-Heap fuer k_malloc/k_free  
CONFIG_HEAP_MEM_POOL_SIZE=65536
```

Diese Datei enthält nur das, was für jede Messung technisch zwingend benötigt wird. Dies gilt für jeden Benchmark und jede Konfiguration, die zum Zeitpunkt der Messung aktiv ist. Der gemeinsame Sockel hält hierbei die vorgesehenen Basisfunktionen konstant. Wechselwirkungen mit weiteren aktivierten Subsystemen sind dadurch nicht ausgeschlossen. Der gemeinsame Sockel hält diese Optionen konstant, entfernt aber keine Wechselwirkungen mit weiteren aktivierten Subsystemen oder Änderungen des Linkerlayouts. Diese Entscheidung ist dabei Teil des konstant gehaltenen Sockels.

CONFIG_BENCHLIB=y

Dies aktiviert das eigenentwickelte Kconfig-Modul der Mess-Bibliothek (siehe 6.2). Dieses eine Flag löst intern durch eine Kconfig-select-Abhängigkeit automatisch CONFIG_TIMING_FUNCTIONS=y aus. Dabei handelt es sich um den Zugriff auf Zephyrs Timing-API (Zyklenzähler der Hardware, siehe 6.2). Diese Flag

erlaubt es, die Zeitmessfunktionen zu benutzen, welche in den Benchmarks verwendet werden.

```
CONFIG_PRINTK=y, CONFIG_SERIAL=y, CONFIG_CONSOLE=y, CONFIG_UART_CONSOLE=y
```

Diese Optionen stellen die vorgesehene Konsolenausgabe der Messanwendung bereit. Über den virtuellen COM-Port vom Onboard-Debugger können die ausgegebenen Messdaten am Host-Rechner direkt empfangen werden. Die Benutzung derselben Ausgabefunktion und derselben Schnittstelle legt jedoch nicht alle internen Verarbeitungsschritte fest. Vor allem kann ein aktiviertes Logging-Subsystem `printk()`-Nachrichten übernehmen. Ob dies im jeweiligen Messlauf passiert, ist anhand der zusammengeführten Build-Konfiguration zu prüfen. In der Logging-Stufe stellt `CONFIG_LOG_PRINTK=n` sicher, dass `printk()` nicht zum Logging-Frontend umgeleitet wird. (Zephyr Project, 2024, „Logging“, Abschnitt „Printk“).

```
CONFIG_CBPRINTF_FP_SUPPORT=y
```

Zephyrs `printk()` unterstützt in der Grundeinstellung aus Ressourcen-gründen keine Gleitkommazahlen (`%f`), da die dafür benötigte Software-Gleitkomma-Formatierung Flash-Speicher kostet. Da die Mess-Statistik aber Mittelwert und Standardabweichung als Kommazahlen ausgibt, muss dieses Flag gesetzt sein. Dabei ist es wichtig zu erwähnen, dass dies in der Messmethodik in allen drei Konfigurationsstufen implementiert ist. Dadurch ist der verursachte Flash-Speicher Verbrauch in jeder Messung gleich hoch und verfälscht dadurch nicht den Vergleich. Dieser Verbrauch ist also Teil des konstanten Sockels und nicht der untersuchten Variablen.

```
CONFIG_HEAP_MEM_POOL_SIZE=65536
```

Dies reserviert in jedem Benchmark-Profil einen 64-KiB-Systemheap. Dieser wird direkt vom Allokationsbenchmark benutzt, steht an sich aber auch anderen aktivierten Subsystemen zur Verfügung. Die Reservierung beeinflusst den statischen RAM-Report und wird deshalb als Teil des gemeinsamen Sockels ausgewiesen. Der Fragmentierungsbenchmark benutzt davon unabhängig einen eigenen 8-KiB-`k_heap`.

6.3.2 Minimal-Konfiguration

```
CONFIG_BOOT_BANNER=n
CONFIG_TIMESLICING=n
CONFIG_ASSERT=n
CONFIG_LOG=n
CONFIG_THREAD_NAME=n
CONFIG_DEBUG_THREAD_INFO=n
CONFIG_ARM_MPU=n
CONFIG_HW_STACK_PROTECTION=n
CONFIG_USE_SEGGER_RTT=n
CONFIG_GPIO=n
```

Diese Konfiguration ist die erste und simpelste Untersuchungsstufe. Ziel dieser Konfiguration ist es, einen RTOS-Kern zu erhalten, der nur das enthält, was für die Mess-Anwendung unbedingt benötigt wird. Jedes Merkmal, das zusätzlich von Zephyr oder dem verwendeten Board standardmäßig aktiviert würde, wird hierbei bewusst deaktiviert. Dadurch kann die Minimalkonfiguration als Referenzstufe (Baseline) fungieren, um später die Logging- und IoT-Stufe mit ihr vergleichen zu können. Dabei ist zu beachten, dass nicht jedes Flag in dieser Datei einen von Zephyr aktivierten Standardwert deaktiviert. Ein Teil der Flags ist bereits die Zephyr-Voreinstellung und wird hierbei explizit dokumentiert, damit es nachvollziehbar ist, dass diese Entscheidungen bewusst getroffen sind. Andere Flags wiederum überschreiben einen abweichenden Standard- bzw. Board-Wert.

`CONFIG_BOOT_BANNER=n`

Zephyr gibt beim Start standardmäßig eine Textzeile mit Versionsinformationen über die Konsole aus, den sogenannten *Boot-Banner*. Diese Ausgabe kostet zusätzlichen Flash-Speicher sowohl für die Zeichenkette als auch für die Laufzeit beim Start. Für die Messungen ist diese Ausgabe unnötig. Durch das Deaktivieren entsteht dabei ein praktischer Nebeneffekt. Falls der Boot Banner trotz Deaktivierung ausgegeben wird, zeigt dies, dass dieses Konfigurationsfragment beim Build nicht angewendet wurde.

`CONFIG_TIMESLICING=n`

Zephyrs Scheduler kann so konfiguriert werden, dass er gleichpriorigen Threads per Zeitscheiben-Rotation abwechselnd Rechenzeit zuteilt (Round-Robin), indem er einen laufenden Thread nach Ablauf eines Zeitfensters unterbricht, auch wenn dieser noch nicht freiwillig abgeben wollte. Für die Benchmarks ist dieses Verhalten jedoch unerwünscht, da eine solche unvorhersehbare Unterbrechung einen Störfaktor in der Zeitmessung darstellt, der nichts mit der untersuchten Metrik zu tun hat. Dadurch würden die einzelnen Messwerte durch sogenannte *Ausreißer* nach oben verfälscht werden. Diese Entscheidung folgt dabei derselben Vorgehensweise wie Zephyrs eigener Referenz-Benchmark (`latency_measure`), der ebenfalls ohne Zeitscheiben-Rotation misst.

`CONFIG_ASSERT=n`

Dies aktiviert bzw. deaktiviert die im Kernel eingebaute Laufzeitprüfung (Assertions), die im Falle eines Fehlers das System kontrolliert anhalten. Dieser Wert entspricht dabei bereits der Zephyr-StandardEinstellung für den hier verwendeten Build-Typ. Dies wird hierbei explizit gesetzt, um die bewusst getroffene Entscheidung zu dokumentieren, anstatt sich auf einen unsichtbaren Standardwert zu verlassen.

`CONFIG_LOG=n`

Dies deaktiviert das Zephyr-Logging-Subsystem vollständig. Dieses Flag ist die entscheidende Kontrollvariable dieser Untersuchung. Es handelt sich hierbei um den einzigen inhaltlichen Unterschied zur Logging-Konfiguration (siehe 6.3.3). Die beobachteten Unterschiede werden hierbei zwischen zwei Profilen untersucht, die sich durch die Aktivierung des Logging-Subsystems unterscheiden. Ihre technische Ursache muss jedoch noch anhand von der aktiven Konfiguration und des erzeugten Programms beurteilt werden.

`CONFIG_THREAD_NAME=n`

Dies deaktiviert die Speicherung eines lesbaren Namensstrings pro Thread (z.B. für Debugging-Zwecke). Jeder gespeicherte Name kostet extra RAM-Platz pro angelegten Thread. Diese Flag ist deaktiviert, da dies für die Messungen nicht benötigt wird.

```
CONFIG_DEBUG_THREAD_INFO=n
```

Dies deaktiviert zusätzliche, für Debugger bestimmte Metadaten über die im System laufenden Threads (z.B. für die Anzeige im Eclipse-/ GDB-Thread-Debugging). Auch dieses Merkmal dient der Entwicklungsunterstützung und wird für das Ausführen der Mess-Anwendung nicht benötigt.

```
CONFIG_ARM_MPU=n & CONFIG_HW_STACK_PROTECTION=n
```

Diese Flags deaktivieren den MPU-basierten Schutz in allen drei Benchmark-Profilen. Bei aktivem Stack-Guard können auch Supervisor-Threads MPU-Regionen brauchen. Userspace-Memory-Domains sorgen für weitere MPU-Konfiguration. Da keine dieser Varianten gemessen wurde, lässt sich der vermiedene Kontextwechselaufwand nicht quantifizieren. Die Messergebnisse gelten daher für die ausgewählte Konfiguration ohne MPU-basierten Stackschutz.

```
CONFIG_USE_SEGGER_RTT=n
```

Dieses Flag wurde durch eine Untersuchung während der Einrichtung gesetzt. Dabei wurde ein Speicherbericht (`ram_report`) erstellt, welcher zeigte, dass der Treiber für das SEGGER-RTT-Protokoll ungefragt in das Firmware-Abbild eingebunden wurde, obwohl die Ausgabe der Mess-Anwendung nur über UART erfolgt. Bei diesem Protokoll handelt es sich um einen alternativen, im Debugger integrierten Ausgabekanal. Dies liegt daran, dass es sich um einen Board-Standardwert des verwendeten nRF52840-DK-Boards handelt, keinen allgemeinen Zephyr-Standardwert. Er wurde daher explizit deaktiviert, nachdem sein ungenutzter Speicherverbrauch (ca. 2,2 KiB Flash und RAM für einen internen Puffer) im Speicherbericht angezeigt wurde.

```
CONFIG_GPIO=n
```

Dieses Flag wurde ebenfalls gesetzt, nachdem ein Board-Standardwert entdeckt wurde. Der GPIO-Treiber (allgemeine digitale Ein-/ Ausgabepins) wurde vom Board-Definitionsdatensatz standardmäßig aktiviert, obwohl die Mess-Anwendung keine GPIO-Funktionalität verwendet. Es wird nur die serielle Schnittstelle, die über eine eigene, unabhängige Pin-Konfiguration verfügt, verwendet. Auch dies wurde durch derselben `ram_report`-Prüfung herausgefunden.

6.3.3 Logging-Konfiguration

```
CONFIG_LOG=y
CONFIG_LOG_PRINTK=n
CONFIG_LOG_MODE_DEFERRED=y
CONFIG_LOG_BUFFER_SIZE=1024
CONFIG_LOG_DEFAULT_LEVEL=3
CONFIG_LOG_BACKEND_UART=y
CONFIG_LOG_FUNC_NAME_PREFIX_DBG=n
```

Diese Konfiguration entspricht exakt der Minimal-Konfiguration mit genau einer großen Änderung: `CONFIG_LOG` wird von `n` auf `y` gesetzt, wodurch das Zephyr-Logging-Subsystem aktiviert wird. Wichtig für die Reproduzierbarkeit ist hierbei, dass die Datei `logging.conf` nicht die Einstellungen aus `minimal.conf` als Text enthält, sondern nur das inhaltliche Delta ist. Damit diese Konfigurationsstufe aus der Minimal- und Logging-Konfiguration besteht, müssen beim Bauen beide Dateien gemeinsam übergeben werden. Dies erfolgt durch den Befehl:

```
-DEXTRA_CONF_FILE="minimal.conf;logging.conf" (siehe Kapitel 4.7)
```

Nur so bleiben auch in dieser Konfigurationsstufe die, in der Minimal-Konfiguration, deaktivierten Merkmale weiterhin deaktiviert. Der einzige inhaltliche Unterschied wird somit auf das Logging-Merkmal beschränkt.

```
CONFIG_LOG=y
```

Aktiviert das Logging-Subsystem als Ganzes. Dies ist die eigentliche unabhängige Variable dieser Konfigurationsstufe.

```
CONFIG_LOG_MODE_DEFERRED=y
```

Zephyrs Logging-Subsystem kennt hierbei zwei Betriebsarten. Im *Immediate*-Modus würde jeder einzelne Log-Aufruf sofort oder synchron die komplette Textformatierung durchführen und über die serielle Schnittstelle ausgeben, bevor der aufrufende Code fortgesetzt wird. Für die Untersuchung wurde jedoch die zweite Betriebsart, der *Deferred*-Modus, verwendet. Hierbei schreibt ein Log-Aufruf stattdessen nur die Rohdaten (Nachricht und Argumente) in einen Ringpuffer. Ein separater, niedriger priorisierter Log-Thread entnimmt dann diese Einträge asynchron im Hintergrund. Diese werden dann erst dort in Text umformatiert und anschließend ausgegeben. Diese Entscheidung wurde bewusst getroffen. Der Immediate-Modus würde direkt im gemessenen Codepfad selbst unvorhersehbar lange, synchrone UART-Schreibvorgänge auslösen und dadurch die Zeitmetriken, die untersucht werden, verfälschen. Der Deferred-Modus entspricht dem in der Praxis üblichen Einsatzszenario von Logging in eingebetteten Systemen und ist Zephyrs Standardeinstellung.

```
CONFIG_LOG_BUFFER_SIZE=1024
```

Dies legt die Größe des eben genannten Ringpuffers auf 1024 Byte fest. Dabei handelt es sich um einen Zephyr-Standardwert, welcher übernommen wurde. Dieser Speicherbereich existiert nur in der Logging-Konfiguration und ist damit ein direkt messbarer RAM-Mehrverbrauch, welcher sich der Logging-Funktion zuordnen lässt.

```
CONFIG_LOG_DEFAULT_LEVEL=3
```

Dies legt für Module ohne eigene Festlegung das Informationsniveau als Standardfilter fest. Daraus ergibt sich dann jedoch keine bestimmte Anzahl von Log-Nachrichten. Die echte Logging-Last hängt zudem davon ab, welche Log-Aufrufe ausgeführt werden, wie häufig sie auftreten und welche Inhalte sie verarbeiten. (Zephyr Project, 2024, „Logging“, Abschnitt „Global Kconfig Options“).

```
CONFIG_LOG_BACKEND_UART=y
```

Dies legt fest, über welchen physischen Ausgabekanal der Log-Thread seine formatierten Nachrichten ausgibt. Hierbei wird derselbe UART-Kanal verwendet, über den auch die `printf()`-Ergebniszeilen der Benchmarks laufen. Diese

Wahl stellt sicher, dass keine, in der Minimal-Konfiguration nicht vorhandene Hardware-Schnittstelle (z.B SEGGER-RTT) für das Logging aktiviert wird.

```
CONFIG_LOG_FUNC_NAME_PREFIX_DBG=n
```

Dieses Flag deaktiviert die automatische Voranstellung des Funktionsnamens vor jede Debug-Log-Nachricht. Dieses Merkmal betrifft nur die Textformatierung der Ausgabe und keine der Benchmark-relevanten Zeitmetriken. Es wurde daher deaktiviert, um den daraus entstehenden Flash-Speicherverbrauch zu vermeiden.

6.3.4 IoT-Stack-Konfiguration

```
CONFIG_NETWORKING=y
CONFIG_NET_SOCKETS=y
CONFIG_NET_SOCKETS_POSIX_NAMES=y
CONFIG_NET_IPV6=y
CONFIG_NET_UDP=y
CONFIG_NET_TCP=n
CONFIG_NET_L2_OPENTHREAD=y
CONFIG_OPENTHREAD_MTD=y
CONFIG_OPENTHREAD_MANUAL_START=y
CONFIG_COAP=y
CONFIG_NET_SOCKETS_SOCKOPT_TLS=y
CONFIG_NET_SOCKETS_ENABLE_DTLS=y
CONFIG_MBEDTLS=y
CONFIG_MBEDTLS_TLS_VERSION_1_2=y
CONFIG_MBEDTLS_ENABLE_HEAP=y
CONFIG_MBEDTLS_HEAP_SIZE=16384
CONFIG_MBEDTLS_KEY_EXCHANGE_PSK_ENABLED=y
CONFIG_MBEDTLS_PSK_MAX_LEN=32
```

Die Benchmark-Stufe kombiniert `minimal.conf` mit `iot_stack.conf` und `iot_stack.overlay`. Aktiviert werden Networking, IPv6, UDP, OpenThread MTD, CoAP und DTLS/ PSK über mbedTLS.

```
CONFIG_NETWORKING=y
```

Diese Option aktiviert Zephyrs allgemeine Netzwerkunterstützung. Sie bildet die Grundlage für Link-Layer, Netzwerkpuffer und höhere Protokolle wie IPv6. Die Option aktiviert allein noch keine bestimmte Netzwerkkommunikation, stellt aber die gemeinsamen Strukturen bereit, auf denen die nachfolgenden Netzwerkoptionen aufbauen.

```
CONFIG_NET_SOCKETS=y
```

Diese Option aktiviert Zephyrs BSD-Socket-Schnittstelle. Anwendungen können dadurch Netzwerkkommunikation über Funktionen wie `socket()`, `bind()`, `recvfrom()` und `sendto()` umsetzen. Die Socket-Architektur wird bei der Implementierung des CoAP-Servers in Kapitel 6.4.2 benutzt. Die allgemeine Rolle der Socket-Schnittstelle wird deshalb hier nicht nochmal erläutert (Zephyr Project, 2024, „BSD Sockets“, Abschnitt „Overview“).

`CONFIG_NET_SOCKETS_POSIX_NAMES=y`

Zephyr versieht seine Socket-Funktionen standardmäßig mit dem Präfix `zsock_`, um Namenskonflikte zu umgehen. Diese Option stellt zudem die bekannten POSIX-Namen wie `socket()`, `bind()`, `recvfrom()` und `sendto()` bereit, ohne dafür die komplette POSIX-Unterstützung zu aktivieren.

Die Option ist in Zephyr v3.7.2 schon als veraltet gekennzeichnet und wird aufgrund der regulären POSIX-Socket-API nur noch aus Kompatibilitätsgründen angeboten. Sie wird hier benutzt, weil der vorhandene Anwendungscode diese Funktionsnamen benutzt (Zephyr Project, 2024, „BSD Sockets“; Zephyr Project, o. D., Kconfig `NET_SOCKETS_POSIX_NAMES`, Tag v3.7.2).

`CONFIG_NET_IPV6=y`

Diese Option aktiviert den IPv6-Teil vom Zephyr-Netzwerkstack. IPv6 ist für das benutzte Thread-Netz erforderlich, da Thread als IPv6-basiertes Mesh-Protokoll arbeitet. Die Verbindung zwischen Thread, IPv6, 6LoWPAN und IEEE 802.15.4 wurde bereits in Kapitel 2.7.3 beschrieben.

`CONFIG_NET_UDP=y`

Diese Option aktiviert UDP als verbindungsloses Transportprotokoll. CoAP benutzt UDP. Auch die DTLS-Absicherung arbeitet in dieser Konfiguration auf Datagrammen. Die Protokollbeziehung zwischen CoAP, UDP und DTLS wurde bereits in Kapitel 2.7.1 und 2.7.2 erläutert.

`CONFIG_NET_TCP=n`

TCP wird deaktiviert, da die Benchmark-Konfiguration keine TCP-Anwendung und kein TLS über einen TCP-Datenstrom benutzt. CoAP und DTLS arbeiten hier auf UDP. Die Deaktivierung verhindert, dass nicht gebrauchte TCP-Funktionen in das Firmware-Abbild aufgenommen werden.

`CONFIG_NET_L2_OPENTHREAD=y`

Diese Option bindet OpenThread als Layer-2-Implementierung in Zephyrs Netzwerkstack ein. Zephyr verbindet dabei den externen OpenThread-Stack mit seiner IEEE-802.15.4-Treiberschnittstelle. Die allgemeinen Eigenschaften des Thread-Protokolls wurden bereits in Kapitel 2.7.3 erklärt.

Die Aktivierung zieht weitere Abhängigkeiten nach sich. Dazu gehören unter anderem die IEEE-802.15.4-Unterstützung, OpenThread selbst, C++-Laufzeitanteile und eine Entropiequelle. Der Speicherbedarf der IoT-Stufe ist daher nicht nur den 18 sichtbaren Zeilen des Konfigurationsfragments zuzuordnen, sondern beinhaltet auch die von Kconfig aufgelösten Abhängigkeiten (Zephyr Project, 2023, „Thread protocol“; Zephyr Project, o. D., Kconfig `NET_L2_OPENTHREAD`, Tag v3.7.2).

`CONFIG_OPENTHREAD_MTD=y`

Diese Option wählt die Gerätekategorie Minimal Thread Device (MTD). Ein MTD beteiligt sich nicht als Router an der Weiterleitung fremder Pakete. Dadurch ist der Funktionsumfang niedriger als bei einem Full Thread Device. Diese Auswahl entspricht dem ressourcenbeschränkten Endgerät, das durch die IoT-Sensoranwendung dargestellt werden soll.

`CONFIG_OPENTHREAD_MANUAL_START=y`

Diese Option verhindert, dass OpenThread während der Initialisierung automatisch versucht, einem vorhandenen Thread-Netzwerk beizutreten. Das

Stack müsste stattdessen über seine API oder eine passende Kommandozeilenschnittstelle gestartet werden.

Für die Benchmark-Anwendung erfolgt ein solcher Start nicht. Dadurch werden keine fortlaufenden Beitrittsversuche ohne vorhandenen Thread-Koordinator erzeugt. Die Messung beinhaltet somit die eingebundene OpenThread-Software und ihre statischen Ressourcen, aber keinen aktiven Netzwerkbeitritt oder Mesh-Verkehr.

Dieser Punkt unterscheidet die Benchmark-Konfiguration von `iot_sensor`. Dort ist `CONFIG_OPENTHREAD_MANUAL_START=n` gesetzt, weil die Demonstrationsanwendung am Netzbetrieb teilnehmen soll (Zephyr Project, o. D., Konfig `OPENTHREAD_MANUAL_START`, Tag v3.7.2).

`CONFIG_COAP=y`

Diese Option bindet Zephyrs CoAP-Bibliothek ein. Sie stellt Funktionen zum Erzeugen und Parsen von CoAP-Paketen und zur Verarbeitung von Ressourcen und Optionen bereit. Die Bibliothek erstellt selbst keine Sockets. Die Anwendung stellt den Kommunikationskanal über die Socket-API bereit und übergibt dabei die empfangenen Puffer zur Verarbeitung an die CoAP-Bibliothek.

Das Protokoll selbst wurde bereits in Kapitel 2.7.1 erläutert. Die bestimmte Verwendung für die Ressource `/sensor` wird in Kapitel 6.4.2 beschrieben (Zephyr Project, 2023, „CoAP“, Abschnitt „Overview“).

In der Benchmark-Anwendung wird keine CoAP-Ressource aufgerufen. Die Option dient dort dazu, die Bibliothek als Teil des IoT-Softwareprofils in den Build einzubeziehen.

`CONFIG_NET_SOCKETS_SOCKOPT_TLS=y`

Diese Option erweitert die Socket-Schnittstelle um TLS-/ DTLS-spezifische Protokolltypen und Socket-Optionen. Dazu gehören bspw. `TLS_SEC_TAG_LIST` zur Auswahl von registrierten Zugangsdaten und `TLS_DTLS_ROLE` zur Festlegung der Client- oder Serverrolle.

Die allgemeine DTLS-/ PSK-Funktionsweise wurde schon in Kapitel 2.7.2 erläutert. Die bestimmte Registrierung eines PSK und seiner Identität über `tls_credential_add()` und deren Zuordnung zum Socket wird in Kapitel 6.4.2 beschrieben (Zephyr Project, 2024, „BSD Sockets“, Abschnitt „Secure Sockets“).

`CONFIG_NET_SOCKETS_ENABLE_DTLS=y`

Diese Option erweitert die sichere Socket-Schnittstelle um DTLS-Unterstützung. Ohne sie wäre über `CONFIG_NET_SOCKETS_SOCKOPT_TLS` standardmäßig nur TLS auf einem TCP-Datenstrom verfügbar. Die Option wählt dabei für Zephyrs nativen Netzwerkstack gleichzeitig die gebrauchte DTLS-Unterstützung in mbedTLS aus.

Obwohl die Funktion in das Benchmark-Firmware-Abbild eingebunden ist, erzeugt `bench` keinen DTLS-Socket und führt keinen Handshake aus. Gemessen wird daher nicht die Laufzeit einer DTLS-Verbindung, sondern der Einfluss des eingebundenen Softwareprofils.

`CONFIG_MBEDTLS=y`

Diese Option bindet die mbedTLS-Kryptografiebibliothek ein. Zephyrs native TLS-/ DTLS-Sockets benutzen deren kryptografische und protokollspezifische

Funktionen. mbedTLS wird außerdem von OpenThread für verschiedene Sicherheitsfunktionen benutzt.

`CONFIG_MBEDTLS_TLS_VERSION_1_2=y`

Diese Option aktiviert die Protokollunterstützung für TLS 1.2 in mbedTLS. In der hier benutzten Zephyr-Version gehört diese Einstellung zu den Voraussetzungen für die aktivierte DTLS-Unterstützung. Die Benennung bezieht sich dabei auf die gemeinsame TLS-1.2-Protokollbasis. Im Socket wird dann anschließend DTLS 1.2 verwendet.

`CONFIG_MBEDTLS_ENABLE_HEAP=y`

Diese Option erlaubt mbedTLS die Verwendung eines eigenen globalen Speicherbereichs für dynamische Anforderungen. Der mbedTLS-Heap soll dabei von dem über `CONFIG_HEAP_MEM_POOL_SIZE` bereitgestellten Zephyr-Systemheap unterschieden werden. Zephyr initialisiert diesen Speicherbereich beim Systemstart automatisch, solange die reguläre mbedTLS-Initialisierung aktiv ist (Zephyr Project, o. D., Kconfig MBEDTLS_ENABLE_HEAP, Tag v3.7.2).

`CONFIG_MBEDTLS_HEAP_SIZE=16384`

Diese Option legt die Größe des mbedTLS-Heaps auf 16.384 Byte bzw. 16 KiB fest. Dieser reservierte Speicherbereich ist ein Teil des RAM-Bedarfs der IoT-Konfiguration.

Die Größe ist eine bewusste Konfigurationsentscheidung für diese Untersuchung. Sie ist kein allgemeiner mbedTLS-Standardwert. Der gebrauchte Speicher hängt von den benutzten Protokollen, Puffergrößen, Cipher Suites und gleichzeitig aktiven Verbindungen ab (Zephyr Project, o. D., Kconfig MBEDTLS_HEAP_SIZE, Tag v3.7.2).

`CONFIG_MBEDTLS_KEY_EXCHANGE_PSK_ENABLED=y`

Diese Option aktiviert PSK-basierte Schlüsselaustauschverfahren in mbedTLS. Dadurch kann die in Kapitel 2.7.2 beschriebene Absicherung mit einem vorher ausgesuchten gemeinsamen Schlüssel benutzt werden.

Die Option stellt nur die gebrauchte kryptografische Funktionalität bereit. In bench werden keine Zugangsdaten registriert und kein Handshake ausgeführt. Die Verwendung eines PSK erfolgt erst in `iot_sensor`, wie in Kapitel 6.4.2 beschrieben.

`CONFIG_MBEDTLS_PSK_MAX_LEN=32`

Diese Option begrenzt die maximale Länge eines Pre-Shared Keys auf 32 Byte. Sie definiert damit eine Kapazitätsgrenze für PSK-basierte Verbindungen, erzeugt aber selbst keinen Schlüssel.

Der in der Demonstrationsanwendung benutzte PSK muss innerhalb dieser Grenze liegen. `MBEDTLS_PSK_MAX_LEN` ist der Symbolname für die benutzte Zephyr-/ mbedTLS-Version.

Devicetree-Overlay für die Entropiequelle

Zum `iot_stack.conf` wird zudem `iot_stack.overlay` eingebunden:

```
/ {
    chosen {
        zephyr,entropy = &rng;
    };
};
```

Das Overlay weist Zephyr den Hardware-Zufallszahlengenerator rng als Entropiequelle zu. OpenThread und die kryptografischen Funktionen brauchen dabei Zufallswerte, bspw. für Protokoll- und Sicherheitsoperationen. Die Entropiequelle wird daher über Devicetree festgelegt, weil sie eine Zuordnung zu einem bestimmten Hardwaregerät beschreibt. Sie ist deshalb nicht als weitere CONFIG_-Option in `iot_stack.conf` enthalten.

Das Overlay wird nur für die IoT-Stack-Konfiguration übergeben. Minimal- und Logging-Konfiguration werde dadurch nicht von dieser Hardwarezuordnung beeinflusst.

6.4 Realistische Anwendung

Für diese Arbeit wurde der Quellcode von einer praxisnahen Demonstrationsanwendung entwickelt. Nach einem erfolgreichem Build, Flash- und Ende-zu-Ende-Test soll sie einen über DTLS geschützten CoAP-Sensordienst und einen Firmware-Update-Pfad demonstrieren.

6.4.1 Zweck und Sensor-Simulation

Für diese Arbeit wurde kein externer Sensor an das nRF52840 DK angeschlossen, um die Reproduzierbarkeit nicht von weiterer, nicht mitgelieferter Hardware zu beeinflussen. Daher wird der interne Die-Temperatursensor des nRF52840 DK ausgelesen. Dieser liefert die aktuelle Chiptemperatur, die dann anschließend über den CoAP-Server zur Verfügung gestellt wird. Die für OpenThread gebrauchte Entropiequelle wird dabei über ein board-spezifisches Devicetree-Overlay umgestellt. Dies erfolgt nach demselben Prinzip wie bei der in 6.3.4 beschriebenen IoT-Stack-Konfigurationsstufe.

6.4.2 CoAP-Server über DTLS/ PSK

Die Datei `coap_server.c` implementiert hierbei einen CoAP-Server, der über Zephyrs BSD-Socket-Schnittstelle und die in Zephyr enthaltene CoAP-Paketbibliothek genau eine Ressource zur Verfügung stellt. Dabei handelt es sich um eine GET-Anfrage auf dem Pfad `/sensor`, welche den aktuellen Temperaturwert als Textnutzlast zurückliefert. Die Absicherung des Servers folgt dabei genau dem in Abschnitt 2.7.2 erwähntem Verfahren. Anstatt dass ein UDP-Socket benutzt wird, wird beim Anlegen des Sockets das Protokoll `IPPROTO_DTLS_1_2` angegeben, sodass sich der Socket nach außen wie ein normaler Datagramm-Socket benutzen lässt (`bind()`, `recvfrom()`, `sendto()`). Der TLS-Handshake erfolgt jedoch intern über mbedTLS. Bevor dieser Socket jedoch angelegt wird, wird ein Pre-Shared Key und eine dazugehörige PSK-Identität über die Funktion `tls_credential_add()` im TLS-Credential-Subsystem registriert. Anschließend wird es dem Socket dann über `setsockopt()` mit dem Optionsnamen `TLS_SEC_TAG_LIST` zugewiesen. Darüber hinaus

wird der Socket über die Option `TLS_DTLS_ROLE` als Server konfiguriert. Der dabei benutzte Pre-Shared Key ist hierbei ein fest hinterlegter Demo-Schlüssel im Quellcode, da es sich hierbei nur um Testzwecke handelt. In einer echten Anwendung würde ein solcher Schlüssel bei einer Geräteinbetriebnahme (Provisioning) individuell erzeugt werden.

6.4.3 Firmware-Update über Sysbuild, MCUboot und MCUmgr

Damit diese Anwendung, wie in Abschnitt 2.7.4 beschrieben, auch Firmware-Updates unterstützt, wird sie über das Sysbuild-System gebaut. Dieses erzeugt aus dem eigentlichen Anwendungscode und einem weiteren MCUboot-Bootloader-Image ein gemeinsames Firmware-Abbild. Das Anwendungs-Image selbst wird hierbei dafür passend für die Flash-Partitionen gebaut und signiert, die von MCUboot verwaltet werden. Auf der Anwendungsseite wird zudem das MCUmgr-Management-Subsystem eingebunden, darunter die Bildverwaltungs- und die Betriebssystem-Kommandogruppe. Darüber kann ein neues Firmware-Abbild empfangen und anschließend ein Neustart ausgelöst werden. Für dieses Management-Protokoll wird hierbei als Übertragungsweg UDP benutzt. Dadurch kann ein Firmware-Update über denselben Thread-Netzwerkpfad übertragen werden, über den auch der CoAP-Sensordienst erreichbar ist.

Messergebnisse

Diskussion

Validität

9.1 Interne Validität

9.2 Externe Validität

Bei der externen Validität von einer Untersuchung handelt es sich eine Beschreibung davon, wie weit sich die Ergebnisse auf andere Bedingungen als die untersuchten, übertragen lassen. Dabei ergeben sich für diese Arbeit mehrere Einschränkungen der Übertragbarkeit.

Erstens wurden alle Messungen auf genau einer Hardware-Plattform durchgeführt. Dabei handelt es sich um den nRF52840 DK mit einem 32-Bit-Arm-Cortex-M4F-Prozessor bei 64 MHz Taktfrequenz (siehe 4.2). Andere Mikrocontroller-Architekturen, wie bspw. Cortex-M0+, dem Cortex-M7 oder RISC-V-basierte Systeme haben andere Pipeline-Tiefen, Speicherzugriffszeiten und Interrupt-Controller-Architekturen. Die absoluten Zyklenzahlen dieser Arbeit können daher nicht direkt auf andere Hardware übertragen werden. Ob sich dabei die relativen Unterschiede zwischen den Konfigurationsstufen auch auf andere Hardware übertragen lassen, wurde nicht untersucht.

Zweitens wurde nur Zephyr in der Version v3.7.2 (LTS) mit dem Zephyr SDK 0.16.9 (GCC 12.2.0) untersucht (siehe 4.6.2, 4.6.4). Der Kernel und auch einzelne APIs können sich zwischen den Zephyr-Versionen ändern.

Drittens wurden nur drei Konfigurationsstufen aus dem größeren Kconfig-Raum von Zephyr untersucht (siehe 4.4). Andere Funktionskombinationen, wie z.B. Bluetooth, USB, Dateisysteme, Userspace/ MMU-Isolation, wurden nicht gemessen und ihre Kosten lassen sich auch nicht aus den ergebnen Daten ableiten.

Viertens wurde als praxisnahes Anwendungsbeispiel nur ein IoT-Sensor-Szenario über Thread, CoAP und DTLS untersucht (siehe 6.4). Andere verbreitete IoT-Kommunikationsstacks, wie bspw. Bluetooth Low Energy, Wi-Fi oder Matter über Wi-Fi/ Ethernet, wurden nicht untersucht und können daher andere Ressourcenkosten aufweisen.

In Kapitel 8.4 wurde eine Einordnung dieser Arbeit gegen Silva et al. (2019) durchgeführt, welche einen externen Referenzpunkt bringt. Wenn die eigenen Fußabdruck-Werte mit denen einer unabhängigen Studie übereinstimmen oder sich ähneln, dann wird das Vertrauen in die Übertragbarkeit der Methodik dadurch verstärkt. Es ersetzt jedoch auf der anderen Seite keine eigene Messung auf anderer Hardware oder mit anderen Zephyr-Versionen.

9.3 Messbedingter Zusatzaufwand

Jede Zeitmessung an einem laufenden System beeinflusst das gemessene Ergebnis durch ihre eigene Durchführung. Dieser Effekt wird Messaufwand genannt. Der Messaufwand in der Messmethodik, die für diese Arbeit entwickelt wurde, ist vor allem das Auslesen des DWT-Zyklenzählers (siehe 6.2.3). Es beansprucht nämlich vor und nach dem zu messenden Vorgang einen kleinen Zeitanteil.

Um diesen Anteil aus den Ergebnissen rauszurechnen, wird der Messaufwand vor jeder Messreihe kalibriert. Der Zyklenzähler wird 100-mal direkt hintereinander ausgelesen, ohne dass dazwischen irgendwas gemessen wird. Dann wird das Minimum aller 100 Differenzen als Kalibrierungswert genommen. Dabei ist bewusst das Minimum ausgesucht worden. Störeinflüsse können nämlich eine einzelne Kalibrierungsmessung nur verlängern, nicht verkürzen. Dadurch wird der echte Grundaufwand durch das Minimum so gut wie möglich angenähert.

Dieser kalibrierte Wert wird dann anschließend von jedem einzelnen rohen Messwert abgezogen, bevor er in die laufende Statistik (siehe 6.2.4) einfließt. In den Messungen lag der kalibrierte Messaufwand bei 12 Zyklen. In den dokumentierten Kalibrierungen wurde derselbe Kalibrierungswert beobachtet. Daraus folgt keine allgemeine Unabhängigkeit des Messaufwands von der Konfiguration.

Die Kalibrierung erfasst jedoch nur den Zeitaufwand des Zähler-Auslesens selbst. Daher werden nicht andere mögliche Seiteneffekte, die durch den Code entstehen können, wie bspw. veränderte Flash-Adressen von benachbarten Funktionen erfasst (siehe 9.1). Wie stark solche Änderungen des Flash-Layouts die einzelnen Messwerte beeinflussen, wurde in dieser Untersuchung nicht separat gemessen. Sie bleiben daher eine mögliche Quelle von systematischen Abweichungen zwischen den Firmware-Builds.

Der Selbsttest (siehe Kapitel 5) ist eine weitere Kontrolle für die Nachvollziehbarkeit der gemeinsam benutzten Zeitmessungs- und Auswertungskette. Er prüft die gemeinsame Wirkung von Zeitbasis, Abzug des Messaufwands und Auswertung anhand einer vorgegebenen Busy-Wait-Dauer auf Nachvollziehbarkeit. Die einzelnen Teile werden dadurch nicht unabhängig voneinander validiert. Das hierbei definierte Abnahmekriterium ist in allen Selbsttests durchgeführt und erfüllt worden.

9.4 Limitierungen

Fazit

Ausblick